

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
МИРЭА – РОССИЙСКИЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ

А.В. ПУГАЧЕВ, К.Ю. ДРУЖИНИН, В.П. ЗЯЗИН

ВВЕДЕНИЕ В ОПЕРАЦИОННЫЕ СИСТЕМЫ

ЧАСТЬ I

УЧЕБНО-МЕТОДИЧЕСКОЕ ПОСОБИЕ

МОСКВА – 2019

УДК 004.451
ББК 32.972.11
П88

Печатается по решению редакционно-издательского совета РТУ МИРЭА

Рецензенты

Григорьев В.Р., канд. техн. наук, чл.-кор. РАЕН, заведующей кафедрой КБ-8 «Информационное противоборство», зам. директора Института комплексной безопасности и специального приборостроения;

Нестеренко А.Ю., канд. физ.-мат. наук, доцент кафедры «Компьютерная безопасность» МИЭМ НИУ ВШЭ

П88 **Пугачев А.В.**

Введение в операционные системы. Часть I : учебно-методическое пособие / А.В. Пугачев, К.Ю. Дружинин, В.П. Зязин. – М.: МИРЭА – Российский технологический университет, 2019. – 122 с.

ISBN 978-5-7339-1509-8

В пособии рассматриваются особенности архитектуры операционных систем, создающие предпосылки для успешного проведения компьютерных атак, в т.ч. внедрения вредоносного программного обеспечения. Основная цель данного учебного пособия состоит в формировании и развитии базовых компетенций специалистов ведомственных и корпоративных центров Государственной системы обнаружения, предупреждения и ликвидации последствий компьютерных атак на информационные ресурсы Российской Федерации (ГосСОПКА).

Пособие ориентировано на специальность 10.05.01 «Компьютерная безопасность», но может быть полезным для всех, интересующихся проблемами информационной безопасности.

Пособие включает примеры компьютерных атак из новейшей истории, проведению которых способствовали рассматриваемые в настоящем пособии особенности архитектуры операционных систем.

УДК 004.451
ББК 32.972.11

ISBN 978-5-7339-1509-8

© Пугачев А.В., Дружинин К.Ю., Зязин В.П., 2019
© МИРЭА – Российский технологический университет, 2019

СОДЕРЖАНИЕ

ПРЕДИСЛОВИЕ	6
1. ОПЕРАЦИОННЫЕ СИСТЕМЫ КАК ОБЪЕКТ ИССЛЕДОВАНИЯ	8
1.1. Пример угрозы	8
1.2. История развития операционных систем	9
1.3. Классификация операционных систем	11
1.4. Критерии сравнения операционных систем	13
1.5. Вопросы для самопроверки	13
2. АРХИТЕКТУРА ОПЕРАЦИОННОЙ СИСТЕМЫ	14
2.1. Программные модули	14
2.2. Взаимосвязь программных модулей	16
2.2.1. Интерфейсы взаимодействия программных модулей	16
2.2.2. Связывание программных модулей	17
2.3. Основные подсистемы операционных систем	21
2.4. Загрузка ЭВМ	25
2.4.1. Общие этапы	25
2.4.2. Основные возможности центрального процессора	26
2.5. Виртуализация	28
2.5.1. Уровни виртуализации	28
2.5.2. Виртуализация уровня ОС	29
2.5.3. Типы гипервизоров	30
2.5.4. Угрозы информационной безопасности, связанные с виртуализацией	31
2.6. Вопросы для самопроверки	32
3. ПРОЦЕССЫ И ПОТОКИ	33
3.1. Определения	33
3.2. Особенности разбиения адресного пространства	35
3.3. Область стека	36
3.4. Модели памяти процесса	40
3.5. Многозадачность	44
3.6. Потоки	45
3.7. Жизненный цикл процесса (потока)	48
3.7.1. Общая модель	48
3.7.2. Жизненный цикл в Linux	49
3.8. Вопросы для самопроверки	50
4. ОПЕРАЦИИ НАД ПРОЦЕССАМИ И ПОТОКАМИ. ВЗАИМОИСКЛЮЧЕНИЯ	51
4.1. Создание процессов и потоков	51
4.2. Механизмы передачи данных и оповещения о событиях	54
4.2.1. Файлы	54
4.2.2. Каналы	55
4.2.3. Сокеты	57

4.2.4. Сигналы	57
4.2.5. Сообщения	58
4.2.6. Разделяемая память	59
4.2.7. Почтовый ящик	59
4.3. Механизмы взаимoisключения	60
4.3.1. Проблема асинхронного выполнения задач	60
4.3.2. Алгоритмические взаимoisключения	63
4.3.3. Аппаратные взаимoisключения	64
4.3.4. Программные взаимoisключения	65
4.3.5. Решение задачи «потребитель-производитель»	67
4.3.6. Монитор	68
4.4. Вопросы для самопроверки	70
5. ВЗАИМОБЛОКИРОВКИ	71
5.1. Введение	71
5.2. Предотвращение взаимоблокировок	73
5.2.1. Общий подход	73
5.2.2. Отказ от взаимoisключений	73
5.2.3. Ожидание дополнительных ресурсов	74
5.2.4. Неперераспределяемость занятых (ожидаемых) ресурсов	75
5.2.5. Круговое ожидание	75
5.2.6. Задача об обедающих философaх	76
5.3. Обход взаимоблокировок	78
5.4. Обнаружение взаимоблокировок	80
5.5. Восстановление после взаимоблокировок	82
5.5.1. Общий подход	82
5.5.2. Приоритетное овладение ресурсами	82
5.5.3. Использование контрольных точек	83
5.5.4. Уничтожение процессов	84
5.6. Вопросы для самопроверки	84
6. ПЛАНИРОВАНИЕ ПРОЦЕССОВ	85
6.1. Основные определения и задачи планирования	85
6.2. Уровни планирования	86
6.3. Планирование в системах пакетной обработки	87
6.3.1. Очередь (FIFO — first input first output)	87
6.3.2. Кратчайший процесс — первый (SPF — shortest process first)	88
6.4. Вытеснение процессов	90
6.5. Планирование в интерактивных системах	91
6.5.1. Циклическая очередь (RR — round robin)	91
6.5.2. Наименьшее оставшееся время (SRT — shortest remaining time)	91
6.5.3. Эгоистическое циклическое планирование (SRR — selfish round robin)	91
6.5.4. Относительное время реакции (HRRN — highest response ratio next)	92

6.5.5. Многоуровневая очередь с обратной связью (<i>MFQ – multilevel feedback queue</i>).....	92
6.5.6. Планировщик <i>Linux</i> («О большое от единицы»)	93
6.6. Планирование в системах реального времени	94
6.7. Вопросы для самопроверки	96
7. ОПЕРАТИВНАЯ ПАМЯТЬ	97
7.1. Иерархия памяти	97
7.2. Диспетчер оперативной памяти.....	99
7.3. Подходы к разбиению памяти.....	100
7.4. Проблемные вопросы распределения памяти	102
7.5. Стратегии загрузки данных в память	104
7.6. Стратегии размещения данных в памяти.....	106
7.7. Вопросы для самопроверки	107
8. ВИРТУАЛЬНАЯ ПАМЯТЬ	108
8.1. Архитектура виртуальной памяти	108
8.2. Типы адресации в виртуальной памяти	110
8.3. Подкачка.....	113
8.3.1. Назначение механизма.....	113
8.3.2. Алгоритм работы механизма подкачки	114
8.3.3. Стратегии замены страниц	115
8.3.4. Стратегии восстановления выгруженных областей	116
8.4. Вопросы для самопроверки	118
СПИСОК ЛИТЕРАТУРЫ	119

Тот военачальник искусен в нападении, чей противник не знает, что ему защищать;
и тот искусен в обороне, чей противник не знает, на что ему напасть.

Сунь-Цзы, V в. до н.э.

ПРЕДИСЛОВИЕ

В пояснительной записке к тексту федерального закона «О безопасности критической информационной инфраструктуры Российской Федерации» от 26 июля 2017 г. № 187-ФЗ отмечено, что осуществляемый в настоящее время в Российской Федерации переход к информационному обществу приводит к тому, что подавляющее большинство систем принятия решений и бизнес-процессов в ключевых отраслях экономики и сфере государственного управления реализуются или планируются к реализации с использованием информационных технологий. В различных информационных системах уже сейчас хранятся и обрабатываются значительные объемы информации, в том числе касающейся вопросов государственной политики и обороны, финансовой и научно-технической сфер, частной жизни граждан.

На этом фоне объекты критической информационной инфраструктуры Российской Федерации постоянно подвергаются компьютерным атакам. Отмечается постоянное усложнение используемых злоумышленниками средств и методов их проведения.

В этой связи, обеспечение информационной безопасности, в частности обеспечение единства, устойчивости и безопасности информационно-телекоммуникационной инфраструктуры Российской Федерации на всех уровнях информационного пространства, отнесено к 5 базовым направлениям развития цифровой экономики в Российской Федерации на период до 2024 года¹.

Принципиально важным элементом системы обеспечения информационной безопасности Российской Федерации стал вышеупомянутый Федеральный закон № 187-ФЗ, который определил центральное место Государственной системы обнаружения, предупреждения и ликвидации последствий компьютерных атак на информационные ресурсы Российской Федерации (ГосСОПКА, создается во исполнение Указа Президента Российской Федерации от 15 января 2013 г. № 31с «О создании государственной системы обнаружения, предупреждения и ликвидации последствий компьютерных атак на информационные ресурсы Российской Федерации») в системе обеспечения безопасности критической информационной инфраструктуры Российской Федерации.

¹Программа «Цифровая экономика Российской Федерации». Утверждена распоряжением Правительства Российской Федерации от 28 июля 2017 г. № 1632-р.

Одной из основных задач обеспечения информационной безопасности Российской Федерации является подготовка квалифицированных специалистов ведомственных и корпоративных центров ГосСОПКА. Одной из составляющих фундамента их знаний и навыков должно являться понимание особенностей архитектуры операционных систем, создающих предпосылки для успешного проведения компьютерных атак, в т.ч. внедрения вредоносного программного обеспечения.

Существующая в настоящее время учебная и учебно-методическая литература по предмету «Операционные системы» ориентирована либо на формирование общего представления об их структуре, либо на формирование эксплуатационных навыков в рамках некоторого конкретного класса операционных систем.

Цель настоящего пособия состоит в формировании и развитии базовых компетенций специалистов ведомственных и корпоративных центров ГосСОПКА. Кроме того, освоение материала, изложенного в пособии, способствует формированию и развитию компетенций, которые будут необходимы при изучении студентами материалов по следующим курсам: «Модели безопасности компьютерных систем», «Защита в операционных системах», «Анализ уязвимостей программного обеспечения», «Разрушающие программные воздействия», «Системы обнаружения компьютерных атак и вторжений», «Основы построения защищённых компьютерных сетей», «Предупреждение, выявление и установление причин и условий компьютерных инцидентов», входящих в программу обучения по специальности 10.05.01 «Компьютерная безопасность».

Для успешного изучения материалов настоящего курса необходимы базовые знания языков программирования Си и Ассемблер, преподаваемых в рамках курса «Языки программирования».

1. ОПЕРАЦИОННЫЕ СИСТЕМЫ КАК ОБЪЕКТ ИССЛЕДОВАНИЯ

1.1. Пример угрозы

Общеизвестный факт: вопросы компьютерной безопасности неразрывно связаны с вопросами безопасности применяемого программного обеспечения (далее — ПО). Этой проблематике посвящено множество книг и статей в различных журналах и блогах. Но вот парадокс: трудно разработать безопасное (защищенное) ПО, если это окажется невыгодным «третьей стороне», создающей окружение для работы этого ПО.

Рассмотрим метаморфозы ПО в процессе его разработки на простом примере. В листинге 1 приведен код программы, подобные которой писал, пожалуй, каждый человек, изучавший язык программирования Си [1].

Листинг 1. Простая программа

```
1 #include <stdio.h>
2
3 int main(int argc, char* argv[])
4 {
5     printf("Hello, \World!\n");
6     return 0;
7 }
```

После компиляции предложенного кода с использованием Microsoft Visual Studio и запуска полученного ПО с использованием программных средств трассировки в операционной системе (далее – ОС) Windows 8 можно получить отчёт о её выполнении, часть которого приведена на рис. 1.1.

1146	7:41:40.689 PM	1	KERNEL32.DLL	RtlSetLastWin32Error (ERROR_SUCCESS)
1147	7:41:40.689 PM	1	hello_world.exe	printf ("Hello, world!" , ...)
1148	7:41:40.689 PM	1	MSVCR100D.dll	EnterCriticalSection (0x00000000676fcd8)
1149	7:41:40.689 PM	1	MSVCR100D.dll	EnterCriticalSection (0x00000000676fcc38)
1150	7:41:40.689 PM	1	MSVCR100D.dll	HeapAlloc (0x0000000000ff0000, 0, 4148)
1151	7:41:40.689 PM	1	MSVCR100D.dll	LeaveCriticalSection (0x00000000676fcc38)
1152	7:41:40.692 PM	1	MSVCR100D.dll	GetLastError ()
1153	7:41:40.692 PM	1	MSVCR100D.dll	FlsGetValue (4)
1154	7:41:40.692 PM	1	MSVCR100D.dll	SetLastError (ERROR_SUCCESS)
1155	7:41:40.692 PM	1	KERNEL32.DLL	RtlSetLastWin32Error (ERROR_SUCCESS)
1156	7:41:40.692 PM	1	MSVCR100D.dll	EnterCriticalSection (0x0000000000ff0c18)
1157	7:41:40.692 PM	1	MSVCR100D.dll	GetLastError ()
1158	7:41:40.692 PM	1	MSVCR100D.dll	FlsGetValue (4)
1159	7:41:40.692 PM	1	MSVCR100D.dll	SetLastError (ERROR_SUCCESS)
1160	7:41:40.693 PM	1	KERNEL32.DLL	RtlSetLastWin32Error (ERROR_SUCCESS)
1161	7:41:40.693 PM	1	MSVCR100D.dll	GetConsoleMode (0x0000000000000020, 0x0000000000e9bc44)
1162	7:41:40.693 PM	1	KERNELBASE.dll	NtDeviceIoControlFile (0x0000000000000010, NULL, NULL, NULL,
1163	7:41:40.693 PM	1	MSVCR100D.dll	WriteFile (0x0000000000000020, 0x0000000000e9bc80, 15, 0x0000...
1164	7:41:40.693 PM	1	KERNELBASE.dll	NtWriteFile (0x0000000000000020, NULL, NULL, NULL, 0x0000...
1165	7:41:40.693 PM	1	MSVCR100D.dll	LeaveCriticalSection (0x0000000000ff0c18)
1166	7:41:40.693 PM	1	MSVCR100D.dll	LeaveCriticalSection (0x00000000676fcd8)

Рисунок 1.1. Трассировка программы в Windows

Фрагмент отчета показывает последовательность вызова процедур с 1146 по 1166. При этом вызывается единственная процедура из листинга 1 – `printf` (под № 1147), что влечет за собой вызов еще 18 процедур.

```

1: execve("./prog.exe", ["./prog.exe"], 0x7ffdbcd84660 /* 44 vars */) = 0
2: strace: [ Process PID=2859 runs in 32 bit mode. ]
3: brk(NULL) = 0x56868000
4: access("/etc/ld.so.nohwcap", F_OK) = -1 ENOENT (No such file or directory)
5: mmap2(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xf7f39000
6: access("/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
7: open("/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
8: fstat64(3, {st_mode=S_IFREG|0644, st_size=174275, ...}) = 0
9: mmap2(NULL, 174275, PROT_READ, MAP_PRIVATE, 3, 0) = 0xf7f0e000
10: close(3) = 0
11: access("/etc/ld.so.nohwcap", F_OK) = -1 ENOENT (No such file or directory)
12: open("/lib32/libc.so.6", O_RDONLY|O_CLOEXEC) = 3
13: read(3, "\177ELF\1\1\1\3\0\0\0\0\0\0\0\3\0\3\0\1\0\0\0\320\205\1\0004\0\0\0"... , 512) = 512
14: fstat64(3, {st_mode=S_IFREG|0755, st_size=1816488, ...}) = 0
15: mmap2(NULL, 1825276, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_DENYWRITE, 3, 0) = 0xf7d50000
16: mmap2(0xf7f08000, 12288, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x1b7000) = 0xf7f08000
17: mmap2(0xf7f0b000, 10748, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xf7f0b000
18: close(3) = 0
19: set_thread_area({...}) = 0 (entry_number:12)
20: mprotect(0xf7f08000, 8192, PROT_READ) = 0
21: mprotect(0x565ae000, 4096, PROT_READ) = 0
22: mprotect(0xf7f63000, 4096, PROT_READ) = 0
23: munmap(0xf7f0e000, 174275) = 0
24: fstat64(1, {st_mode=S_IFCHR|0620, st_rdev=makedev(136, 1), ...}) = 0
25: brk(NULL) = 0x56868000
26: brk(0x56889000) = 0x56889000
27: write(1, "Hello, World!\n", 14) = 14
28: exit_group(0) = ?
29: +++ exited with 0 +++

```

Рисунок 1.2. Трассировка программы в Linux

Результат аналогичного эксперимента для операционной системы GNU/Linux приведен на рис. 1.2. Стоит отметить, что в полученном отчете функция `printf` отсутствует, а её роль выполняет функция `write`.

При рассмотрении полученных результатов могут возникнуть два вполне логичных вопроса: «Что делают все эти процедуры?» и «Могут ли они быть потенциально опасными?». Процедуры используются для взаимодействия ПО с операционной системой (далее – ОС), в рамках которой они функционируют. И, без сомнения, они могут быть потенциально опасны.

Важно понимать, что это не единственное воздействие на ПО, которое может оказывать ОС. Именно об этом влиянии пойдет речь в курсе «Операционные системы».

1.2. История развития операционных систем

Рассмотрим развитие ОС с точки зрения основных этапов их технической эволюции, без привязки к конкретным временным промежуткам.

Используя этот подход, всю историю ОС разделяют на 7 этапов [2]:

1. «Открытое предприятие» (Open Shop).
2. Системы пакетной обработки (Batch Processing).
3. Многозадачные системы (Multiprogramming).
4. Системы с разделением времени (Timesharing).

5. Параллельное программирование (Concurrent Programming).
6. Персональный компьютер (Personal Computer).
7. Распределенные системы (Distributed Systems).

Не сложно понять, что каждый новый этап развития ОС приводил к усложнению структуры ОС и увеличению числа решаемых ей задач.

«Открытое предприятие» – это название раннего этапа развития вычислительных систем (далее – ВС), в котором не существовало понятия ОС. Разработчик-программист получал монопольный доступ к ВС и тратил примерно 2/3 своего времени на настройку оборудования и только 1/3 – на вычисление. По этой причине, финансовые затраты, связанные с простым ЭВМ, оцениваются в 146,000 \$ в месяц. Это не удивительно, например, один день суперкомпьютера «Ломоносов» (МГУ) стоит \$20 000, а один день суперкомпьютера «Titan» (ORNL) стоит \$300 000[3].

Системы пакетной обработки стали решением проблемы неоптимального использования ресурсов. Замысел состоял в том, чтобы избавить разработчика от необходимости самостоятельной настройки ВС для решения конкретной задачи. Для оптимизации вычислительного процесса была разработана специальная программа-монитор, которая, по сути, и стала первой ОС. Она автоматически считывала задание, загруженное оператором ВС в специальную очередь, запускала его выполнение, распечатывала результаты выполнения и сразу приступала к выполнению следующего задания. Программисту оставалось только подготовить задание, которое формировалось в виде «пакета», который содержал набор команд и данных, и передать его в вычислительный центр.

Следующий виток развития ОС связан с появлением **многозадачных систем**, где стало возможным давать каждой задаче свой вычислитель. Это связано в первую очередь с качественным скачком в развитии аппаратного обеспечения, что привело как к удешевлению компонентной базы, так и к уменьшению ее габаритов. В связи с этим в ВС появилась возможность реализовывать несколько вычислительных модулей, а также блоков памяти. Таким образом, каждый вычислитель использовался для выполнения отдельных задач.

Практически в тот же период была предложена новая концепция выполнения задач, которая заключалась в том, что ВС могла начать выполнение новой задачи, пока текущая задача ожидала ответа от устройства ввода-вывода. Системы, реализующие подобный подход, назывались **системами с разделением времени**.

Рост числа одновременно выполняемых задач на фоне дефицита вычислительных ресурсов привел к появлению проблемы бесконечного ожидания ресурсов. В связи с этим проводились различные исследования, направленные на

предотвращение подобных ситуаций, результатом которых стало формирование фундаментальных принципов **параллельного программирования**.

Следующий этап развития ОС был связан с изменением приоритетов и переориентацией ряда компаний на работу с обычным пользователем, положив тем самым начало эры **персональных компьютеров**. ОС стали получать более «дружелюбный» (по отношению к пользователю) интерфейс. Одновременно с этим произошло удешевление ЭВМ за счет упрощения конструкции и удешевления элементной базы. Именно в этот период, благодаря проведенным в НИИ Стэнфорда исследованиям Дагома Энгельбарта, появляется графический интерфейс пользователя (GUI), идею которого переняли разработчики компании Xerox PARC для производимых ими ЭВМ. Немногим позже ценность GUI осознал один из основателей компании Apple Стив Джобс, далее его примеру последовала корпорация Microsoft.

Последний этап в развитии ОС связан с появлением новых технологий передачи данных: создание «Агентством передовых исследовательских проектов» (ARPA) компьютерной сети ARPANET, а также описание и разработка технологии Ethernet. Это позволило разработчикам ВС создать новый класс процедур, вызов которых осуществлялся удаленно, что привело к появлению нового класса ресурсов, получивших название «удаленные ресурсы», и, как следствие, нового класса систем, использующих эти ресурсы. Новые системы получили название **распределенных**.

Итак, определим понятие ОС.

Операционная система — набор программ (программных модулей), позволяющих пользователю вычислительной системы решать собственные задачи и (или) производить запуск и обеспечивать работоспособность стороннего программного обеспечения (программных модулей).

1.3. Классификация операционных систем

Очевидно, что каждый человек по-своему видит набор необходимого программного обеспечения и, следовательно, саму ОС. Чтобы каким-то образом охарактеризовать множество подобных наборов, рассмотрим следующие варианты классификации операционных систем.

1. По назначению.
2. По способу взаимодействия с пользователем.
3. По числу одновременно выполняемых задач.
4. По количеству пользователей.
5. По поддерживаемой аппаратной архитектуре.
6. По уровню переносимости.

По назначению операционные системы можно разделить на:

- **общего назначения** — подходят для решения большого круга разнообразных задач;
- **реального времени** — обеспечивают реакцию на какое-либо событие в течение жестко ограниченного отрезка времени;
- **нежесткого реального времени** — в отличие от систем реального времени, допускают отклонение от границ заданного временного промежутка;
- **специальные** — разрабатываются под конкретную задачу и аппаратуру.

По способу взаимодействия с пользователем выделяют следующие типы:

- **пакетные** — занимаются решением задач, подающихся на вход в виде пакетов;
- **интерактивные** — работают во взаимодействии с пользователем. Существуют два вида интерактивных ОС:
 - **диалоговая** — система, работающая по принципу «вопрос-ответ». Например, система, имеющая командную строку;
 - **с графическим пользовательским интерфейсом (GUI)** — система, ориентированная на использование манипуляторов типа мышь.
- **встроенные** — не имеющие интерфейса непосредственного взаимодействия с пользователем. Взаимодействие с такими системами может происходить за счет включения или выключения какого-либо устройства или с использованием дополнительного программного обеспечения. Например, система, установленная в современных телевизорах.

По числу одновременно выполняемых задач, а также числу пользователей, ОС классифицируются исходя из возможности разграничения доступа к ресурсам между задачами (пользователями). Выделяют:

- **однозадачные (однопользовательские)**;
- **многозадачные (многопользовательские)**.

По поддерживаемой аппаратной архитектуре системы подразделяются на:

- **однопроцессорные**.
- **многопроцессорные**.
- **распределенные**, то есть ОС, функционирующие на нескольких физически разных аппаратных составляющих, например, на нескольких ПК.

По уровню переносимости ОС классифицируются исходя из возможности системы функционировать на различных архитектурах процессоров (Intel, ARM, Sparc). Выделяют следующие классы:

- переносимые;
- непереносимые.

1.4. Критерии сравнения операционных систем

Для того, чтобы определиться с тем, какая ОС нужна в данный момент, могут быть использованы следующие критерии их сравнения.

1. Надежность.
2. Эффективность.
3. Удобство.
4. Масштабируемость.
5. Способность к развитию.
6. Совместимость.
7. Стоимость использования:
 - стоимость получения ОС;
 - совокупная стоимость лицензий на использование ОС;
 - зарплата штату специалистов, обеспечивающих работоспособность ОС;
 - стоимость технической поддержки сторонними специалистами;
 - стоимость аппаратных средств, необходимых для нормального функционирования ОС.

1.5. Вопросы для самопроверки

1. Приведите и объясните примеры ОС по четырем «назначениям» (общего, реального времени, нежесткого реального времени, специальных).
2. Может ли одна и та же ОС относиться к нескольким типам?
3. Оцените такие ОС, как Linux, Windows, Mac OS, воспользовавшись критериями сравнения.
4. Приведите примеры однозадачных и однопроцессорных систем.

2. АРХИТЕКТУРА ОПЕРАЦИОННОЙ СИСТЕМЫ

2.1. Программные модули

Поскольку ОС – это набор программных модулей, справедливо начать с описания сущности «программных модулей».

Программный модуль – программа или функционально завершенный фрагмент программы, предназначенный для хранения, трансляции, объединения с другими программными модулями и загрузки в оперативную память [4].

Исходя из определения, программные модули можно разделить по следующим основаниям.

1. По возможности исполнения.
2. По способу исполнения.

По возможности исполнения программные модули делятся на:

1. **Исполняемые** – программные модули, предназначенные для непосредственного исполнения.
2. **Неисполняемые** – программные модули, не предназначенные для исполнения, но содержащие процедуры, функции и данные, используемые исполняемыми программными модулями при выполнении.
3. **Гибридные** – программные модули, которые в зависимости от способа использования могут использоваться как *исполняемые* либо *неисполняемые*.

Основным критерием, который определяет возможность исполнения либо неисполнения программного модуля, является возможность перемещения в памяти программного модуля, а также наличие соответствующей «точка входа» в программный модуль.

Точка входа — адрес процедуры или функции, по которому передается управление в первую очередь в программном модуле.

Следует отметить, что точка входа, определяемая спецификациями языков программирования, не всегда совпадает с реальной точкой входа в программный модуль. Так, в соответствии со стандартом [5], точкой входа в программный модуль, написанный на языке программирования Си, является процедура `main`. На самом деле это не так. Дело в том, что код, который пишет программист, преобразуется в объектный модуль², который в дальнейшем связывается с иными

²Объектный модуль – программный модуль, получаемый в результате компиляции исходного модуля [4].

программными модулями, предусмотренными при сборке стандартом. Именно в этих модулях и находится реальная точка входа в исполняемый программный модуль.

Условность «точки входа» легко показать на примере гибридного программного модуля и привести код, написанный на языке Python (листинг 2).

Листинг 2. Гибридный программный модуль

```
1 #!/usr/bin/python
2 def func():
3     print "Hello, world!"
4
5 if __name__ == '__main__':
6     func()
```

В случае непосредственного выполнения приведенного программного модуля, переменная `__name__` будет содержать строковую константу `'__main__'`. В случае подключения данного модуля в код иного программного модуля посредством директивы `import`, переменная `__name__` имеет иное значение, а, следовательно, код в строке 6 не будет выполнен.

По способу выполнения программные модули делятся на следующие классы [6].

1. **Непосредственно исполняемый** – программный модуль, хранящийся в формате, пригодном для выполнения центральным процессором (PE, MZ, ELF), т.е. в машинном коде.
2. **Транслируемый** – программный модуль, преобразованный в промежуточный формат (байт-код), в дальнейшем преобразуемый в машинный код специальной программой – транслятором (Java, DotNET).
3. **Интерпретируемый** – программный модуль, хранимый в виде исходного кода и содержащий инструкции для выполнения специализированной программой – интерпретатором (Python, Bash).

Следует отметить, что непосредственно исполняемые программные модули, как правило, обладают более высокими скоростными характеристиками. Однако, модули указанного класса менее переносимы. Они не способны функционировать на вычислительных системах, центральный процессор которых не поддерживает набор команд, заложенный в модуле.

Транслируемые и интерпретируемые программные модули, напротив, обладают более высокой степенью переносимости, однако, проигрывают в производительности. Основная разница между транслируемыми и интерпретируемыми программными модулями заключается в том, что интерпретируемые программные модули полностью ограничены набором инструкций интерпретатора, в то время как транслируемые – могут перед

выполнением преобразовываться в непосредственно исполняемый программный модуль (например, посредством JIT-компилятора).

2.2. Взаимосвязь программных модулей

2.2.1. Интерфейсы взаимодействия программных модулей

Независимо от типа программного модуля, он практически всегда взаимосвязан с иными программными модулями. Выделяются следующие типы взаимосвязей:

- по данным;
- по коду.

В случае взаимосвязи по данным, программные модули используют данные, хранящиеся внутри иных программных модулей либо получаемые в результате их функционирования.

Взаимосвязь по коду предполагает использование части функционала одного программного модуля другим с целью обеспечения своего функционирования.

Вернемся к примеру, рассмотренному в предыдущей лекции (листинг 1). В исходном коде программного модуля единственная процедура – `main`. При этом, как было отмечено выше, эта процедура запускается не первой, а следовательно для выполнения написанного программного модуля требуется дополнительный код. Кроме того, сама процедура `main` вызывает процедуру `printf`, не описанную в исходном коде. Все это делает программный модуль, получаемый из листинга 1, взаимосвязанным с программными модулями, содержащими реализацию недостающих процедур. Именно различие в дополнительно вызываемых процедурах приводит к тому, что из одного исходного кода получается два различных непосредственно исполняемых программных модуля (рис. 1.1, 1.2).

Для определения порядка и способа взаимодействия программных модулей используются двухуровневая системы. *На первом уровне* определяются «соглашения о взаимодействии» в формате «разработчик – программный модуль», а именно задаются описания структур, классов, переменных, констант, процедур, включая и перечень входных параметров и возможные коды возвратов. Данные «соглашения» получили название «интерфейс прикладного программирования» (application programming interface – API). *На втором уровне* определяются «соглашения о взаимодействии» в формате «программный модуль – программный модуль», а именно формат представления данных в памяти, правила передачи параметров и вызова процедур, последовательность обращения к функционалу операционной системы и так далее. Данные «соглашения» получили название «бинарный интерфейс приложения» (application binary interface – ABI).

2.2.2. Связывание программных модулей

Независимо от типа взаимосвязей, программные модули могут быть.

1. **Статические** – сформированные на этапе создания программного модуля.
2. **Динамические** – формируемые в процессе запуска и выполнения программного модуля.

Динамические взаимосвязи могут реализовываться как за счет использования функционала сторонних программных модулей (динамических компоновщиков), так и путем непосредственного обращения к другому программному модулю.

Рассмотрим пример, демонстрирующий статическое и динамическое связывания. В листинге 3 приведен исходный код на языке Си, который станет основой для создания двух непосредственно исполняемых программных модулей и двух библиотек (статической и динамической). Следует обратить внимание, что код написан таким образом, чтобы, используя различные параметры компиляции, получать различный результат.

Листинг 3. Тестовая библиотека

```
1 #include <stdio.h>
2 void export_function(void);
3
4 #ifdef MAIN_MODULE
5 int main(int argc, char* argv[])
6 {
7     printf("%s: ", argv[0]);
8     export_function();
9     return 0;
10 }
11 #else
12 void export_function(void)
13 {
14     #ifdef STATIC
15         printf("Function '%s' call in static mode\n", __func__);
16     #else
17         printf("Function '%s' call in shared mode\n", __func__);
18     #endif
19 }
20 #endif
```

Для получения из исходного кода двух различных библиотек и двух различных исполняемых модулей следует выполнить следующий скрипт для интерпретатора командной строки bash (листинг 4).

Листинг 4. Команды создания 4-х программных модулей на основе исходного кода листинга 3

```
1 #!/bin/bash
2 gcc -c -DSTATIC lib_ex.c
3 ar cr libex.a lib_ex.o
4 gcc -shared -fpic -o libex.so lib_ex.c
5 gcc -c -DMAIN_MODULE lib_ex.c
6 gcc -o static.exe -static lib_ex.o -L. -lex
7 gcc -o shared.exe lib_ex.o -L. -lex -Wl,-rpath,'.'
```

В строках 2 и 3 листинга 4 создается статическая библиотека `libex.a`, в строке 4 – создается динамическая библиотека `libex.so`. В строке 5 из исходного кода создается объектный модуль, из которого в строках 6 и 7 создаются непосредственно исполняемые модули, статически связанные с библиотекой и динамически, соответственно. Для более подробного ознакомления с процедурами сборки программных модулей с использованием компилятора `gcc` рекомендуется воспользоваться [7].

Демонстрация различий получившихся программных модулей в ОС Linux (рис. 2.1) может быть осуществлена путем выполнения программ, сравнения размеров (утилита `du`), проверки зависимости от внешних программных модулей (утилита `ldd`), а также получения общей информации об исполняемом программном модуле (утилита `file`)³.

```
# ./shared.exe ; ./static.exe Выполнение программ
./shared.exe: Function 'export_function' call in shared mode
./static.exe: Function 'export_function' call in static mode
# du -b ./shared.exe ./static.exe Получение размера модулей
8464    ./shared.exe
810240  ./static.exe
# ldd ./static.exe ./shared.exe Проверка зависимости от внешних модулей
./static.exe:
    не является динамическим исполняемым файлом
./shared.exe:
    linux-vdso.so.1 (0x00007ffff49cb1000)
    libex.so => ./libex.so (0x00007f289242a000)
    libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f2892087000)
    /lib64/ld-linux-x86-64.so.2 (0x00007f289282e000)
# file ./static.exe ./shared.exe Получение общей информации о модулях
./static.exe: ELF 64-bit LSB executable, x86-64, version 1 (GNU/Linux), statically
linked, for GNU/Linux 2.6.32, BuildID[sha1]=ealc88c5b89627bbe12ba08ece6aebfb226dd
956, not stripped
./shared.exe: ELF 64-bit LSB shared object, x86-64, version 1 (SYSV), dynamically
linked, interpreter /lib64/ld-linux-x86-64.so.2, for GNU/Linux 2.6.32, BuildID[sha
1]=1bd7ca4aa6e4cb476cd70da0f005385bc7cb1879, not stripped
```

Рисунок 2.1. Различие программных модулей, полученных из исходного кода листинга 3

³Для получения информации об указанных утилитах можно воспользоваться системой документации Linux – `man`

Следует обратить внимание, что для исполняемого модуля `shared.exe` указана программа «интерпретатор» — `ld-linux`, при этом сам программный модуль является непосредственно исполняемым. Все дело в том, что `ld-linux` является динамическим компоновщиком, который и производит загрузку программного модуля и всех необходимых для него модулей в оперативную память ЭВМ, а потом передает ему управление. Напомним, что для статически собранного модуля, данная операция осуществляется на этапе создания.

Не зависимо от типа связи, работа программы компоновщика предполагает, что в каждом программном модуле формируется две таблицы:

- **экспортируемых символов** – перечень объектов программного модуля, к которым может быть получен доступ из внешних программных модулей;
- **импортируемых символов** – перечень объектов внешних программных модулей, к которым должен быть получен доступ из программного модуля.

Таким образом, задача компоновщика сводится к поиску соответствующих пар объектов в таблицах экспорта и импорта и связыванию объектов между собой (рис. 2.2).

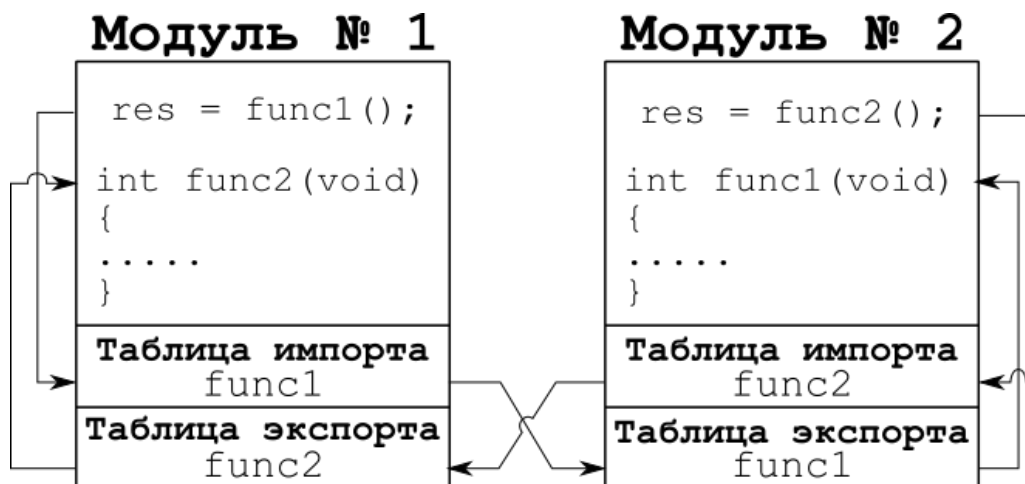


Рисунок 2.2. Связывание объектов таблиц экспорта и импорта

В случае с экспортируемыми объектами в процессе связывания никаких трудностей не возникает. Что касается импортируемых объектов, процесс связывания может производиться по одному из двух сценариев [8].

Первый сценарий предполагает, что внутри программного модуля вместо адреса импортируемого объекта записывается адрес следующего участка программного модуля, где присутствует ссылка на импортируемый объект. На последнее место в программном модуле, где встречается адрес импортируемого объекта, записывается терминальное значение. В таблицу импорта, в этом случае, записывается адрес первого участка программного модуля, где встречается обращение к импортируемому объекту (рис. 2.3а).

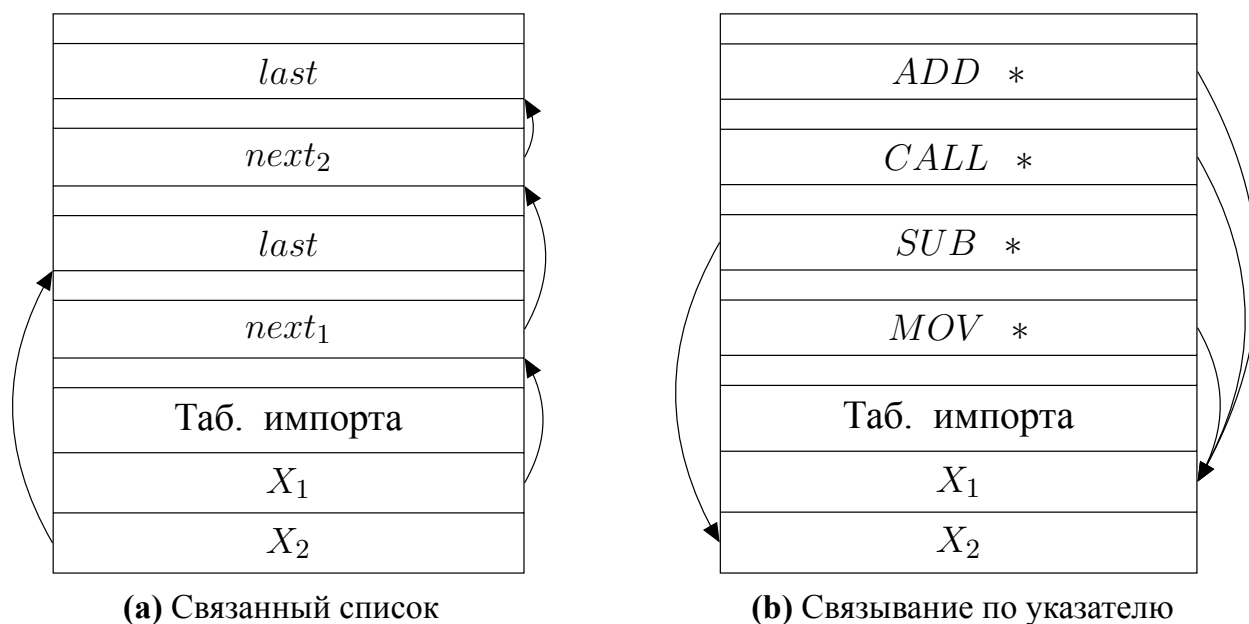


Рисунок 2.3. Возможные подходы к связыванию программных модулей

Программа компоновщик анализирует таблицы экспорта всех модулей, которые должны быть использованы при запуске программы и сохраняет у себя в памяти перечень всех экспортируемых из этих модулей объектов. Далее компоновщик обращается к таблице импорта и для каждого объекта, находит первый участок, в котором к нему осуществляется обращение, сохраняет во временной переменной ссылку на следующий участок, заменяя его на адрес объекта во внешнем модуле.

Второй сценарий предполагает, что в программном модуле создается специальная «прослойка», позволяющая унифицировать обращение к объекту. Любое обращение к импортируемому модулю производится исключительно через данную «прослойку» (рис. 2.3b).

В этом случае программа компоновщик просто модифицирует все «прослойки» имеющиеся в программном модуле, записывая в них адрес соответствующих объектов во внешних модулях.

Оба сценария обладают достоинствами и недостатками. Первый сценарий позволяет обеспечить большую производительность программного модуля. Второй подход обладает большей простотой реализации и не требует удовлетворения всех зависимостей, которых в современных программных модулях может быть очень много. По этой причине в современных ОС реализуется именно второй сценарий.

В качестве примера реализации второго сценария рассмотрим организацию обращения к импортируемым объектам в ОС Linux (рис. 2.4). В качестве демонстрационной будем использовать программу из листинга 3.

В прямоугольнике внизу рисунка показана реализация «прослойки», которая используется для доступа к внешним объектам. Адрес внешних объектов

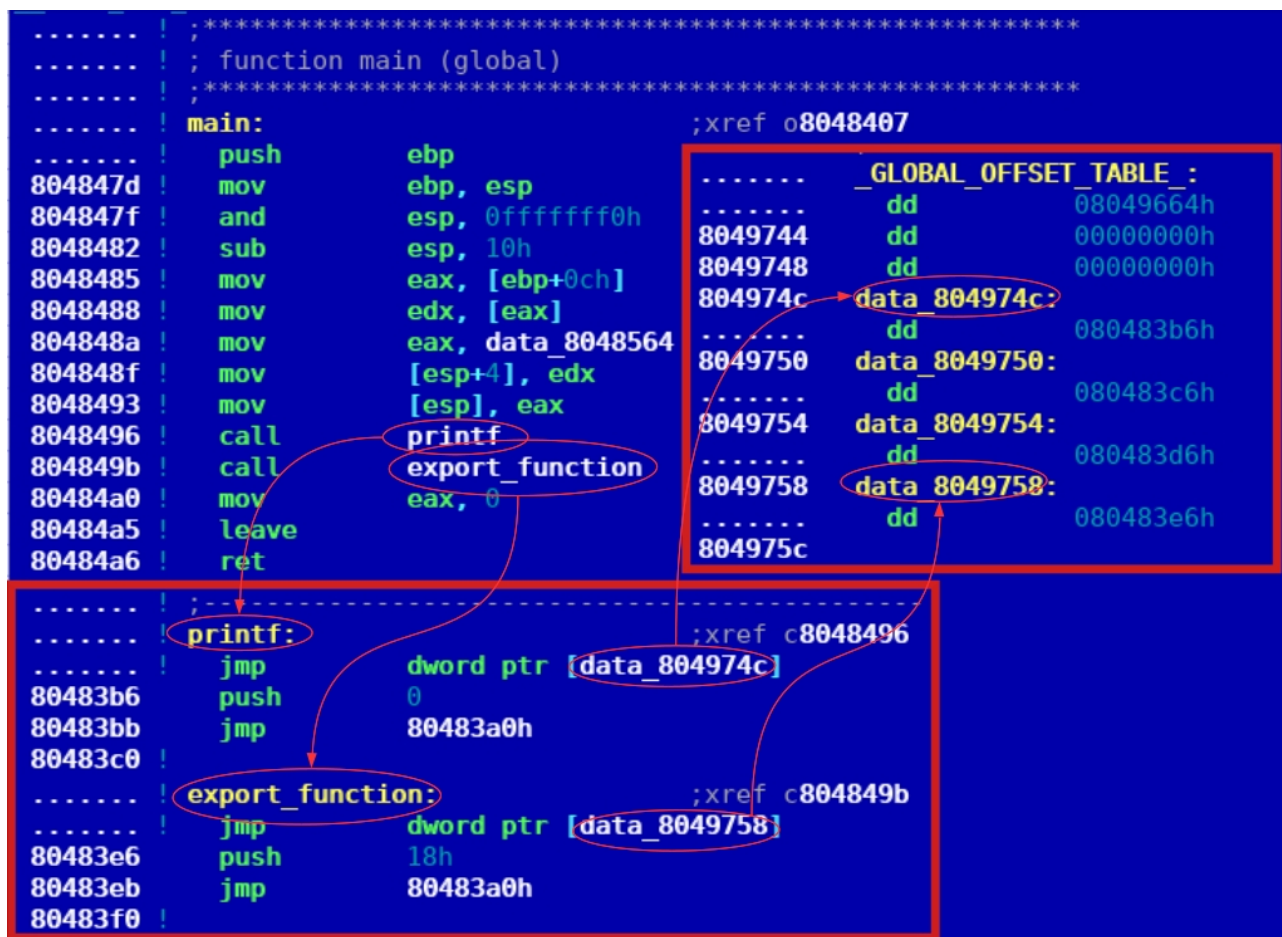


Рисунок 2.4. Пример реализации связывания по указателю в ОС Linux

находится в массиве `_GLOBAL_OFFSET_TABLE_`. Этот массив автоматически создается компилятором и заполняется в процессе связывания. Именно этот массив и является таблицей импорта программного модуля.

2.3. Основные подсистемы операционных систем

Поскольку одной из основных задач ОС является обеспечение взаимодействия программного обеспечения с аппаратным обеспечением ЭВМ, сначала рассмотрим аппаратное обеспечение.

Условно все ЭВМ могут быть декомпозированы на следующие компоненты.

1. **Процессор** – аппаратный компонент СВТ, отвечающий за исполнение команд, находящихся в программных модулях. Например, центральный процессор СВТ, графический процессор, программируемая логическая микросхема (FPGA, ASIC).
2. **Оперативная память** – энергозависимая память СВТ, к которой процессор имеет непосредственный доступ с целью чтения и(или) записи данных, а также чтения команд для последующего выполнения.

В дополнение...

В зависимости от архитектуры СВТ команды и данные могут храниться как в единой памяти (Гарвардская архитектура, также известна как «архитектура фон Неймана»), так и в физически разных областях, специально предназначенных для хранения команд и данных (Принстонская архитектура).

3. **Долговременная память** – энергонезависимая память СВТ, доступ к которой процессор может получить путем выполнения специальных команд.
4. **Периферийные устройства** – иные устройства, подключаемые к СВТ, взаимодействие с которыми процессор может осуществлять посредством специальных команд либо путем обращения к универсальным механизмам взаимодействия с периферийными устройствами.

В дополнение...

Данного уровня абстракции достаточно для дальнейшего повествования. Вместе с тем, более глубокое изучение архитектуры СВТ является необходимым для формирования комплексного понимания угроз информационной безопасности. По этой причине, рекомендуется в рамках самостоятельной подготовки ознакомиться с рядом дополнительных источников. Например, [9], [10].

Исходя из описанного выше, выделяются следующие основные подсистемы ОС.

1. **Менеджер процессов** – компонент ОС, отвечающий за обеспечение функционирования программ (п. 3), взаимодействие между ними (п. 4 и 5) и распределение ресурсов процессора между ними (п. 6).
2. **Менеджер памяти** – компонент ОС, отвечающий за размещение программ в оперативной памяти (п. 7) и управление механизмами работы с памятью, предоставляемыми аппаратным обеспечением (п. 8).
3. **Подсистема ввода-вывода** – набор компонентов ОС, реализующий доступ к периферийным устройствам СВТ. В том числе⁴:
 - **менеджер файловых систем** – подсистема ОС, отвечающая за размещение данных в долговременной памяти.
 - **сетевая подсистема** – подсистема ОС, отвечающая за взаимодействие ЭВМ с другими ЭВМ, соединенными посредством компьютерных сетей.

⁴Подсистема ввода-вывода, менеджер файловых систем, а также сетевая подсистема ОС рассматриваются во второй части курса и не описываются в рамках данного учебно-методического пособия.

Все подсистемы ОС, будучи реализованными относительно независимо, в итоговой сборке могут быть представлены как единый программный модуль, либо как набор независимых программных модулей. Независимо от этого, выделяется базовый набор функций, который обязательно реализуется в качестве единого программного модуля и является основой всей операционной системы. Именно принципы взаимосвязи остальных подсистем с этим модулем и определяют структуру ОС. Этот программный модуль называется «ядро».

В зависимости от набора реализованных функций, выделяются следующие типы ядер ОС.

Монолитное ядро — ядро ОС, реализованное в качестве единого модуля, который содержит реализацию всех основных функций ОС. В связи с тем, что ядро полностью формируется в процессе компиляции, все элементы ядра могут быть оптимально связаны друг с другом, что позволяет добиться высокой производительности. Однако, такой подход требует повторной компиляции и перезапуска всей системы в случае обновления какой-либо подсистемы ОС⁵. Но главным недостатком монолитного ядра является то, что появление критической ошибки в одной процедуре приводит к фатальной ошибке во всем ядре, а следовательно и в ОС. Примерами ОС с монолитным ядром могут считаться операционная система DOS, а также ОС Linux с версией ядра ниже 2.2.

Модульное ядро — модификация монолитного ядра, в которой реализован функционал, позволяющий загружать в ядро дополнительные модули без необходимости перезапуска всего ядра. Указанная модификация исключает необходимость перекомпиляции всего ядра. Это позволяет изменять функционал ядра без необходимости перезапуска системы, однако снижает производительность ядра из-за необходимости применения универсальных интерфейсов модулей. Остальные недостатки монолитного ядра также присущи модульному ядру. Примером ОС с модульным ядром может считаться операционная ОС Linux с версией ядра выше 2.2.

Микроядро (μ -ядро) — ядро ОС, содержащее базовый функционал, который включает процедуры обработки прерываний аппаратного обеспечения, а также менеджмента системных процессов (сервисов) и распределения памяти между ними. Весь остальной функционал ОС реализуется сервисами, реализованными в виде отдельных программных модулей. Такой подход позволяет обеспечить высокую устойчивость работы ОС. В случае аварийного завершения какого-то из сервисов, ОС производит его перезапуск и восстанавливает работоспособность системы. Вместе с тем, реализованный в микроядерных системах механизм взаимодействия сервисов является весьма сложным, что негативно отражается на производительности ОС и усложняет процесс поиска ошибок. Примерами ОС с микроядром являются MINIX, GNU\L4.

⁵Для серверных систем подобный перезапуск может потребовать значительных временных затрат.

В дополнение...

В качестве механизма взаимодействия модулей в ОС, построенной на основе микроядерной архитектуры, используется пересылка сообщений. Проблемы, возникающие при его использовании, заключаются в необходимости обеспечения гарантированной доставки сообщений и сохранения первоначального хронологического порядка отправки. В случае сбоя в одном из этих процессов результат работы ОС будет отличаться от ожидаемого.

Гибридное ядро – модификация микроядра, в которой полностью реализуется функционал, связанный с менеджментом процессов и памяти. Это позволяет частично решить проблему синхронизации сообщений, поскольку их основной поток в микроядерных системах направляется именно этим подсистемам. Кроме того, включение этих подсистем в ядро просто расширяет уже реализованный в нем базовый функционал. Также стоит отметить, что частично функционал менеджмента процессов может быть по-прежнему реализован вне ядра, но он используется для управления не критичными процессами или потоками (подробнее в п. 3). Примером ОС с гибридным ядром являются Windows класса NT (2000, XP, Vista, 7 и т.д.).

На рис. 2.5 продемонстрированы различия в распределении функционала операционных систем, построенных на основе монолитного (модульного) ядра, микроядра и гибридного ядра ОС.

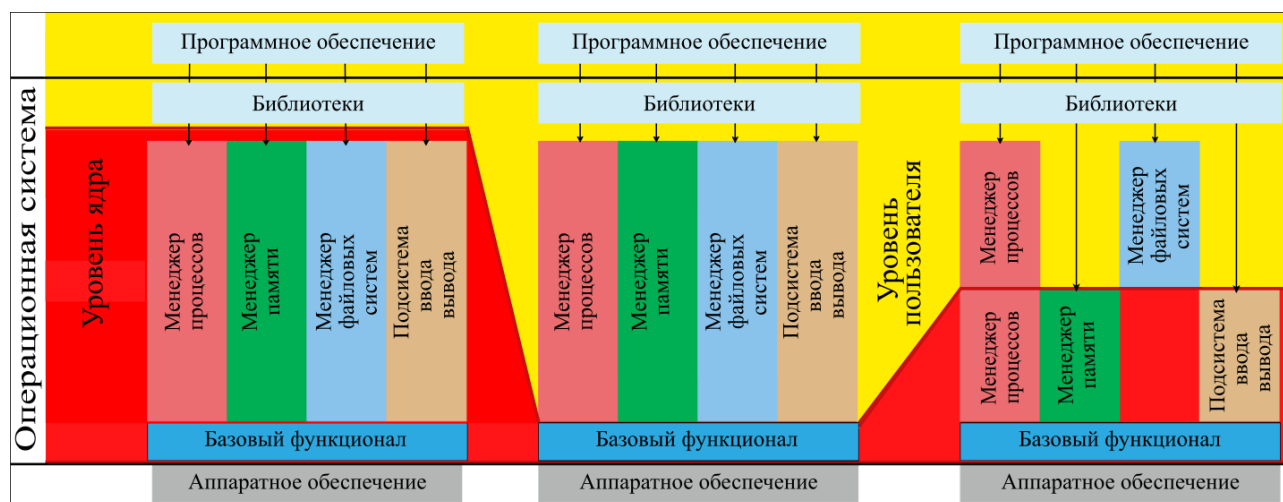


Рисунок 2.5. Распределение функционала ОС в зависимости от архитектуры ядра

Кроме классического подхода к построению ОС, в центре которого стоит ядро, существует отдельный класс систем, построенных на базе **экзоядра**.

Экзоядро – ядро операционной системы, реализованное в виде статической библиотеки (набора библиотек), реализующей базовые функции ОС.

Задача, решаемая при использовании экзоядра, сводится к предоставлению базовых функций ОС программному обеспечению. Таким образом, программное обеспечение, собранное с такими библиотеками, становится полноценной ОС, которая является замкнутой относительно внешних программ. Например, прошивка для специальных устройств. По своей сути экзоядро для программ исполняет ту же роль, что и экзоскелет для человека.

2.4. Загрузка ЭВМ

2.4.1. Общие этапы

После рассмотрения общей структуры ОС следует перейти к описанию процесса загрузки в оперативную память и связывания разрозненного набора программных модулей на долговременном носителе информации во взаимоувязанный «организм».

Данным процессом является загрузка СВТ.

Рассмотрим этапы загрузки СВТ от момента подачи электропитания до момента запуска пользовательских приложений.

На первом этапе производится инициализация основной платы аппаратного обеспечения (материнской платы) и запуск базовой микропрограммы (прошивки), которая управляет работой всех элементов на данной плате. В классической архитектуре СВТ (персональных и серверных) наименования прошивок совпадают с именем реализованных в них интерфейсов взаимодействия с внешними программными модулями. Выделяются два основных интерфейса:

- базовая система ввода-вывода (BIOS);
- интерфейс расширяемой прошивки (EFI).

На втором этапе микропрограмма «опрашивает» все аппаратные интерфейсы, к которым могут быть подключены периферийные устройства и, в случае обнаружения устройств, осуществляет их инициализацию.

В дополнение...

Некоторые периферийные устройства, в частности, подключенные к шине PCI, перехватывают управление у основной микропрограммы при своей инициализации. При этом данным устройствам может быть доступен весь функционал СВТ.

Ярким примером такого периферийного устройства является аппаратно-программный модуль доверенной загрузки (АПМДЗ), в том числе с функцией сквозного шифрования жестких дисков.

Инициализация периферийных устройств занимает некоторое время, а в связи с тем, что современные ОС при запуске повторно инициализируют

все периферийные устройства, в EFI присутствует режим «быстрой загрузки» (fastboot). В данном режиме EFI инициализирует минимальный набор периферийных устройств (клавиатуру, мышь, монитор, загрузочные носители) и переходит к следующему этапу загрузки.

На третьем этапе микропрограмма определяет источник получения загрузчика ОС⁶. После его определения осуществляется «чтение» загрузчика в оперативную память и передача ему управления.

В дополнение...

Загрузчик ОС – это программный модуль, хранящийся во внешней по отношению к материнской плате памяти. Фактически он может выполнять любые функции. В частности, может совершать некоторые противоправные действия, например, похищать персональные данные пользователя, после чего передавать управление реальному загрузчику ОС.

Вероятность подобного события возрастает в том случае, когда основным источником для чтения загрузчика являются периферийные носители информации (usb flash или привод оптических дисков) или сетевые ресурсы.

Последним этапом загрузки является передача управления ядру ОС. Начиная с этого этапа, процесс загрузки зависит исключительно от архитектуры ОС и возможностей, предоставляемых центральным процессором СВТ.

2.4.2. Основные возможности центрального процессора

С развитием аппаратных средств СВТ многие механизмы, ранее реализовывавшиеся исключительно программно, стали реализовываться на аппаратном уровне. Так произошло и с некоторыми базовыми механизмами ОС, которые теперь реализуются на основе функционала центрального процессора⁷.

Основным механизмом, широко используемым в работе ОС, является разграничение доступа к областям памяти и командам. Классическим подходом к разграничению памяти в процессорах Intel являются «кольца защиты». Всего разработчику программного обеспечения доступны четыре кольца (*ring0*, *ring1*, *ring2*, *ring3*). Код, выполняемый в кольце с некоторым, индексом имел доступ к данным своего кольца, а также кольца с большим индексом. В связи с этим, по задумке разработчиков процессоров, должна была образоваться вложенная структура, на каждом уровне которой располагались бы программные модули различных типов (рис. 2.6).

Со временем разработчики ОС упростили данную модель и стали использовать для размещения программных модулей в основном третье кольцо.

⁶Источником может являться как долговременный носитель информации, так и сетевой ресурс.

⁷Эти механизмы доступны в защищенном режиме работы процессора и более подробно с ними можно ознакомиться в [11].

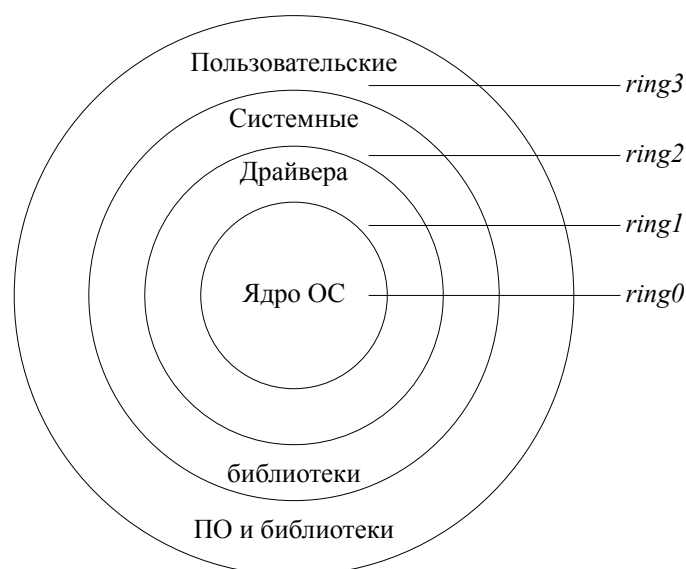


Рисунок 2.6. «Кольца защиты»

Исключение было сделано только для ядра ОС, которое так и осталось на нулевом кольце. Вслед за разработчиками ОС, модификацию уровней защиты сделали разработчики процессоров, добавив режим, в котором код может функционировать в 2 режимах: в пользовательском режиме, либо в режиме ядра.

Фактически, разграничение доступа достигалось за счет того, что память делилась на отдельные блоки⁸, информация о которых хранилась в специальных таблицах. Одно из полей данной таблицы отвечает за уровень привилегий и может иметь размер 2 или 1 бит, что позволяет задать 4 или 2 уровня, соответственно. Модификация этих таблиц изначально доступна только в привилегированном режиме, однако разрешение на их модификацию может получить и процесс из пользовательского режима. Например, менеджер памяти, реализуемый в виде сервиса микроядерной ОС. Помимо этих таблиц, выделяются дополнительные области служебных данных процессора, доступ к которым возможен только из привилегированного режима. Для работы с указанными областями в процессоре существует ряд специальных («привилегированных») команд, которые могут быть доступны исключительно из уровня ядра.

Для организации взаимодействия между уровнем ядра и пользовательским уровнем используются специальные механизмы, которые позволяют передавать управление между программными модулями на разных уровнях. К таким механизмам относятся: прерывания, ловушки и системные вызовы. Именно эти механизмы обеспечивают возможность обращения пользовательского ПО к функциям ядра ОС.

⁸Детали в п. 8.

В дополнение...

Как и многие системы защиты, механизмы аппаратного разграничения доступа имеют уязвимости. В частности, выявленные в 2018 году уязвимости Spectre и Meltdown, за счет ошибок в работе блока ассоциативной трансляции, используемом в процессорах для ускорения трансляции адресов виртуальной памяти [12].

Основная опасность, исходящая от аппаратных уязвимостей, заключается в трудности их изоляции программными средствами. Парой это в принципе невозможно.

Вторым, не менее важным механизмом, реализованным в современных процессорах СВТ, является поддержка многозадачности. По сути, сам механизм не обеспечивает многозадачность, а лишь предоставляет базовый набор функций для его реализации. Основными из них являются:

- защита обработчика прерывания системного таймера;
- задание активного сегмента состояния задачи.

Используя этот функционал, ОС может гарантировать вызов программного модуля, отвечающего за переключение задач⁹, и указать процессору необходимый аппаратный контекст¹⁰ для программного модуля, который будет выполняться следующим. При этом, вся необходимая информация для возобновления остановленного программного модуля будет сохранена в его контексте автоматически.

2.5. Виртуализация

2.5.1. Уровни виртуализации

Механизм виртуализации предназначен для эмуляции присутствия нескольких ЭВМ на одном и том же физическом оборудовании.

Основную роль в процессе визуализации играет специальное программное обеспечение – *монитор виртуальных машин*. Его также называют *гипервизор*.

В зависимости от задач, которые решает гипервизор, выделяются два уровня виртуализации:

- виртуализация на уровне процессов;
- виртуализация на уровне ОС.

Виртуализация на уровне процессов предназначена для выполнения программных модулей в ОС, отличных от тех для которых они не были

⁹Данный программный модуль называется «планировщик процессов» и будет рассмотрен в п. 6.

¹⁰Подробнее о контексте программного модуля будет рассказано в п. 3.

разработаны. Это может касаться как различия версий и семейств ОС, так и архитектуры процессоров, на которые они были рассчитаны. Примером программных модулей, реализующих виртуализацию на уровне процессов, являются: *WINE*, *QEMU*¹¹.

Виртуализация уровня ОС предназначена для обеспечения функционирования одновременно нескольких ОС. ОС, выполняемая в виртуальной среде, называется *гостевой*, а функционирующая вне виртуальной среды – *хостовой*.

2.5.2. Виртуализация уровня ОС

В зависимости от возможностей аппаратного обеспечения, а также используемого гипервизора, выделяют следующие виды виртуализации на уровне ОС.

1. Виртуализация без использования аппаратного обеспечения.
2. Виртуализация с использованием аппаратного обеспечения:
 - паравиртуализация;
 - полная виртуализация.

Виртуализация без использования возможностей аппаратного обеспечения реализуется специальным гипервизором – *интерпретатором*. Данный гипервизор последовательно выполняет инструкции, которые должны передаваться на выполнение процессору. Таким образом, гипервизоры рассматривают носитель информации (образ носителя), как большой программный модуль, который последовательно выполняют. В качестве одного из самых «интересных» гипервизоров данного типа можно назвать программное обеспечение *Bochs*¹². Данный гипервизор привлекателен тем, что в нем реализован отладчик уровня процессора, что позволяет с его помощью отлаживать код загрузчиков или иного программного обеспечения, которое невозможно отладить средствами ОС.

Виртуализация с использованием аппаратного обеспечения предполагает наличие в аппаратном обеспечении СВТ поддержки технологии виртуализации¹³. Данная технология позволяет перенести решение части задач виртуализации на аппаратное обеспечение. При этом за управление всем процессом по-прежнему отвечает гипервизор.

¹¹Проект *QEMU* представляет гипервизоры, обеспечивающие виртуализацию на всех уровнях. Основное достоинство данного проекта заключается в широком спектре виртуализируемых архитектур, в т.ч. *arm*, *sparc*, *x86* и *x86_64* [13].

¹²В разговорной речи также известный как «Бочка».

¹³В центральных процессорах *Intel* поддержка виртуализации реализуется за счет *Virtualization Technology (VT, технология виртуализации)*, а на процессорах *AMD* – *Secure Virtual Machine (SVM, безопасная виртуальная машина)*.

Паравиртуализация отличается от полной виртуализации тем, гипервизор не стремится эмулировать виртуальную машину, которая бы выглядела как настоящее основное оборудование. Вместо этого он представляет машиноподобный программный интерфейс, который явно раскрывает факт наличия виртуальной среды. Например, он предлагает набор гипервизоров (hypercall), позволяющих гостевой ОС отправлять явные запросы к гипервизору (во многом это похоже на то, как системный вызов предлагает службы ядра приложениям). Явное взаимодействие гостевой ОС и гипервизора позволяет обеспечить более высокую производительность [14].

В дополнение...

Некоторые создатели гипервизоров разрабатывают специальные драйвера для различных ОС, позволяющие этим ОС функционировать в режиме паравиртуализации. В качестве примера можно привести проект Oracle VirtualBox, для которого на официальном сайте доступен образ CD-диска, содержащего драйвера для различных ОС [15].

2.5.3. Типы гипервизоров

Как говорилось выше, независимо от уровня виртуализации, основную роль в обеспечении ее работы играет гипервизор. Сам гипервизор также может реализовываться различными способами и на различных уровнях информационной системы. В связи с этим, в зависимости от уровня реализации, выделяют два основных типа гипервизоров.

1. Гипервизор *первого типа*, реализуемый как прослойка между ОС и аппаратным обеспечением ЭВМ (рис. 2.7a).
2. Гипервизор *второго типа*, реализуемый как прикладное программное обеспечение ОС (рис. 2.7b).

Посредством гипервизоров первого типа реализуются все виды визуализации уровня ОС.

В дополнение...

При использовании гипервизоров первого типа все ОС функционируют в виртуальной среде, что исключает возможность выделения хостовой ОС. Вместе с тем, управление самим гипервизором может осуществляться одной из гостевых ОС. Для выделения среди всех гостевых ОС главной, для гипервизоров данного типа введена специальная терминология, в рамках которой ОС, управляющая гипервизором, получила наименование Dom-0, а все остальные – Dom-U.

Посредством гипервизоров второго типа может реализовываться любая виртуализация, за исключением паравиртуализации.

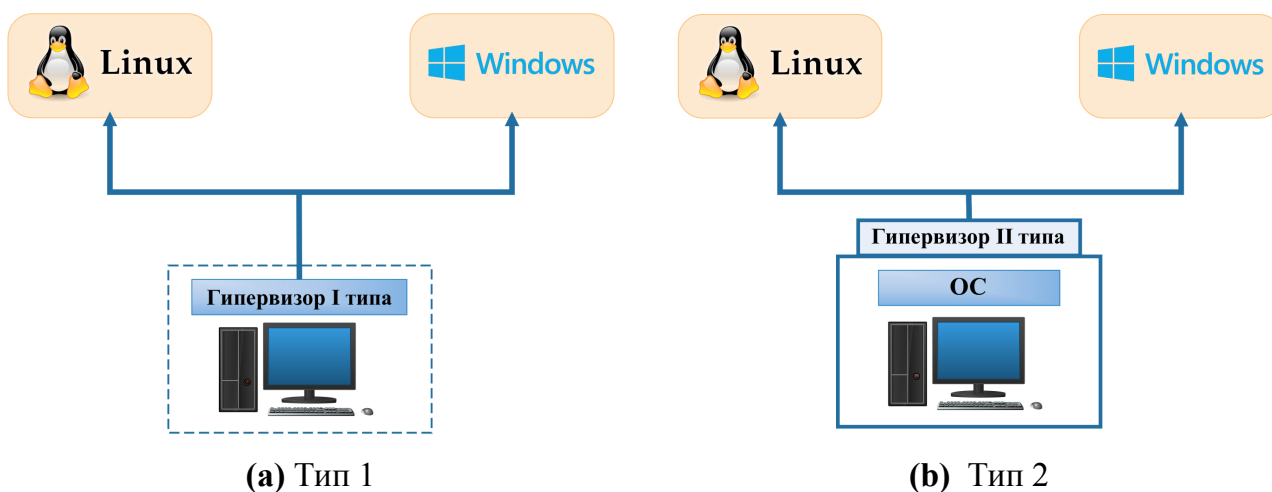


Рисунок 2.7. Типы гипервизоров

2.5.4. Угрозы информационной безопасности, связанные с виртуализацией

В завершении раздела следует отметить угрозы, исходящие от данной технологии. Наибольшей угрозой информационной безопасности является вредоносное программное обеспечение, реализуемое в виде гипервизора первого типа (рис. 2.8).

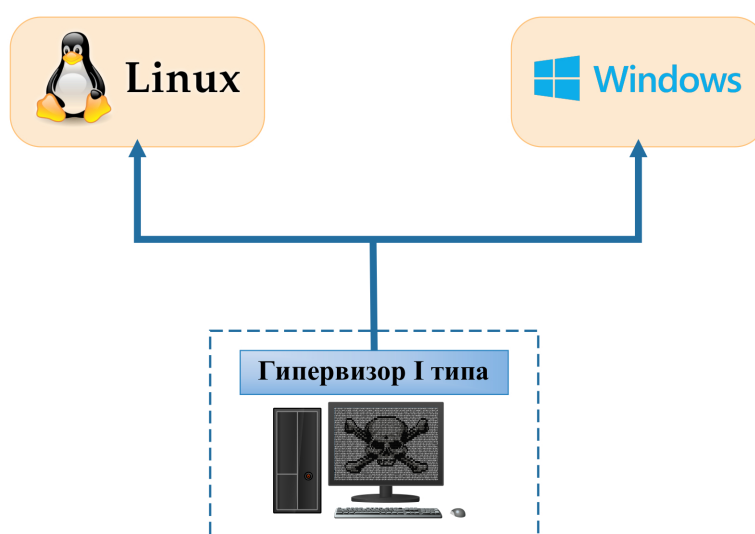


Рисунок 2.8. Реализация вредоносного программного обеспечения в виде гипервизора первого типа

Подобное вредоносное программное обеспечение функционирует на уровень ниже ОС, а следовательно может получать полный доступ к ресурсам системы и оставаться незаметным для антивирусных средств защиты.

Выделяют три подхода к внедрению подобного вредоносного программного обеспечения.

1. Программная закладка в прошивке аппаратного обеспечения, получающего управление при запуске СВТ, но до ОС.
2. Загрузка со съемного носителя информации, где в загрузчике содержится реализация вредоносного программного обеспечения.
3. Активация вредоносного программного обеспечения непосредственно в процессе функционирования ОС.

Наибольший интерес с точки зрения информационной безопасности представляет последний подход, поскольку его использование открывает возможность доставки и активации гипервизора на СВТ жертвы без необходимости получения физического доступа к СВТ и может быть реализовано широким кругом злоумышленников. Принципиальную возможность такого внедрения продемонстрировала в 2006 году польская исследовательница в области информационной безопасности Йона Рудковская, создав «синюю таблетку» [16].

2.6. Вопросы для самопроверки

1. Приведите примеры задач, при решении которых наиболее эффективно использование тех или иных типов программных модулей.
2. В чем достоинства и недостатки непосредственно исполняемых программных модулей?
3. Какие подсистемы являются необходимыми для любой ОС, а реализация каких не обязательна? Поясните на примерах.
4. На каких этапах загрузки ОС могут быть осуществлены внедрение и активация вредоносного программного обеспечения?
5. Приведите примеры гипервизоров каждого типа.

3. ПРОЦЕССЫ И ПОТОКИ

3.1. Определения

Как уже было определено ранее, операционная система это некоторый набор программ. Настало время ввести соответствующее понятие.

Программа — файл, хранящийся в памяти вычислительной системы и предназначенный для выполнения [17].

Из определения видно, что программа сама по себе является статическим объектом, не выполняющим какие-либо действия. При функционировании вычислительных систем основная роль в решении всех задач отдаётся *процессам*.

Процесс — программа на стадии выполнения [17].

Помимо столь тривиального определения, понятие *процесс* в контексте операционных систем может быть определено следующим образом.

Процесс — логический объект, обладающий собственным адресным пространством [17].

В дальнейшем станет более понятно, почему подобное определение также имеет значения. Теперь же раскроем понятие одного из терминов, вошедших в это определение.

Адресное пространство — это набор адресов оперативной памяти, доступных процессу для чтения, записи, выполнения [17].

В дополнение...

Важно заметить, что адресное пространство это именно набор адресов, а не память, доступная процессу. В п. 8 будет показано, что процесс может осуществлять обращения к адресам памяти, физически не существующие в вычислительной системе.

Адресное пространство может быть условно разделено на следующие области.

1. **Область кода** (*text*) — область адресного пространства, предназначенная для хранения команд (кода) программы.
2. **Статическая область данных** — область адресного пространства, имеющая фиксированный размер и предназначенная для хранения значений статических переменных, констант (в том числе строковых) и т.д. Разделяется на:

- **область инициализированных данных** (`data`) — предназначена для хранения глобальных переменных и (или) массивов, значение которых должно быть определено до начала работы процесса.
- **область констант** (`rodata`) — предназначена для хранения строковых констант и глобальных переменных, доступных только для чтения (например, с модификатором `const`).
- **область неинициализированных данных** (`bss`) — предназначена для хранения глобальных переменных и (или) массивов, начальное значение которых явно не задано в исходном коде программы.

3. **Динамическая область данных** — область адресного пространства переменной длины, используемая для хранения служебных данных и значений динамически создаваемых переменных (объектов). Разделяется на:

- **«куча»** (`heap`) — область данных, динамически распределяемая с использованием процедур работы с памятью `malloc`, `calloc`, `free` и т.д. Как правило, все выделенные блоки в «куче» организуются в виде специальной структуры (например, двусвязанного списка).
- **стек** (`stack`) — область данных, предназначенная для хранения локальных переменных, адреса возврата и параметров вызова процедур и функций. Работа со стеком на уровне языков высокого уровня осуществляется неявно для разработчика.

В дополнение...

В связи с тем, что область `bss` используется для хранения переменных, начальное значение которых не задано, при компиляции программы в итоговом файле, как правило, содержится только информация о размере этой области и начальном адресе в памяти. Пул адресов под эту область выделяется в процессе загрузки программы в память. Указанный способ хранения области делает невозможным определение начального значения переменной (в случае необходимости) без новой сборки программы.

Рассмотрим, как распределяются в памяти объекты, описанные в исходном коде программы в зависимости от места и способа их объявления (листинг 5).

Листинг 5. Распределение памяти

```
1 #include <stdlib.h>
2 int var1 = 0;
3 int var2;
4 int main(int argc, char* argv[])
5 {
6 char *var3 = "Hello, \World!\n";
```

```

7 char var4[10] = "Good_bay!\n";
8 var3 = (char*)malloc(1);
9 var3[0] = 0;
10 return 0;
11 }

```

В приведённом примере переменная `var1` будет размещена в области `data`, переменная `var2` — в `bss`, а переменные `argc`, `argv`, `var3` и `var4` — в стеке. При этом в 9 строке значение элемента массива `var3[0]` размещается в «куче». Также следует отметить, что изначально в переменной `var3` хранился адрес строки `"Hello,_World!\n"`, сама же строка размещается в области `rodata`. Одновременно, в связи со способом объявления переменной `var4`, строка `"Good_bay!\n"` не будет выделяться в виде отдельной строковой константы, вместо этого в код процедуры будет добавлен фрагмент, заполняющий массив `var4` соответствующими значениями. Такой подход замедляет выполнение процедуры, но гарантированно позволяет модифицировать все элементы строки.

3.2. Особенности разбиения адресного пространства

Приведенное разбиение адресного пространства на области является наиболее полным и может отличаться при использовании разных версий и типов компиляторов. В частности, область `rodata` может отсутствовать вовсе, а все данные из неё будут размещаться в области `data`. Указанная особенность может критическим образом сказаться на переносимости кода между различными платформами. Рассмотрим простой пример (листинг 6), в котором производится итерация элемента текстовой строки, изначально содержащего символ `'0'`. Поскольку в таблице ASCII символы `'0'`, `'1'`, `'2'`, `'3'`, `'4'`, `'5'`, `'6'`, `'7'`, `'8'`, `'9'` следуют последовательно, то увеличение кода символа также увеличит отображаемое значение. Однако, реальный результат выполнения программы зависит от компилятора, который используется при создании бинарного образа программы. Если строка `"Message_#0\n"` будет размещена в области `rodata`, то выполнение программы будет прервано системой из-за ошибки доступа к памяти.

Листинг 6. Разнообразие использования областей данных

```

1 #include <stdio.h>
2 int main(int argc,char* argv[])
3 {
4     char* msg="Message_#0\n";
5     int i;
6     for ( i = 0 ; i < 5 ; i++) {
7         msg[9]++;
8         puts(msg);
9     }
10    return 0;
11 }

```

Подобное различие в подходах к разбиению адресного пространства может привести к печальным последствиям в тех случаях, когда разработка и отладка программного обеспечения проводится в одной среде, а эксплуатация – в другой.

3.3. Область стека

Еще одним узким местом с точки зрения безопасности является стек. Как уже было отмечено, стек является специальной областью памяти, данные в которой могут размещаться независимо от разработчика. Вместе с тем, весь набор этих данных может стать ему доступен. Для этого достаточно определить адрес размещения любого элемента в стеке и далее адресоваться относительно него. Рассмотрим возможный набор данных, которые могут размещаться в стеке, и причины по которым они туда попадают (рис. 3.1):

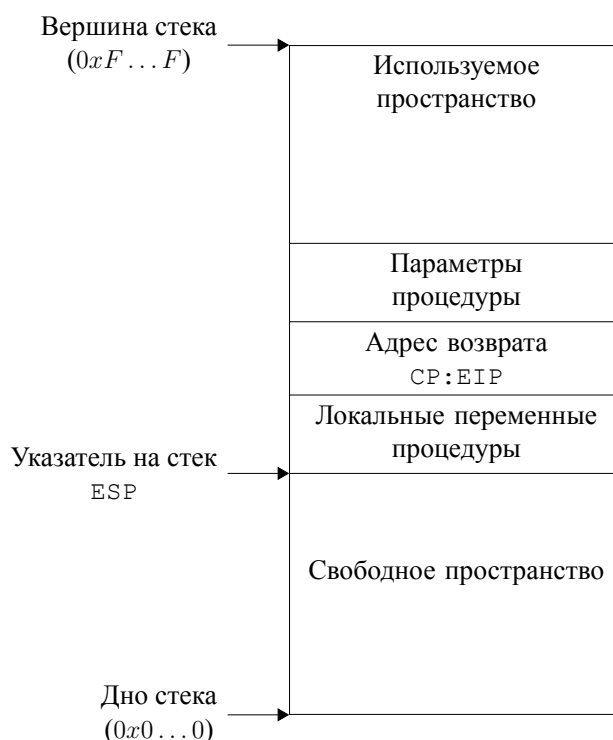


Рисунок 3.1. Структура данных в стеке

- *адрес возврата из процедуры* — данные размещаются в стеке перед вызовом процедуры и определяют адрес памяти, по которому располагается команда, следующая за командой вызова процедуры;
- *параметры при вызове процедур*¹⁴ — параметры вызова процедуры размещаются в стеке непосредственно перед адресом возврата, что позволяет без труда адресоваться к ним. Кроме того, это позволяет создавать процедуры с переменным числом параметров;

¹⁴Для процедур типа cdecl, pascal, stdcall.

- *локальные переменные* — поскольку локальные переменные до момента вызова процедуры не требуются, то логично не выделять для их хранения отдельную ячейку памяти. Особенно если локальных переменных в программном обеспечении очень много. Вместо этого каждый раз, когда процедура вызывается, в области стека от адреса возврата и далее резервируется специальная область, предназначенная для хранения данных локальных переменных. К слову, такой подход обеспечивает возможность многократного вложенного (в т.ч. рекурсивного) вызова процедур;
- *временные данные* — в отличие от предыдущих пунктов, в данном случае нельзя провести чёткую параллель между совершаемой операцией на языке высокого уровня и модификацией стека. Существуют операции со стеком, которые могут дополнительно реализовываться компилятором, при трансляции программ. Указанные операции производятся на уровне машинных команд и скрыты от разработчика¹⁵.

В дополнение...

*Область стека, в которой размещаются локальные переменные и временные данные процедуры, называется **кадром стека (фреймом стека)** процедуры. Адрес начала кадра стека (**указатель на кадр стека**) традиционно хранится в регистре **ebp**. Поскольку у каждой подпрограммы свой указатель на кадр стека, при компиляции компилятор в начало каждой подпрограммы добавляет следующие строки.*

```
push %ebp
mov %esp, %ebp
```

*Таким образом, сразу после перехода к очередной подпрограмме в стеке сохраняется указатель на фрейм стека вызывающей подпрограммы, который восстанавливается непосредственно перед выходом из подпрограммы. Например, посредством команды **leave[11]**.*

Не сложно заметить, что служебные данные процедуры (адрес возврата, параметры вызова) имеют большие адреса в сравнении с локальными переменными. Такое расположение данных приводит к тому, что запись в массив, выделенный в стеке¹⁶, элементов, количество которых превосходит размер массива, повлечет переписывание служебных данных и, как следствие, повлияет на работоспособность программы. Указанная особенность стала причиной появления уязвимости, получившей название «переполнение стека».

¹⁵ Данные ограничения не касаются программистов, использующие в качестве языка программирования ассемблер.

¹⁶ В связи с особенностью структуры «стек», часто можно услышать термин «выделить память над стеком».

Суть уязвимости заключается в том, что злоумышленник передаёт в качестве входных данных процедуры специальным образом сформированную строку. В этой строке будут содержаться новый адрес возврата из процедуры и некоторая «полезная нагрузка», называемая *shellcode*, управление на который должно быть передано. В том случае, когда данные из указанной строки копируются в локальный массив без контроля его заполнения, строка полностью размещается в стеке и перезаписывает реальный адрес возврата. После этого выход из процедуры приведёт к передаче управления по адресу точки входа в *shellcode*. Реже данная методика используется для обхода процедуры аутентификации.

Рассмотрим учебный пример эксплуатации указанной уязвимости. В листинге 7 приведён исходный код программы, при компиляции и исполнении которого в некоторых 32-х разрядных дистрибутивах ОС Linux¹⁷ будет выполнен специальным образом сформированный код.

Листинг 7. Эксплуатация уязвимости «переполнение стека»

```
1 #include <string.h>
2 unsigned char shellcode[38] =
3     "\x90\x90\x90\x90\x89\xe1\x81\xc1\xd1\x00\x00\x00"
4     "\x31\xc0\x40\x89\xc3\xc1\xe0\x02\x89\xc2\x01\xda"
5     "\xcd\x80\x31\xc0\x40\x31\xdb\xcd\x80\x48\x61\x63"
6     "\x6b\x0a";
7 int main(int argc, char *argv[])
8 {
9     void *ptr = *((void**)shellcode) = (void*) (&argc);
10    ptr -= sizeof(void*);
11    memcpy(ptr, shellcode, 38);
12    return 0;
13 }
```

Рассмотрим подробнее приведённый код, основываясь на распределении данных в стеке (рис.3.2). Первой операцией (строка № 9) мы определяем адрес в стеке, по которому размещён первый параметр процедуры `main` — `argc`. Указанный адрес сохраняем в первые 4 байта сформированной строки и в переменной `ptr`. Поскольку основной задачей является определение нового значения адреса возврата из процедуры, значение указателя `ptr` уменьшается на величину соответствующую размеру памяти, отводимой для хранения адреса. На последнем шаге (строка № 11) по адресу, записанному в `ptr`, копируется все содержимое массива `shellcode`. Не сложно заметить, что после этой операции адрес возврата из процедуры будет соответствовать адресу переменной `argc`, которая уже была перезаписана содержимым массива `shellcode`. Таким образом, после выполнения оператора `return`, управление процессом будет передано на коде в массиве `shellcode`.

¹⁷Тестирование кода проводилось на дистрибутиве ОС Linux SliTaz версии 5.0 с использованием компилятора gcc 4.6.3.

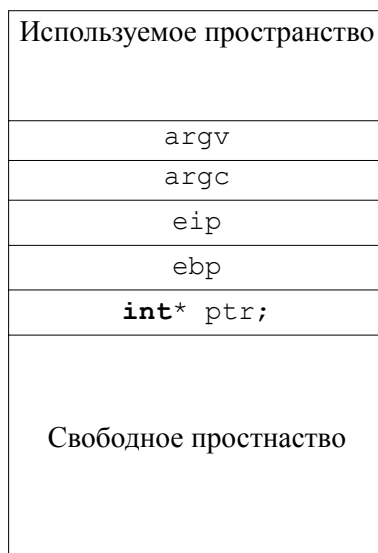


Рисунок 3.2. Стек программы из листинга 7

Для пояснения эффекта от успешной эксплуатации уязвимости, рассмотрим shellcode в виде машинных команд (листинг 8). Не трудно заметить, что в коде осуществляется обращение к двум системным вызовам с номерами 4 (write – запись в файл) и 1 (exit – завершение процесса) посредством 128 прерывания (**int** \$0x80).

Листинг 8. Исходный код shellcode

```

1  0x90          nop
2  0x90          nop
3  0x90          nop
4  0x90          nop
5  0x89 0xe1     mov %esp, %ecx
6  0x81 0xc1 0x1d 0x00 0x00 0x00 add $0x1d, %ecx
7  0x31 0xc0     xor %eax, %eax
8  0x40         inc %eax
9  0x89 0xc3     mov %eax, %ebx
10 0xc1 0xe0 0x02 shl $0x2, %eax
11 0x89 0xc2     mov %eax, %edx
12 0x01 0xda     add %ebx, %edx
13 0xcd 0x80     int $0x80
14 0x31 0xc0     xor %eax, %eax
15 0x40         inc %eax
16 0x31 0xdb     xor %ebx, %ebx
17 0xcd 0x80     int $0x80
18 0x48 0x61 0x63 0x6b 0x0a     .ascii "Hack\n"

```

Таким образом, в результате выполнения кода на экран будет выведено сообщение «Hack», а приложение будет корректно завершено. В качестве дополнительного источника информации можно воспользоваться [18].

В дополнение...

Создание полезной нагрузки при реализации подобной уязвимости требует соблюдения ряда условий. Одно из них – отсутствие нулевых байт. Это связано в том, что чаще всего переполнение производится в связи с использованием процедур работы со строками (например, `strcpy` или `scanfa`). При реальном функционировании ВС такое возможно лишь при работе с текстовыми данными, на основании которых трудно сформировать полезную нагрузку. В случае с бинарными данными различных форматов, нулевое значение присутствует всегда и не однократно. В связи с этим, отсутствие нулевых байт в бинарных данных может свидетельствовать о вредоносном содержимом.

*Еще одним критерием для отнесения данных к потенциально опасным является наличие «заполнения» (*padding*), предшествующего полезной нагрузке. Чаще всего данная область генерируется злоумышленниками автоматически и полностью состоит из некоторого фиксированного байта. В рассмотренном примере это значение `0x90`.*

^aИменно ошибка в реализации данной процедуры легла в основу создания первого в истории сетевого вируса – «червя Мориса».

Следует отметить, что в современных операционных системах реализуется комплекс мер, позволяющий усложнить возможность реализации указанной уязвимости. В том числе, на аппаратном уровне, за счёт использования *NX-бита*.

Приведенный пример является не единственной уязвимостью связанной со стеком. В качестве дополнительного примера можно привести операцию «раскручивания стека». Она заключается в том, что подпрограмма обращается к фрейму стека вызывающей подпрограммы с целью получения данных из локальных переменных, а также указателя на следующий фрейм. Таким образом, процедура из сторонней библиотеки может получить доступ к локальным переменным, использующей ее программы, и получить информацию о ключах шифрования, паролях и иную чувствительную для пользователя информацию.

3.4. Модели памяти процесса

Для завершения рассмотрения темы адресного пространства, следует разобраться с тем, каким образом секции программ могут располагаться в памяти ЭВМ. В зависимости от архитектуры вычислительной системы, порядок размещения областей данных и правила обращения к ним различаются. В тех случаях, когда размер одного блока, доступного для адресации, меньше чем размер реально доступной памяти, для адресации используются подходы, связанные с разбиением памяти на блоки меньшего размера. Так, в реальном режиме работы процессоров Intel для адресации используются 16-ти битные

регистры, что позволяет адресовать не более чем 64 Кб памяти. Блок данных, в рамках которого происходит адресация, получил название *сегмент*.

В тех случаях, когда требуется обратиться к памяти большего объёма, используются несколько сегментов, адресация между которыми осуществляется с использованием сегментных регистров [11]. При этом исполнение команд, в которых необходимо обращения к разным сегментам, происходит медленнее, чем обращение к адресам в одном сегменте. По этой причине при разработке программ на языках высокого уровня может потребоваться указать модель памяти, определяющую правила разбиения программы на сегменты. Выделяются следующие модели, которые программист может указать в качестве одного из параметров при компиляции программы [19].

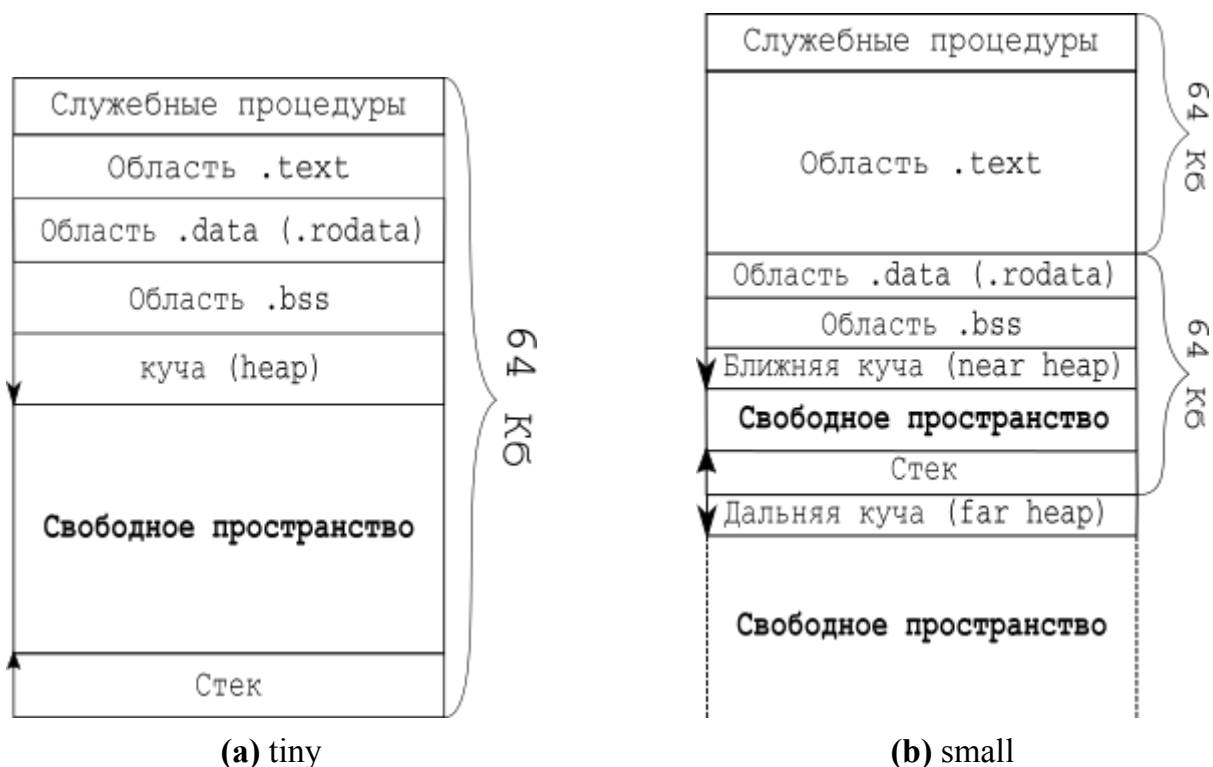


Рисунок 3.3. Модели памяти **tiny** и **small**

Модель **tiny** — модель распределения памяти, в рамках которой область кода и область данных (в том числе стека и «кучи») программы размещаются в едином сегменте данных. При этом заполнение «кучи» происходит от области статических данных «в направлении» старших адресов памяти, а стека — от старших адресов памяти к младшим. Подобная ситуация создаёт предпосылки к нарушению штатного функционирования программного обеспечения за счёт возможного пересечения данных стека и «кучи» (рис. 3.3а).

Модель **small** — модель распределения памяти, в рамках которой под область кода и область данных (в том числе стека и «кучи») выделяются два отдельных сегмента. Кроме того, в данной модели доступна память,

расположенная за сегментом данных и занимающая все оставшееся адресное пространство. Это область получила название «дальняя куча» (**far heap**). По аналогии с этим названием, «куча» внутри сегмента данных получила название «ближняя куча» (**near heap**) (рис. 3.3b).

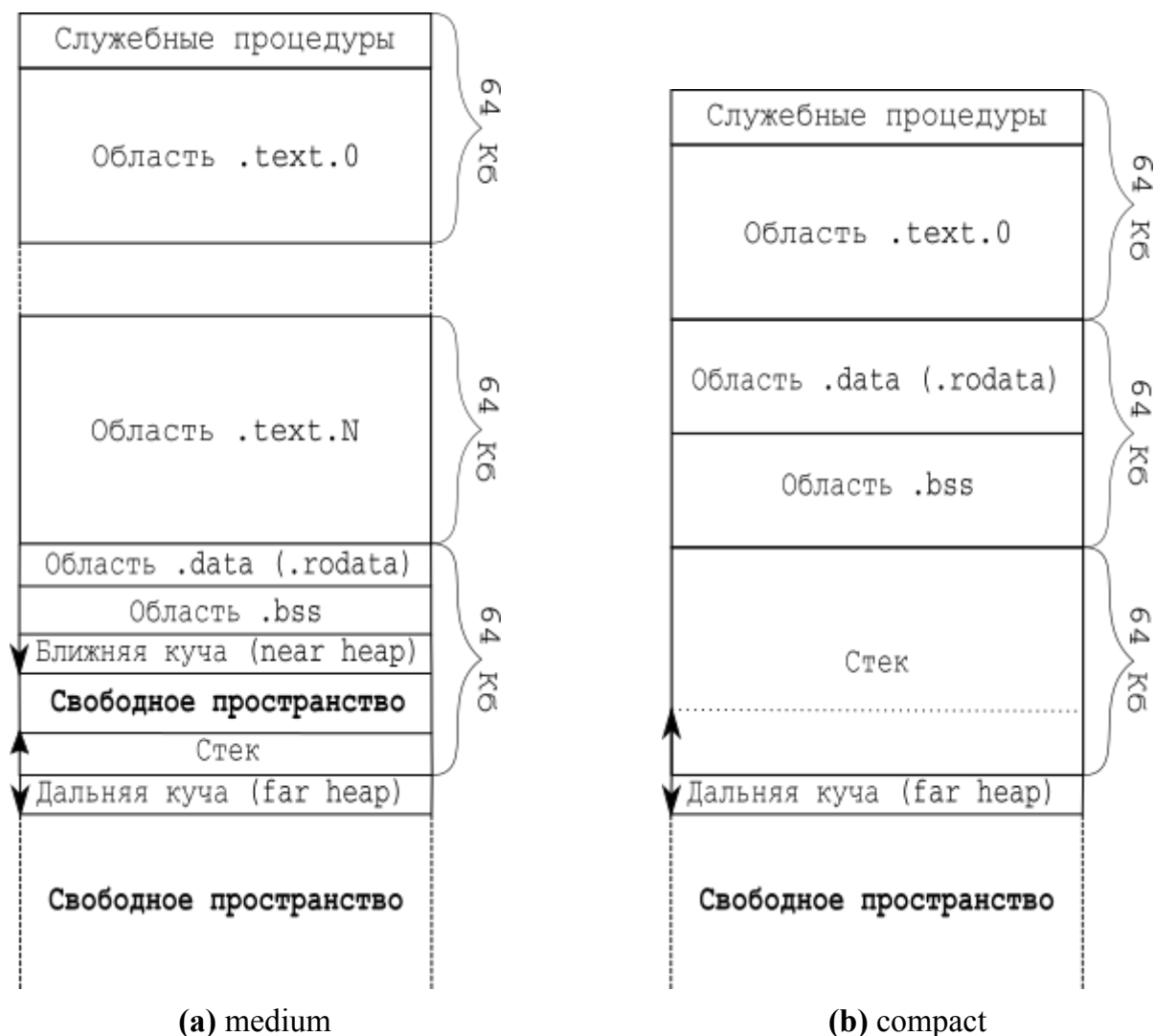


Рисунок 3.4. Модели памяти medium и compact

Модель памяти **medium** — модель памяти, похожая на модель small, однако, в ней предусмотрено несколько сегментов для хранения команд. Указанная модель используется для программ с большим количеством программного кода и небольшим объёмом статической памяти.

Модель памяти **compact** — модель распределения памяти, в рамках которой под область кода, область данных и стек выделяются три отдельных сегмента. При этом, в отличие от предыдущих моделей, в указанной модели отсутствует «ближняя куча».

Модель памяти **large** используется при построении очень больших программ, с большим объёмом статических данных, но не превышающим 64 Кб.

По умолчанию доступ, как к подпрограммам, так и к данным осуществлялся через «дальний указатель», т.е. при обращении используется как смещение, так и адрес сегмента. Это, естественно, приводит к замедлению выполнения программы. По своей сути модель похожа на модель памяти *medium*, с той лишь разницей, что в данной модели предполагается только одна «куча», а статические данные и стек разнесены в отдельные сегменты.

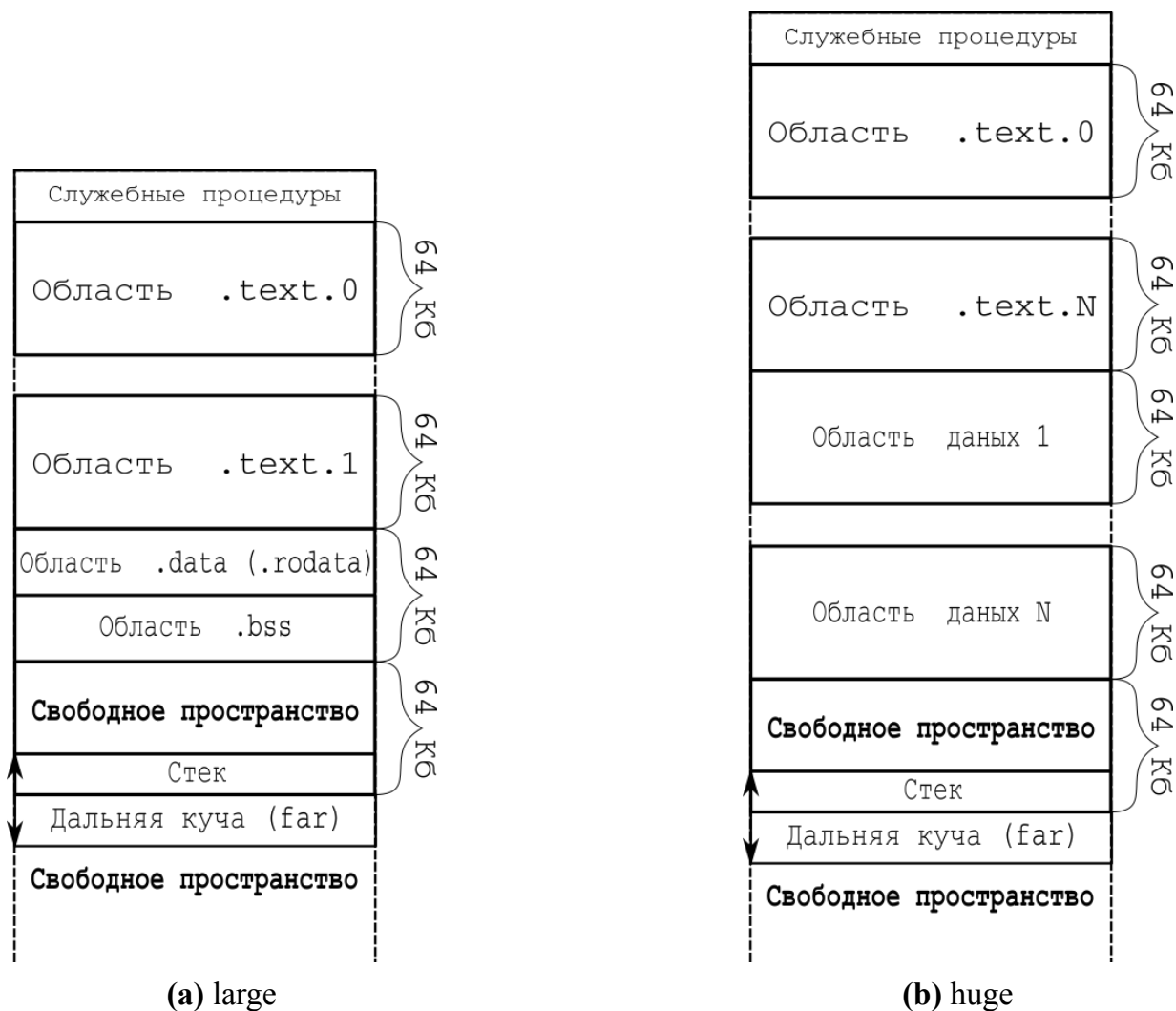


Рисунок 3.5. Модели памяти large и huge

Модель памяти **huge** даёт возможность использовать статические данные объёмом более 64 Кб. Это достигается за счёт модификации модели large путём добавления нескольких сегментов данных.

Указанное разнообразие моделей, в первую очередь, связано с невозможностью адресоваться с использованием регистров общего назначения к большому объёму памяти. В связи с тем, что современные процессоры имеют регистры общего назначения, которые лишены указанного недостатка, в настоящее время наиболее популярной является модель памяти **flat**.

Модель памяти flat в основном совпадает с моделью tiny. Единственным отличием является размер сегмента памяти, в котором располагается программа. Для модели flat размер сегмента равен 2^{32} байт (4 Гб) в 32-х разрядных системах и 2^{64} (16 Эб^{18}) байта — в 64-х разрядных.

Распределение сегментов памяти можно продемонстрировать посредством программы из листинга 9. В строке 8 на экран выводится адрес в стеке, где размещается локальная переменная *a*, а в строке 9 выводится адрес в «куче», первый доступный для распределения. Не сложно заметить, что адрес в стеке значительно превышает адрес объекта в «куче».

Листинг 9. Программный код, демонстрирующий границы «кучи» и области стека процесса

```
1 #include <stdlib.h>
2 #include <stdio.h>
3
4 int main(int argc, char* argv)
5 {
6     char *a;
7     a = malloc(1);
8     printf("stack: 0x%08X (%d)\n", &a, &a);
9     printf("heap: 0x%08X (%d)\n", a, a);
10    free(a);
11    return 0;
12 }
```

3.5. Многозадачность

Одной из основных технологий, позволившей коренным образом увеличить эффективность эксплуатации вычислительных систем, стала *многозадачность*

Многозадачность — технология, позволяющая выполнять несколько процессов параллельно (псевдопараллельно).

В случае, если речь идёт о многозадачных системах, где каждому процессу в монопольное пользование передается собственный вычислитель, процессы полностью выполняют свою задачу от начала до конца.

В случае систем с разделением времени, возникает задача приостановки и возобновления работы процессов. При этом выделяются два типа псевдомногозадачности, реализуемых за счет механизма разделения времени:

- **кооперативная (невытесняющая)** — процесс сам принимает решение, в какой момент времени следует приостановить свою работу;
- **вытесняющая** — решение о приостановки процесса принимает операционная система.

¹⁸Приставка «экса» (сокр. — «Э») обозначает десятичный множитель 10^{18} [20].

В случае приостановки процесса операционная система сохраняет его текущее состояние, необходимое для последующего возобновления работы. Совокупность этих данных называется **контекстом процесса**.

Контекст процесса может быть разделен на две составляющие:

- **аппаратный** – данные о состоянии процессора в момент приостановки его выполнения;
- **программный** – данные, хранящиеся во внутренних структурах на уровне операционной системы и ассоциированные с процессом.

В дополнение...

Для процессоров Intel x86 аппаратный контекст называется «сегмент состояние задачи» (TSS – task segment station)[11].

К данным программного контекста относятся: состояние процесса, используемые ресурсы и блокировки, карта памяти, права доступа и т.д.

3.6. Потоки

Развитием технологии многозадачности стала технология **многопоточности**.

Многопоточность – технология, позволяющая формировать программу из фрагментов кода (наборов команд), которые могут выполняться одновременно (параллельно).

Поток – фрагмент кода (набор команд) процесса, который может быть выполнен одновременно (параллельно) с набором остальным команд процесса.

Каждый поток по своей структуре имеет сходство с процессом, т.е. обладает своим набором локальных данных, стеком и контекстом (рис. 3.6). При этом все потоки существуют в общем адресном пространстве процесса, в котором они функционируют, по этой причине данные процесса доступны для каждого потока.



Рисунок 3.6. Структура процесса

Независимо от способа создания потоков, в ОС может присутствовать одна из трёх моделей:

- поток уровня пользователя;
- поток уровня ядра;
- гибридная модель потока.

В дополнение...

Для некоторых ОС выбор модели потока невозможен, поскольку она уже заложена внутри системы. В большей степени это касается ОС, где нет реализации потоков уровня ядра. Для иных систем, выбор зависит от разработчика и библиотек, которые он использует. Например, библиотека `µlibc` при своей компиляции позволяет указать модель потока, которые будут использоваться при работе.

Модель потока уровня пользователя предполагает, что на уровне операционной системы процесс представляется в качестве единого объекта и не разделяется на отдельные потоки (рис. 3.7).

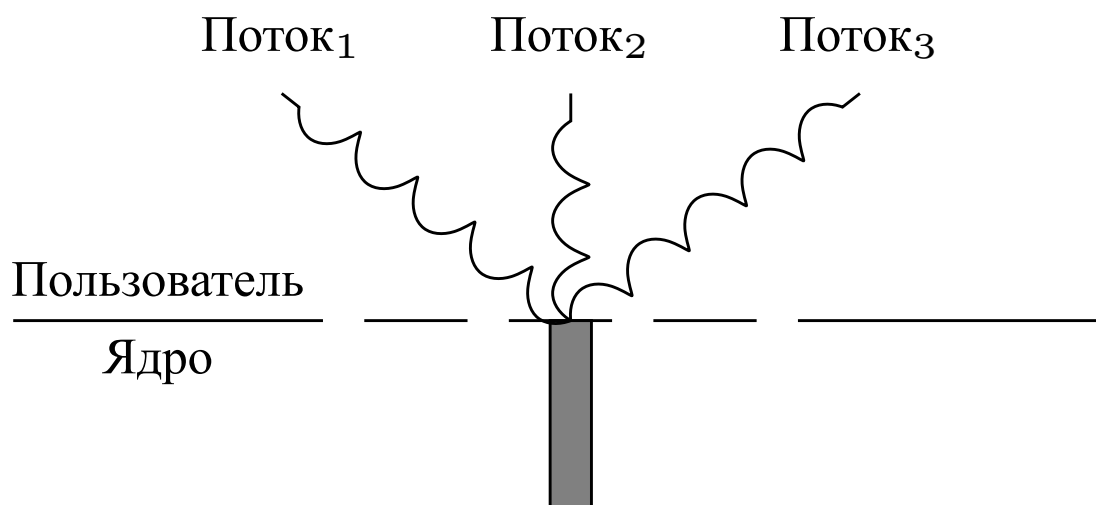


Рисунок 3.7. Модель потока уровня пользователя

Операционная система выделяет квант времени для процесса, который впоследствии самим процессом разделяется между потоками.

Преимуществом потоков уровня пользователя является гибкость планирования и достигаемая за счёт этого скорость выполнения.

В дополнение...

Гибкость планирования достигается за счёт более эффективного распределения времени между потоками. Т.е. время будет выделяться именно тому потоку, выполнение которого наиболее критично для всего процесса. Таким образом, достигается общая эффективность работы всей вычислительной системы.

Это похоже на работу хорошо организованной команды, ограниченной в ресурсах. Например, бригады механиков на трассе Формулы 1 в момент технической остановки болида.

Модель потока уровня ядра предполагает, что операционная система рассматривает каждый поток в виде упрощённой модели процесса. Единственным отличием остаётся только распределение памяти – для потоков не выделяется отдельное адресное пространство (рис. 3.8).

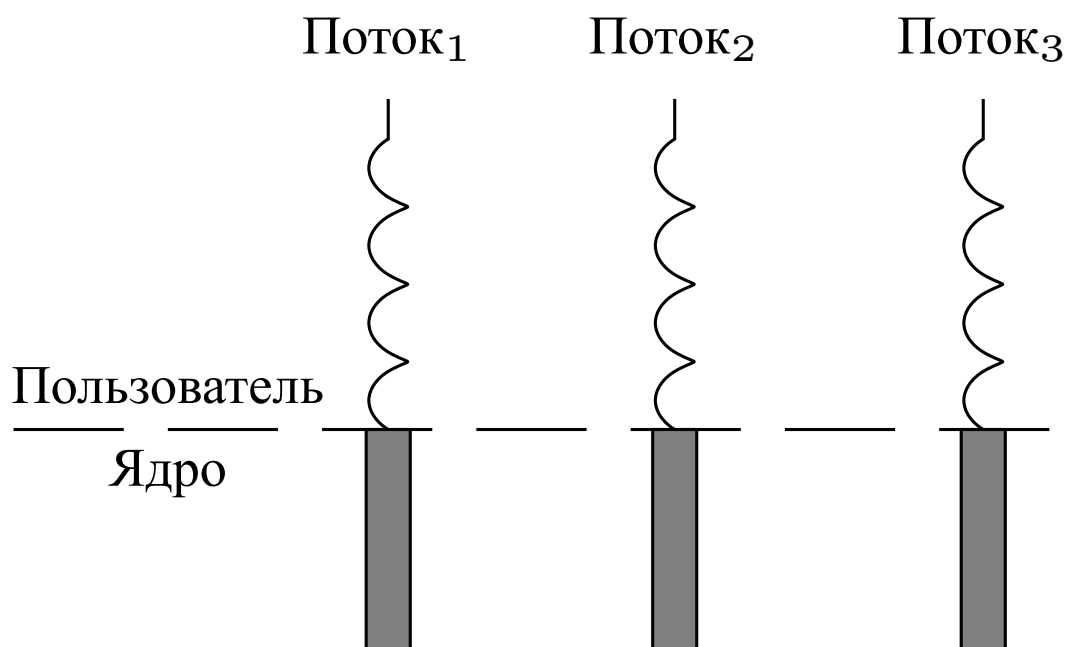


Рисунок 3.8. Модель потока уровня ядра

В рамках указанной модели для каждого потока выделяется собственный квант времени. Такой подход позволяет упростить реализацию потоков на уровне пользователя, однако снижает гибкость планирования, поскольку система «не знает» о задачах, решаемых потоком, и их критичности в конкретный момент для всего процесса.

Гибридная модель потока позволяет использовать преимущества обеих моделей, позволяя разработчику лавировать между эффективностью планирования потоков и количеством квантов времени, выделяемых на процесс (рис. 3.9).

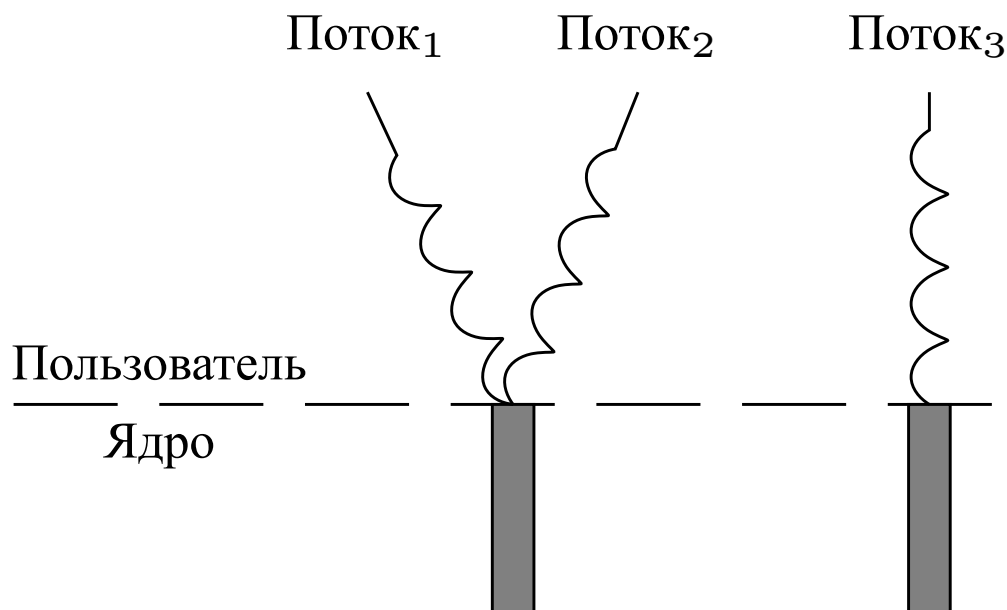


Рисунок 3.9. Гибридная модель потока

3.7. Жизненный цикл процесса (потока)

3.7.1. Общая модель

Процесс функционирования любого объекта предполагает некоторый набор состояний, между которыми осуществляется переход на протяжении всего периода существования этого объекта.

Для процессов (равно как и для потоков) выделяются три основных состояния (рис. 3.10):

- **готовность** – состояние, в котором процесс ждет своей очереди быть запущенным на исполнение и обладает всеми необходимыми ресурсами;
- **выполнение** – состояние, в котором процесс выполняется;
- **блокировка** – состояние, в котором процесс по каким-либо причинам не может быть выполнен. Например, ожидание ресурсов или принудительная блокировка.

После создания и инициализации процесс переходит в состояние *готовности*. В момент, когда до него доходит черед, система переводит его в состояние *выполнение*, где он работает отведенный ему промежуток времени. Если в момент выполнения процесс осуществляет запрос к некоторому ресурсу, который не может быть сразу предоставлен, либо инициирует свою блокировку, он переводится в состояние *блокировка*. В противном случае, после добровольного или принудительного¹⁹ освобождения процессора, процесс переходит обратно в состояние *готовность*.

¹⁹Например, в результате исчерпания отведенного процессу кванта времени

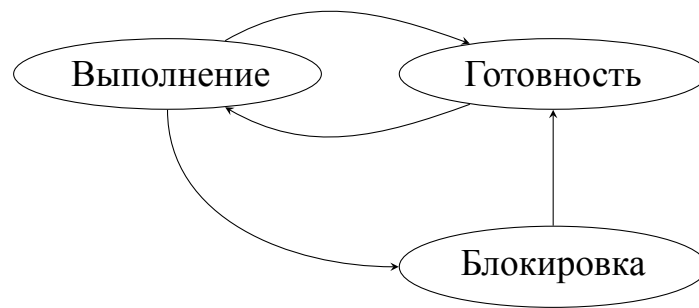


Рисунок 3.10. Общая модель жизненного цикла

3.7.2. Жизненный цикл в Linux

В качестве частного примера жизненного цикла процессов рассмотрим жизненный цикл в операционных системах Linux (рис. 3.11) [21].

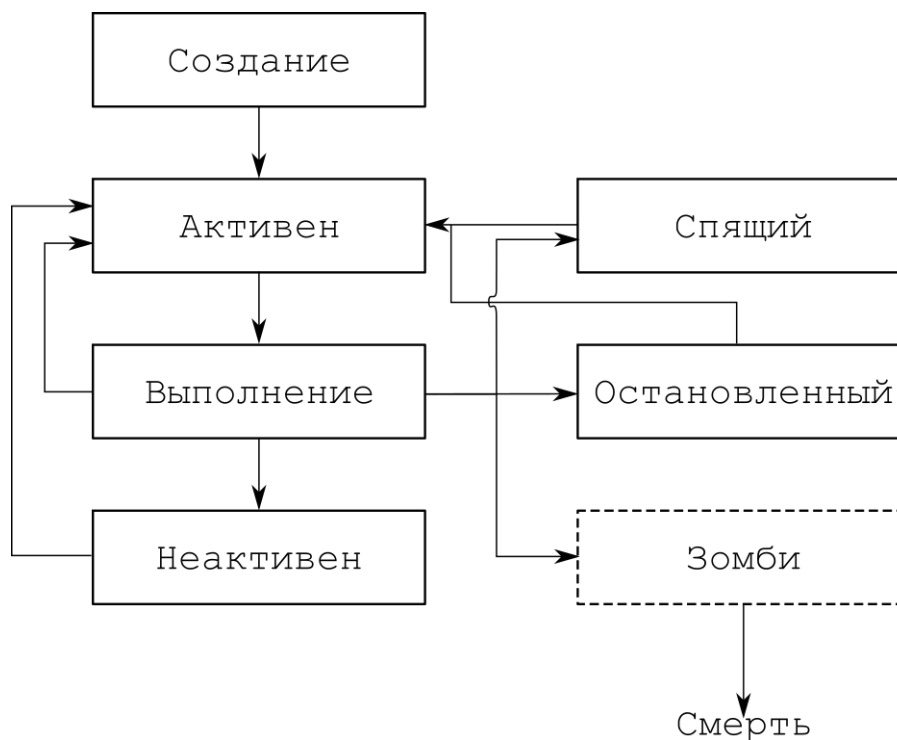


Рисунок 3.11. Модель жизненного цикла в Linux

В рамках данной модели состояние *готовности* может быть ассоциировано с подсостояниями *активен* и *неактивен*. Смысл данных состояний будет подробнее пояснен в п. 6.5.6.

Состояние *блокировка* реализуется в рамках двух отдельных состояний:

- **спящий** – процесс, добровольно перешедший в режим блокировки (например, с использованием системного вызова `pause()`);
- **остановленный** – процесс, заблокированный операционной системой (например, при запросе данных с клавиатуры).

Состояние **зомби** используется в качестве переходного состояния перед завершением процесса. В данном состоянии процесс завершил свое выполнение, но находится в памяти системы. В таком состоянии процесс находится до тех пор, пока **родительский процесс**²⁰ не запросит информацию о причинах завершения этого процесса. Запрос производится с использованием системного вызова `wait`.

В дополнение...

В тех случаях, когда процесс активно создает дочерние процессы, но не отрабатывает корректно их завершение, возможно ситуация при которой в систем появится большое число «процессов-зомби». Основания угроза от подобной ситуации заключается в том, что каждому процессу присваивается идентификатор (`pid`), число которых ограничено в системе. Следовательно, при неконтролируемом увеличении числа «процессов-зомби», указанные идентификаторы могут закончиться и ОС не сможет создавать новые процессы.

Для минимизации ущерба от подобной ситуации можно наложить ограничение на максимально допустимое число дочерних процессов. А для уничтожения большого числа неконтролируемы «процессов-зомби» достаточно завершить родительский процесс. В этом случае, все дочерние процессы будут преданы «родителю родителя», в котором уже может быть реализовано корректное завершение дочерних процессов.

3.8. Вопросы для самопроверки

1. Показать, что определения, введенные в п. 3.1, являются тождественными соответствующим определениям, приведенным в государственных стандартах Российской Федерации [4].
2. Реализуйте программный модуль, который будет отображать распределение адресов собственных секций.
3. Приведите примеры задач, при решении которых целесообразно использовать процессы.
4. Приведите примеры задач, при решении которых целесообразно использовать потоки.
5. Каждому состоянию из жизненного цикла процесса в Linux поставьте в соответствие состояние из общей модели жизненного цикла процесса. Обоснуйте.

²⁰Родительским процессом по отношению к некоторому процессу называется процесс, который его создал. Созданный процесс называется **дочерним**.

4. ОПЕРАЦИИ НАД ПРОЦЕССАМИ И ПОТОКАМИ. ВЗАИМОИСКЛЮЧЕНИЯ

4.1. Создание процессов и потоков

В ходе работы как у процессов, так и у потоков возникает необходимость обмена какой-либо информацией между собой. При этом для потоков организация подобного взаимодействия не вызывает трудностей, в связи с тем, что потоки располагаются в общем адресном пространстве и имеют доступ к общим глобальным переменным. В случае с процессами это невозможно. Для организации взаимодействия процессов используются специальные механизмы, получившие название **механизмов межпроцессного взаимодействия (ИРС)**.

Выделяются две основные цели использования механизмов ИРС.

1. Передача данных (в т.ч. оповещение о событиях).
2. Синхронизация доступа процессов к ресурсам (взаимоисключения).

Следует отметить, что речи не идет о выделении двух взаимоисключающих целей – достижение одной цели может автоматически приводить к достижению другой. Далее будут рассмотрены основные механизмы, позволяющие достичь каждую из целей.

Однако, прежде чем перейти к рассмотрению механизмов ИРС, следует рассмотреть ряд вспомогательных примеров, а именно примеров создания процесса и потока. На основе указанных примеров, будет построено описание возможного использования механизмов ИРС.

В первую очередь рассмотрим пример создание процесса (листинг 10).

Листинг 10. Создание нового процесса в ОС Linux

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(int argc, char* argv[])
5 {
6     pid_t pid;
7     int result;
8     if ( (-1) == (pid = fork()) ) {
9         perror("fork");
10    } else if ( pid == 0 ) {
11        printf("Сообщение_от_потомка\n");
12    } else {
13        printf("Сообщение_от_родителя\n");
14        wait(&result);
15        printf("Код_завершение_процесса_%d=_%d\n", pid, WEXITSTATUS
            (wstatus));
16    }
17    return 0;
18 }
```

Классическим способом создания процесса в Linux является использование функции `fork` (строка 7). В случае ошибки функция возвращает значение «-1». Результатом успешной работы функции является создание практически полной копии процесса, в котором она была вызвана. Единственным отличием двух процессов будет являться код возврата процедуры `fork`. Для дочернего процесса он будет равен нулю. В родительском процессе – идентификатору созданного процесса. В связи с этим строка 11 будет выполнена в дочернем процессе, а строка 13 – в родительском. Перед завершением работы родительский процесс блокирует свое выполнение и ожидает завершения дочернего процесса (строка 14).

В ходе работы контекст любого процесса можно заменить на контекст какого-то иного приложения. Для этих целей в Linux существует класс процедур `exec`. В случае успешного выполнения процедуры указанного класса, текущий процесс полностью заменяется на процесс созданный на основе программы, указанной в качестве параметра процедуры (листинг 11).

Листинг 11. Замена контекста процесса в ОС Linux

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main(int argc, char* argv[], char* env[])
5 {
6     char* args[3] = {"/bin/ls", "-a", NULL};
7     execve("/bin/ls", args, env);
8     perror("execve");
9     return -1;
10 }
```

Таким образом, для запуска новой программы используется двухэтапный подход. На первом этапе создается новый процесс, а на втором – происходит замена контекста текущего процесса на требуемый. Такой, на первый взгляд сложный, подход необходим для обеспечения возможности настройки дочернего процесса до замены его контекста. В следующем разделе при рассмотрении механизмов IPC в листинге 14 будет приведен соответствующий пример.

Создание потока происходит менее экзотично, с использованием процедуры `pthread_create`. Данная процедура позволяет создать поток на основе специально подготовленной подпрограммы, называемой **поточковой процедурой**. Пример создания потока приведен в листинге 12.

Листинг 12. Создание нового потока в ОС Linux

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 struct thread_args {
5     int val1;
```

```

6   int val2;
7 };
8 static void* thread_func(void *arg)
9 {
10  struct thread_args *par;
11  par = (struct thread_args*) arg;
12  printf("%s: val1=%d, val2=%d\n",
13         __func__, par->val1, par->val2);
14  return NULL;
15 }
16 int main(int argc, char* argv[])
17 {
18  pthread_t pid;
19  struct thread_args args = {.val1=1, .val2=2};
20  void *retval;
21  pthread_create(&pid, NULL, thread_func, (void*)&args);
22  printf("Message from main thread\n");
23  pthread_join(pid, &retval);
24  return 0;
25 }

```

Прототип потоковой процедуры предполагает, что она должна принимать в качестве входного параметра указатель на область памяти, в которой располагаются все исходные данные для ее выполнения (строка 8). В данном примере в качестве исходных данных предполагается использовать две целочисленные переменные, которые сгруппированы в качестве полей структуры описанной в строке 4.

Таким образом, при вызове процедура создание потока (строка 21) в качестве входных параметров ей передаются:

- указатель на переменную, в которую будет записан идентификатор созданного потока (`pid`);
- дополнительные атрибуты создаваемого потока (в данном примере они не используются);
- адрес потоковой процедуры (`thread_func`);
- «значение» входного параметра потоковой процедуры (`&args`).

Как и в случае с процессами, основной поток должен дожидаться завершения выполнения созданного потока и получить код его завершения (строка 23).

Следует заметить, что существуют иные способы создания процессов и потоков, доступные разработчику программного обеспечения. Однако, для дальнейшего изложения материала описанных выше подходов будет достаточно.

4.2. Механизмы передачи данных и оповещения о событиях

4.2.1. Файлы

Наиболее очевидным механизмом обмена данными между процессами²¹ является использование **файлов**. Данный механизм предполагает, что один процесс осуществляет запись данных в файл, а второй эти данные читает (листинг 13).

Листинг 13. Обмен данными между процессами с использованием файлов в ОС Linux

```
1 #include <fcntl.h>
2 #include <stdio.h>
3 #include <unistd.h>
4 #include <sys/wait.h>
5 #define F_NAME "/tmp/child.pid"
6 #define W_FLAGS (O_CREAT|O_TRUNC|O_WRONLY)
7 #define R_FLAGS (O_RDONLY)
8 #define O_MODE (S_IRUSR|S_IWUSR)
9 int main(int argc, char* argv[])
10 {
11     pid_t pid;
12     int fd;
13     if ( (-1) == (pid = fork()) ) {
14         perror("fork");
15     } else if ( pid == 0 ) {
16         fd = open(F_NAME, W_FLAGS, O_MODE);
17         pid = getpid();
18         write(fd, &pid, sizeof(pid));
19     } else {
20         wait(NULL);
21         fd = open(F_NAME, R_FLAGS);
22         read(fd, &pid, sizeof(pid));
23         printf("Идентификатор: %u\n", pid);
24     }
25     close(fd);
26     return 0;
27 }
```

Несмотря на простоту механизма, следует отметить, что он позволяет организовать взаимодействие процессов, функционирующих на различных СВТ. Такая возможность появляется, например, при размещении файла в сетевом хранилище, доступном для каждого СВТ. Кроме того, файлы позволяют долгое время хранить записанную в них информацию и многократно обращаться к ней.

К недостаткам файлов можно отнести низкую скорость работы (файловые операции ограничены скоростью записи/чтения на носитель информации), а

²¹При дальнейшем изложении материала понятия процесс и поток будут расцениваться как тождественные, если не будет явно указано иное.

также возможностью доступа к записанным данным тех процессов, для которых эти данные не предназначались, в том числе после удаления файла.

4.2.2. Каналы

В тех случаях, когда требуется организовать оперативный обмен информацией между процессами в рамках единой системы, то для исключения вышеописанных недостатков следует использовать *каналы*.

Канал — механизм межпроцессного взаимодействия, формирующий прямой поток данных между двумя процессами [17].

Выделяют два вида каналов:

- **именованные** — каналы, доступ к которым можно получить с использованием глобальных имен, присвоенных им при создании.
- **неименованные** (анонимные) — каналы, доступ к которым имеют исключительно процессы, их создавшие, а также их дочерние процессы.

Одним из распространенных способов использования каналов является **конвейеризация процессов**. Суть конвейеризации заключается в возможности передачи данных из стандартного потока вывода одного процесса в качестве входных данных стандартного потока ввода для другого процесса (рис. 4.1).

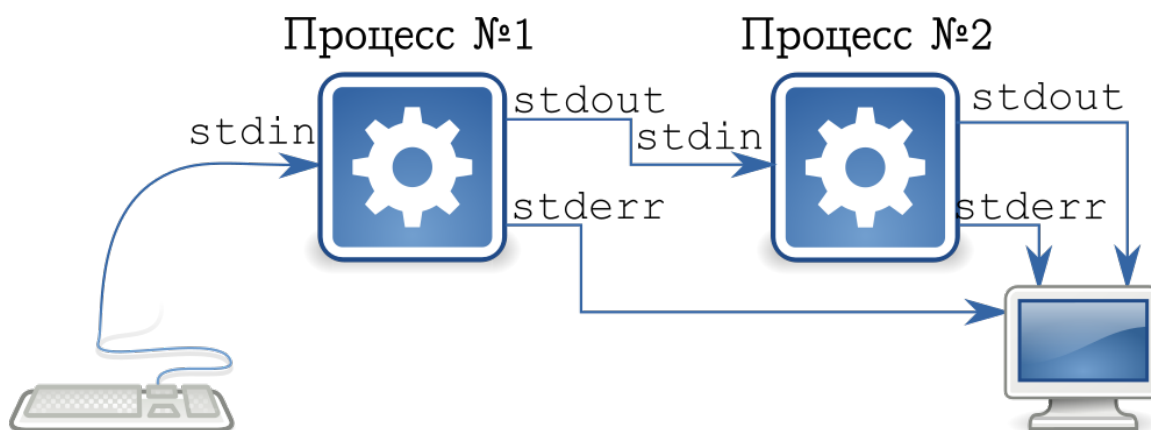


Рисунок 4.1. Схема объединения процессов в конвейер

Например, выполнение команды, представленной ниже, приведет к тому, что на экран будет выведен список всех файлов с расширением *.txt в текущей директории (учитывая все вложенные), упорядоченный по возрастанию.

```
# find -iname '*.txt'|sort
```

Такая возможность достигается за счет подмены дескрипторов стандартных потоков ввода-вывода процесса на соответствующие дескрипторы каналов, например с использованием процедуры dup2.

В листинге 14 приведен пример создания конвейера с использованием анонимного канала.

Листинг 14. Пример создание конвейера в ОС Linux

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/wait.h>
4 int main(int argc, char* argv[], char* env[])
5 {
6     pid_t pid;
7     int pipefd[2];
8     char* args[2] = {"/usr/bin/sort", NULL};
9     if ( (-1) == pipe(pipefd) ) {
10         perror("pipe");
11         return __LINE__;
12     }
13     if ( (-1) == (pid = fork()) ) {
14         perror("fork");
15     } else if ( pid == 0 ) {
16         close(pipefd[1]);
17         close(0);
18         dup2(pipefd[0], 0);
19         execve(args[0], args, env);
20         perror("execve");
21     } else {
22         close(pipefd[0]);
23         close(1);
24         dup2(pipefd[1], 1);
25         printf("Str_#3\n");
26         printf("Str_#2\n");
27         printf("Str_#1\n");
28         close(pipefd[1]);
29     }
30     return 0;
31 }
```

В строке 9 создается канал, доступ к которому осуществляется с использованием двух дескрипторов – `pipefd[0]` и `pipefd[1]`, используемых для чтения и записи, соответственно. В строке 18 закрывается текущий дескриптор стандартного потока²² ввода дочернего процесса, а в строке 19 в качестве стандартного потока ввода назначается соответствующий дескриптор созданного ранее канала. В строках 23, 24 производятся аналогичные операции для стандартного потока вывода родительского процесса. После завершения всех операций, строки, выводимые процедурой `printf`, вместо отображения на экране будут переданы дочернему процессу. Дочерний процесс, в контексте которого будет запущена программа `sort`, осуществит сортировку строк и самостоятельно отобразит их на экран.

²²В данном случае речи идет о потоках ввода-вывода.

4.2.3. Сокеты

Как было отмечалось выше, каналы используются для организации обмена данными в рамках локальной системы. Для организации оперативного взаимодействия процессов, выполняющихся на различных СВТ в сети, следует использовать **сокеты**.

Работа с сокетами, во многом похожа на работу с файлами и каналами. Исключением является то, что данные, записанные в сокет, передаются по сети в соответствии с конфигурацией сокета и читаются из сети иными процессами также посредством сокетов. Наиболее часто используемая конфигурация сокетов позволяет производить обмен между двумя СВТ с использованием стека протоколов TCP/IP.

При этом выделяется особый класс сокетов – «сырые сокеты» (raw socket), при использовании которых формирование и разбор специальных структур данных полностью ложатся на плечи разработчика. Сокет в этом случае является исключительно средством передачи данных в канал связи и чтения всего набора данных из сети [18].

В дополнение...

В ряде ОС используется комбинированный подход к организации IPC, позволяющий производить обращение к именованным каналам ОС из сети. Зачастую, с использованием именно удаленного доступа к каналам реализуются компьютерные атаки на ОС Windows. Так, использование особенностей реализации удаленного доступа к каналу \$IPC позволило создать средства распространения вредоносного программного обеспечения по протоколу SMB, которые послужили инструментами для организации «эпидемий» WannaCry и NotPety.

4.2.4. Сигналы

Следующим и наиболее часто используемым в ОС Linux механизмом IPC являются **сигналы**.

Сигнал – вырабатываемое программным обеспечением сообщение, информирующее о наступлении определенного события либо возникновении ошибки. Сигналы не позволяют передавать какие-либо данные [17].

В том случае, когда для процесса предусмотрен обработчик сигнала, который по сути является функцией обратной связи (*callback function*), ОС передает ему управление. Когда же обработчик отсутствует, ОС использует собственный обработчик. В листинге 15 приведен пример обработчика для сигнала SIGINT в ОС Linux.

Листинг 15. Обработка сигнала в Linux

```
1 #include <stdio.h>
2 #include <signal.h>
3 #include <unistd.h>
4
5 int work_flag = 1;
6 void interrupt(int num)
7 {
8     work_flag = 0;
9 }
10 int main(int argc, char *argv[])
11 {
12     signal(SIGINT, interrupt);
13     while(work_flag) {
14         printf("Тик\n");
15         sleep(1);
16     }
17     printf("Завершение\n");
18     return 0;
19 }
```

Сигнал SIGINT генерируется в случае нажатия пользователем сочетания клавиш **Ctrl** + **C**. При отсутствии в процессе собственного обработчика данного сигнала, ОС передает управление процедуре принудительного завершения процесса.

Следует отметить, что не для каждого сигнала в процессе можно реализовать обработчик. Сигналы, которые невозможно обработать в процессе называются *неперехватываемые*.

Сигналы неявным образом генерируются в результате работы различных процедур и утилит. Для передачи процессу произвольного сигнала в ОС Linux используется системный вызов `kill`, либо одноименная утилита.

Недостатком сигналов является отсутствие возможности передачи дополнительной информации о событии, которое привело к генерации сигнала.

4.2.5. Сообщения

Результатом развитием механизма сигналов, стало появления механизма, основанного на передаче **сообщений**.

Реализация данного механизма межпроцессного взаимодействия предполагает, что для каждого процесса должна быть сформирована очередь, либо несколько очередей, в которые ОС будет добавлять сообщения, получаемые от различных процессов. Процесс-получатель поочередно извлекает сообщения из очереди, анализирует их и принимает решения по их дальнейшей обработке.

Работа с очередью может осуществляться как в *инициативном* (с точки зрения процесса) порядке, так и *по факту поступления* сообщений. Для

обработки сообщения «по мере поступления», процесс, по аналогии с обработкой сигналов, должен «зарегистрировать» обработчик сообщений.

Механизм сообщения получил широкое распространение в различных ОС. Например, при создании графических приложений в ОС Windows предполагается, что при каждой манипуляции с окном процессу-владельцу окна, отсылается соответствующее сообщение, которое оно должно обработать. Именно по этой причине, если процесс в ОС Windows «зависает», перетаскивание окна приложения по рабочему столу может проходить с задержками. Кроме того, даже передвижение курсора мыши в области окна «задумавшегося процесса» повлечет за выполнение собой большого числа операций, которые могли бы пойти на завершение текущей задачи.

В дополнение...

Механизм сообщений активно используется не только приложениями пользовательского уровня. Микроядерные системы используют сообщения в качестве основного механизма взаимодействия между компонентами ОС. С одной стороны, это усложняет реализацию процесса взаимодействия в части согласования времени передачи сообщений от различных источников (определения того, какое сообщение «пришло» раньше, а какое позже), а с другой стороны – позволяет в значительной мере сделать процесс взаимодействия модулей ОС более универсальным, а сами модули независимыми друг от друга.

4.2.6. Разделяемая память

Разделяемая память – это механизм межпроцессного взаимодействия, позволяющий отобразить в адресное пространство различных процессов общий участок оперативной памяти. Подробнее указанный механизм будет рассмотрен в п. 8. Использование данного механизма позволяет оптимизировать процесс пересылки больших объемов данных, а также исключить необходимость согласования содержимого массивов данных, обработка которых производится посредством нескольких процессов. Следует обратить внимание, что указанный механизм по-умолчанию доступен всем потокам одного процесса, поскольку они функционируют в едином адресном пространстве. Эта особенность, дает преимущество многопоточным программам, исключая необходимость организации общего адресного пространства с использованием дополнительных манипуляций со стороны операционной системы.

4.2.7. Почтовый ящик

Наиболее экзотичный механизм межпроцессного взаимодействия — это **почтовый ящик**. Данный механизм основывается на использовании для хранения и передачи данных электронных сообщений *e-mail*, причем как на

локальных машинах (например, с использованием утилиты mail), так и с использованием соответствующих серверов электронной почты в сети Интернет.

В дополнение...

В 1999 году была выпущена игра под названием X-COM: First Alien invasion. Она представляла собой пошаговую тактическую игру. Два игрока, желавшие сыграть партию, указывали в игре аутентификационные данные собственной почты и адрес партнера. После этого они могли играть. Игровой процесс сводился к тому, что игрок совершал ряд манипуляций своим персонажем и указывал, что его ход завершен. В этот момент информация о его ходе посредством электронной почты передавалась второму игроку. При запуске игры вторым игроком, программа в автоматическом режиме проверяла соответствующий почтовый ящик, применяла на стороне второго игрока все действия первого, после чего второй игрок вступал в игру.

Одна партия в эту игру могла длиться несколько дней и напоминала игру в шахматы по переписке.

4.3. Механизмы взаимного исключения

4.3.1. Проблема асинхронного выполнения задач

Используя описанные выше методы, можно обеспечить совместную работу нескольких процессов (потоков) с общим ресурсом. Однако, как отмечалось в п.1.2, подобная работа чревата негативными последствиями. Рассмотрим пример. В листинге 16 приведен код программы, в которой два потока модифицируют общую глобальную переменную `ret` (строка 4). Первый поток `COUNT`²³ раз увеличивает ее значение на единицу (строка 9), а второй поток – `COUNT` раз уменьшает на единицу (строка 16). Данная программа является реализацией классической задачи «потребитель-производитель».

Листинг 16. Задача «потребитель-производитель»

```
1 #include <pthread.h>
2 #include <stdio.h>
3 #define COUNT (5000000)
4 volatile unsigned int ret =0;
5 static void* func1(void* arg )
6 {
7     int i;
8     for ( i = 0 ; i < COUNT ; i++ )
9         ret++;
10    return NULL ;
11 }
```

²³Значение константы `COUNT` является субъективным для каждого процессора и в случае, если при выполнении программы получится верный результат, то достаточно просто увеличить на порядок значение данной константы.

```

12 static void* func2(void* arg )
13 {
14     int i;
15     for ( i=0 ; i < COUNT ; i++ )
16         ret--;
17     return NULL ;
18 }
19
20 int main(int argc, char* argv[])
21 {
22     pthread_t thread;
23     void* retv ;
24
25     printf ("COUNT=0x%X\n",COUNT);
26     pthread_create(&thread,NULL,func1, NULL);
27     func2( NULL );
28     pthread_join ( thread ,& retv ) ;
29     printf ( "RET=0x%X\n", ret ) ;
30     return ret ;
31 }

```

Основываясь на элементарных познаниях математики, можно говорить о том, что в результате выполнения обоих потоков в переменной должно находиться исходное значение, т.е. 0. Однако, по результатам десяти запусков данной программы, указанного результата получено не было (рис. 4.2).

```

# count=10 ; while [ "$count" != "0" ] ; do \
> /tmp/1.exe ; let count=count-1 ; done
COUNT=0x4C4B40      COUNT=0x4C4B40
RET=0xB7796000        RET=0xB7745000
COUNT=0x4C4B40      COUNT=0x4C4B40
RET=0xB7796000        RET=0xB7759000
COUNT=0x4C4B40      COUNT=0x4C4B40
RET=0xB7738000        RET=0xB775F000
COUNT=0x4C4B40      COUNT=0x4C4B40
RET=0xB76D7000        RET=0xB7708000
COUNT=0x4C4B40      COUNT=0x4C4B40
RET=0xB7712000        RET=0xB772D000

```

Рисунок 4.2. Результат многократного выполнения программы из листинга 16

Для пояснения странного поведения программы следует рассмотреть дизассемблированный код обеих процедур (рис.4.3). Не сложно заметить, что их код практически идентичен, за исключением наименования меток переходов и выделенных инвертированным цветом строк. В данных строках реализуется увеличение и уменьшение на единицу значения, размещенного в регистре `eax`.

Строкой ранее в данный регистр записывается значение из переменной `ret`, а в последующей за выделенной строкой – из регистра обратно в переменную. Таким образом, модификация переменной `ret` осуществляется последовательным выполнением трех машинных команд. Если при выполнении набора данных команд произойдет вытеснение потока и иной поток модифицирует значение переменной, то после возобновления своей работы первый поток продолжит свое выполнение, имея «устаревшее значение» переменной.

<pre> ;***** ; function func1 (local) ;***** func1: push ebp mov ebp, esp sub esp, 10h mov dword ptr [ebp-4], 0 jmp loc_8048588 loc_8048577: mov eax, [ret] add eax, 1 mov [ret], eax add dword ptr [ebp-4], 1 loc_8048588: cmp dword ptr [ebp-4], 4c4b3fh jng loc_8048577 mov eax, 0 leave ret </pre>	<pre> ;***** ; function func2 (local) ;***** func2: push ebp mov ebp, esp sub esp, 10h mov dword ptr [ebp-4], 0 jmp loc_80485b8 loc_80485a7: mov eax, [ret] sub eax, 1 mov [ret], eax add dword ptr [ebp-4], 1 loc_80485b8: cmp dword ptr [ebp-4], 4c4b3fh jng loc_80485a7 mov eax, 0 leave ret </pre>
---	---

Рисунок 4.3. Дизассемблированный код процедур из листинга 16

Таким образом, решение задачи «производитель-потребитель» сводится к корректной организации работы **асинхронных процессов (потоков)**.

В дополнение...

Следует отметить, что замена трех команд, используемых для модификации переменной, на одну (`inc [ret]` для увеличения значения переменной и `dec [ret]` – для уменьшения) не избавляет от описанной проблемы. Подобный эффект обусловлен особенностью реализации современных процессоров.

Асинхронные параллельные потоки — потоки, которые существуют в системе одновременно и выполняются независимо друг от друга, но периодически должны синхронизироваться и взаимодействовать [17].

При работе с общими ресурсами асинхронными потоками могут быть реализованы следующие методы доступа.

1. **Последовательный** – выполнение запроса одного процесса (потока) будет осуществлено только после завершения выполнения запроса другого процесса (потока), сделанного ранее либо иным образом связанного с первым запросом. В качестве примера можно привести обращение к неименованному каналу: запрос на чтение данных может быть выполнен

только после выполнения запроса на запись данных. До этого процесс, пожелавший прочитать данные, будет заблокирован.

2. Параллельный – выполнение запроса производится независимо от иных запросов на обращение к ресурсу.

Ситуация, при которой два или более асинхронных потока или процесса пытаются получить доступ к общему ресурсу, называется **состоянием гонки**; участок кода программы, в котором возникает состояние гонки – **критической секцией**.

Таким образом, во избежание возможных негативных последствий при работе с общими ресурсами, асинхронные потоки (процессы) должны согласовывать свои действия при входе в критическую секцию, и при выходе из нее, исключая возможность появления состояния гонки. Для согласования работы процессов (потоков) используется специальный класс механизмов межпроцессного взаимодействия – **взаимоисключения**.

Выделяют следующие подклассы взаимоисключений:

- алгоритмические;
- аппаратные;
- программные.

4.3.2. Алгоритмические взаимоисключения

Алгоритмические взаимоисключения основаны на реализации внутри программы специального алгоритма, позволяющего исключить возможность появления состояния гонки. Проще говоря, алгоритмические взаимоисключения предполагают, что программист самостоятельно реализует все необходимые примитивы.

Наиболее простым представителем данного подкласса является **алгоритм Деккера** (рис.4.4) [22].

<pre>procedure Func1(..) f₁ ← 1 while f₂ = 1 do while turn = 2 do f₁ ← 0 end while f₁ ← 1 end while Критическая секция turn ← 2 f₁ ← 0 end procedure</pre>	<pre>procedure Func2(..) f₂ ← 1 while f₁ = 1 do while turn = 1 do f₂ ← 0 end while f₂ ← 1 end while Критическая секция turn ← 1 f₂ ← 0 end procedure</pre>
---	---

Рисунок 4.4. Алгоритм Деккера

Кроме алгоритма Деккера, выделяются также алгоритм Петерсона [23] и алгоритм Лампорта [24]. Однако, в связи с тем, что в современных вычислительных системах алгоритмические механизмы взаимного исключения не эффективны, подробно они рассматриваться не будут.

4.3.3. Аппаратные взаимного исключения

Аппаратные взаимного исключения предполагают, что для блокировки доступа в критическую секцию программист должен использовать возможности, предоставляемые аппаратным обеспечением. К ним относятся:

- запрет прерываний;
- команды типа «test-and-set»;
- команды обмена.

Как отмечалось в предыдущих разделах, механизм прерываний напрямую связан с механизмом вытеснения задач. Таким образом, *запрет прерываний* исключает возможность вытеснения, а как следствие обеспечивает атомарность выполнения операций. Стоит отметить, что запрет прерывания дает эффект лишь при использовании на уровне ядра. Использование команд запрета прерываний на пользовательском уровне приведет либо к возникновению ошибки, либо соответствующие команды будут проигнорированы.

Основное отличие использования команд класса «test-and-set» и команд класса обмена от использования запрета прерываний заключается в том, что данные команды требуются для проверки значения некоторой глобальной переменной, которое определяет, можно ли получить доступ к критической секции или нет. В случае, если доступ к критической секции запрещен, процесс (поток) производит повторные обращения к этой переменной до тех пор, пока не получит доступ на вход. Таким образом организуется «циклическая блокировка» (spinlock).

Использование аппаратных команд класса «test-and-set» позволяет одновременно проверить текущее состояние переменной, ограничивающей доступ в критическую секцию, и задать ей значение соответствующее блокированию доступа. В качестве примера команд описанного класса можно привести команды **bts** и **btc**.

В дополнение...

*Команда **bts** (**btc**) считывает в флаг **cf** и устанавливают в 1 (0) значение бита из битовой строки, определенной первым операндом – битовой базой (регистр или переменная), со смещением, указанным во втором операнде – битовом смещении (число или регистр) [11].*

Использование команд класса обмена, принципиально не отличается от использования команд класса «test-and-set». Единственное отличие заключается в том, что при использовании команд класса обмена требуется анализировать

результат, загруженный из переменной. Наиболее простым примером такой команды является команда **xchg**.

В листингах 17 и 18 приведен пример реализации механизмов взаимоблокировки с использованием команд класса «test-and-set» и команде обмена, соответственно.

Листинг 17. Пример использования команды класса «test-and-set»

```
1  .data
2  is_open: .byte 0
3  .text
4  spin_lock:
5  0:
6  bts $0, is_open
7  jc 0b
8  ret
9
10 spin_unlock:
11 pushl %eax
12 xor %eax, %eax
13 movl %eax, is_open
14 popl %eax
15 ret
```

Листинг 18. Пример использования команды класса обмена

```
1  .data
2  is_open: .byte 1
3  .text
4  spin_lock:
5  pushl %eax
6  xorl %eax, %eax
7  0:
8  xchg is_open, %eax
9  test %eax, %eax
10 jz 0b
11 popl %eax
12 ret
13 spin_unlock:
14 pushl %eax
15 movl $1, %eax
16 movl %eax, is_open
17 popl %eax
18 ret
```

4.3.4. Программные взаимоисключения

Все описанные выше механизмы взаимоблокировки (кроме запрета прерываний) используют в своей работе объекты, расположенные в адресном пространстве процесса. Таким образом, они подходят только для взаимоблокировки потоков. Для взаимоблокировки процессов требуется использовать дополнительные механизмы межпроцессного взаимодействия, позволяющие отобразить данные объекты в адресное пространство всех процессов, которые требуется синхронизировать.

Для решения указанной проблемы применяется подкласс *программных* взаимоисключений. Их суть заключается в том, что для разработчика программные взаимоисключения реализуются в виде некоторого API, при обращении к которому производится гарантированная блокировка доступа к критической секции. При этом, реализация данного механизма за пределами программного модуля может быть произвольной.

В качестве примера реализации программной взаимоблокировки можно привести циклическую блокировку, основанную на атомарной (с точки зрения пользовательского процесса) операции ОС – создании файла (листинг 19).

Листинг 19. Использование файловых операций для реализации механизма программного взаимного исключения

```
1 #include <fcntl.h>
2 #include <unistd.h>
3 #include <errno.h>
4 #define IS_OPEN "/tmp/file.lock"
5 #define O_FLAGS (O_CREAT|O_EXCL|O_RDWR)
6 #define O_MODE (S_IRUSR|S_IWUSR)
7 void spin_lock(char* name) {
8     while ((-1) == open(name,O_FLAGS,O_MODE)) {
9         if (errno != EEXIST) {
10             perror("open");
11             return __LINE__;
12         }
13     }
14 }
15 void spin_unlock(char* name) {
16     unlink(name);
17 }
18
19 int main(int argc, char *argv[])
20 {
21     spin_lock(IS_OPEN);
22     /* Критическая секция */
23     spin_unlock(IS_OPEN);
24     return 0;
25 }
```

Процедура `open` с параметрами `O_CREAT|O_EXCL` проверяет существование файла перед его созданием и, в случае если он уже существует, процедура завершается с ошибкой. С использованием данной особенности реализованы механизмы блокировки доступа в критическую секцию (строки 7-14) и разблокирования (строки 15-17).

Следует отметить, что приведенный пример без каких-либо дополнительных операций может быть использован для синхронизации доступа различных процессов, т.к. для блокировки используется объект, который изначально доступен для всех процессов системы.

Основным недостатком циклических блокировок является постоянная необходимость производить запрос на попытку входа в критическую секцию. С целью исключения данной проблемы, в начале 60-х годов 20-го века *Эдсгер Виле Дейкстра* (Нидерланды) предложил новую (для того времени) концепцию программных взаимных исключений – **семафор**.

Семафор – объект ОС, ограничивающий выполнение в критической секции более чем для N процессов (поток) [17].

Для взаимодействия с семафорами разработчику предоставляется доступ к двум атомарным процедурам.

1. **P** (нидерл. *Passering* – проходить) — при обращении к данной процедуре, ОС проверяет допустимое число процессов (потоков) в критической секции и, если допустимое число не исчерпано, процесс проходит в критическую секцию, если исчерпано – процесс ставится в очередь и блокируется.
2. **V** (нидерл. *Vrijgave* – освобождение) – вызов данной процедуры производится непосредственно перед выходом из критической секции. Если очередь процессов, ожидающих вход в критическую секцию, не пуста, то первый процесс из очереди разблокируется и получает доступ в критическую секцию. Если очередь пуста – счетчик «свободных мест» в критической секции увеличивается на 1.

```
procedure P(S)
  if  $S > 0$  then
     $S \leftarrow (S - 1)$ ;
    Поток проходит
  else
    Поток в очередь
  end if
end procedure
```

```
procedure V(S)
  if Очередь не пуста then
    Выбираем новый процесс
  else
     $S \leftarrow (S + 1)$ ;
  end if
end procedure
```

Рисунок 4.5. Процедуры реализуемые семафором

В дополнение...

Использование семафоров в ядре ОС недопустимо. Это связано с возможностью блокировки потока ядра, отвечающего за работу с очередью ожидающих процессов. Как следствие, вызов процедуры V не будет иметь никакого смысла. По этой причине на уровне ядра используются исключительно spin-блокировки.

В практическом применении семафоров наибольшую популярность получил их частный случай – мьютекс.

Мьютекс (англ. *mutex*, сокр. *mutual exclusion* – взаимное исключение) — одноместный семафор, т.е. объект ОС, ограничивающий выполнение в критической секции более одного процесса (потока).

4.3.5. Решение задачи «потребитель-производитель»

Вернемся к программному коду, представленному в листинге 16. Для решения задачи «потребитель-производитель» в первую очередь следует

определить критическую секцию, а точнее – две. В приведенном коде это строки 9 и 16.

На следующем этапе следует выбрать конкретный метод взаимного исключения, который будет использован.

В листинге 20 приведен пример решения задачи «потребитель-производитель» с использованием POSIX-мьютекса.

Листинг 20. Решение задачи «потребитель-производитель»

```
1 #include <pthread.h>
2 #include <stdio.h>
3
4 #define COUNT (5000000)
5 volatile unsigned int ret =0;
6 pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
7
8 static void* func1(void* arg )
9 {
10     int i;
11     for ( i = 0 ; i < COUNT ; i++ ) {
12         pthread_mutex_lock( &mutex );
13         ret++;
14         pthread_mutex_unlock( &mutex );
15     }
16     return NULL ;
17 }
18 static void* func2(void* arg )
19 {
20     int i;
21     for ( i=0 ; i < COUNT ; i++ ) {
22         pthread_mutex_lock( &mutex );
23         ret--;
24         pthread_mutex_unlock( &mutex );
25     }
26     return NULL ;
27 }
```

Следует отметить, что увеличение размера критической секции с целью включения в нее цикла **for** приведет к тому, что два потока выполнятся последовательно.

4.3.6. Монитор

Наиболее высокоуровневым механизмом взаимного исключения является монитор.

Монитор – это конструкция параллелизма, которая содержит как данные, так и процедуры, необходимые для управления взаимным исключением при распределении общего ресурса или группы общих ресурсов [17].

Формально монитор является конструкцией языка программирования и для него должно обеспечиваться выполнение следующих свойств.

1. Все переменные и структуры данных должны быть скрыты от внешних по отношению к монитору процедур, а доступ к ним должен осуществляться исключительно посредством процедур, реализованных в мониторе.
2. В любой момент времени в мониторе может быть активен только один процесс.

В связи с тем, что монитор является конструкцией языка программирования, обеспечение корректной работы в критической секции достигается за счет дополнительной обработки информации при компиляции.

Одним из языков программирования, в котором реализован описываемый механизм, является язык программирования Java. В листинге 21 приведен пример реализации монитора для решения задачи «потребитель-производитель».

Листинг 21. Пример реализации монитора на языке Java

```
1 public class ConsumerProducer {
2     private int counter = 0;
3
4     public synchronized void Consumer() {
5         counter--;
6     }
7     public synchronized void Producer() {
8         counter++;
9     }
10    public synchronized int Result() {
11        return counter;
12    }
13 }
```

В дополнение...

*Следует отметить, что в некоторые аппаратные платформы на уровне архитектуры заложена возможность синхронизации выполнения параллельных процессов, похожая на монитор. Примером такой платформы является графический процессор видеокарты. Так, для графических процессоров компании Nvidia определяется специальное понятие **варп** (warp). Варп – набор нитей (thread)^а, выполняемых физически одновременно. При этом, для каждой версии видеокарты размер варпа может быть различным. Так для процессоров Fermi размер варпа составляет 32 нити[25].*

^аВ парадигме Nvidia распараллеливание осуществляется посредством запуска множества экземпляров подпрограммы, параллельно каждая на своем процессоре. Каждый экземпляр в этом случае называется **нитью**.

4.4. Вопросы для самопроверки

1. Какие механизмы IPC могут быть использованы для связи процессов на физически не связанных СВТ?
2. Какой механизм IPC с вашей точки зрения является наиболее универсальным? Поясните.
3. Какие подклассы взаимоисключений являются наиболее переносимыми?
4. Напишите программу, в которой будет реализован механизм, исключающий возможность одновременного запуска ее нескольких экземпляров.
5. Реализуйте программный аналог семафора, используя файловые операции.

5. ВЗАИМОБЛОКИРОВКИ

5.1. Введение

Как и многие технологии взаимоисключения, в случае неправильного использования, могут принести больше вреда, чем пользы. Одним из таких результатов являются *взаимоблокировки*.

Взаимоблокировка (тупиковая ситуация) — состояние системы, при котором хотя бы один процесс (поток) вынужден ожидать событие, которое никогда не наступит.

Основной причиной взаимоблокировок является конкурентная борьба за ресурсы и некорректная обработка «состояния гонки».

Рассмотрим пример некорректного решения задачи «потребитель-производитель», приводящего к появлению взаимоблокировки (листинг 22).

Листинг 22. Блокировка в многопоточном процессе

```
1  #include <pthread.h>
2  #include <stdio.h>
3
4  #define COUNT (5000000)
5  volatile unsigned int ret1 = 0, ret2 = 0;
6  pthread_mutex_t mutex1, mutex2;
7  static void* func1(void* arg )
8  {
9      int i;
10     for ( i = 0 ; i < (int)arg ; i++ ) {
11         pthread_mutex_lock( &mutex1 ); ret1++;
12         pthread_mutex_lock( &mutex2 ); ret2--;
13         pthread_mutex_unlock( &mutex1 );
14         pthread_mutex_unlock( &mutex2 );
15     } return NULL ;
16 }
17 int main(int argc, char* argv[])
18 {
19     pthread_t thread;
20     void* retv ;
21     int i;
22     pthread_mutex_init(&mutex1,NULL);
23     pthread_mutex_init(&mutex2,NULL);
24     pthread_create(&thread,NULL,func1, (void*)COUNT);
25     for ( i = 0 ; i < COUNT; i++ ) {
26         pthread_mutex_lock( &mutex2 ); ret2++;
27         pthread_mutex_lock( &mutex1 ); ret1--;
28         pthread_mutex_unlock( &mutex2 );
29         pthread_mutex_unlock( &mutex1 );
30     }
31     pthread_join ( thread ,& retv ) ;
```

```

32  printf ( "RET: 0x%X; 0x%X\n", ret1, ret2 ) ;
33  return 0 ;
34  }

```

В случае, вытеснения основного потока после выполнения процедуры в 11 строке и передачи управления второму потоку в строке 27 второй поток будет заблокирован. Передача управления обратно первому потоку приведет к его блокировке в 12 строке. При такой ситуации разблокировка будет уже невозможна.

Следует отметить, что подобная ситуация может произойти и в однопоточном процессе (листинг 23).

Листинг 23. Блокировка в однопоточном процессе

```

1  #include <pthread.h>
2  #include <stdio.h>
3
4  int main(int argc, char* argv[])
5  {
6      int i, ret = 10;
7      pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
8      for ( i = 0 ; i < 10 ; i++ ) {
9          pthread_mutex_lock( &mutex );
10         if ( (--ret) == 7 ) continue;
11         printf("%d\n",ret);
12         pthread_mutex_unlock( &mutex );
13     }
14     return 0;
15 }

```

Так что же происходит в тот момент, когда процессы переходят в состояние вечной блокировки? В этот момент в системе возникает состояние, получившее название «круговое ожидание» (рис. 5.1).

На рис. 5.1 стрелка от процесса к ресурсу означает, что процесс пытается завладеть ресурсом, стрелка от ресурса к процессу – процесс владеет ресурсом.

Следует отметить, что круговое ожидание является одним из четырех условий появления взаимоблокировки. Выделяются следующие причины возникновения взаимоблокировок [17].

1. Использование механизмов взаимoisключений.
2. Ожидание дополнительных ресурсов.
3. Неперераспределяемость ранее занятых ресурсов.
4. Круговое ожидание.

При этом, первые три условия являются необходимыми, а четвертое – достаточным.

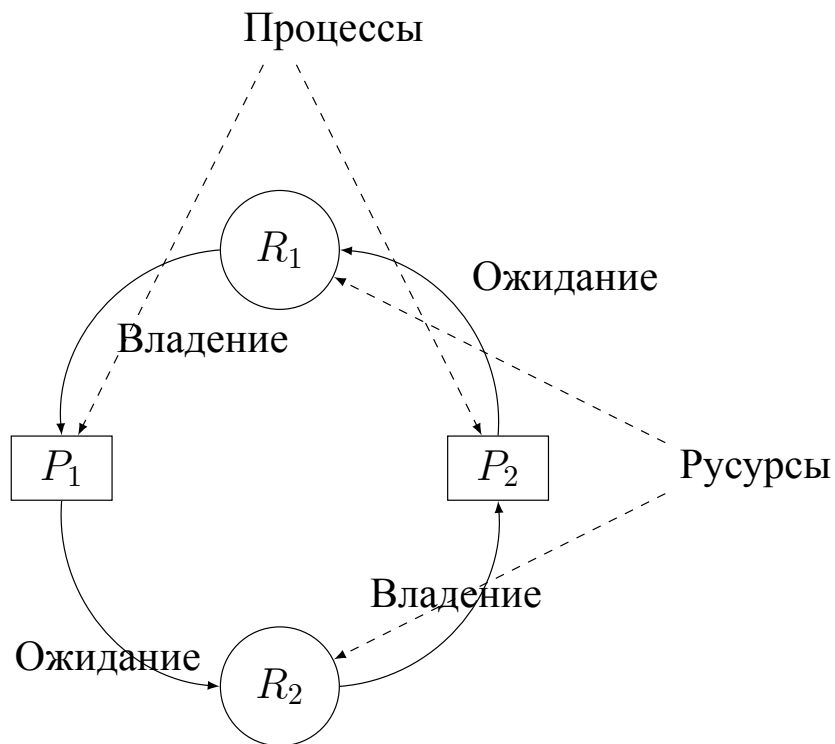


Рисунок 5.1. Круговое ожидание

Для решение проблем, связанных с взаимоблокировками, выделяются следующие группы механизмов.

1. Предотвращение взаимоблокировок.
2. Обход взаимоблокировок.
3. Обнаружение взаимоблокировок.
4. Восстановление после взаимоблокировок.

5.2. Предотвращение взаимоблокировок

5.2.1. Общий подход

Суть механизмов предотвращения взаимоблокировок сводится к описанию требований к построению системы, выполнение которых позволяет исключить саму возможность появления взаимоблокировок.

Таким образом, каждый из описанных ниже механизмов сводится к нарушению одного из условий возникновения взаимоблокировок.

5.2.2. Отказ от взаимоисключений

Поскольку взаимоисключения являются механизмами предотвращения неправильного функционирования вычислительных систем при использовании

ресурсов в многозадачных системах, отказ от этих механизмов должен быть обоснован.

В однозадачных системах, особенно в системах пакетной обработки, где единственный процесс монопольно владеет всеми ресурсами, использование механизмов взаимного исключения не требуется.

В иных системах это неверно. Вместе с тем в многозадачных системах возможно минимизировать использование данных механизмов. Для этого следует разделить все ресурсы на два класса и выделить класс ресурсов, для которых использование механизмов взаимного исключения не обязательно.

Разделение на классы производится в зависимости от возможности совместного использования ресурсов несколькими процессами. В соответствии с данным классификатором ресурсы разделяются на:

- перераспределяемые;
- неперераспределяемые.

Перераспределяемые ресурсы могут быть одновременно использованы несколькими процессами, и это не приведет к нарушению функционирования системы. При доступе к ресурсам данного класса состояние гонки не опасно, а следовательно блокировать доступ к критической секции не имеет смысла.

Примерами перераспределяемых ресурсов являются: строковые константы, системный счетчик времени и иные ресурсы, доступные только для чтения.

Для неперераспределяемых ресурсов отказ от использования механизмов взаимного исключения недопустим.

5.2.3. Ожидание дополнительных ресурсов

Для того, чтобы исключить ожидание дополнительных ресурсов, процесс может заранее запросить все необходимые для своей работы ресурсы. Однако, такой подход имеет свои отрицательные стороны. Во многом это связано не столько с самой программой, сколько с его «обвязкой». В том случае, когда разработчик знает полный перечень ресурсов, которые могут потребоваться создаваемой программе, он может попытаться оптимизировать код таким образом, чтобы минимизировать потребности в ресурсах. Например, он может использовать один экземпляр ресурса в различных частях программы вместо того, чтобы запросить большее число экземпляров. Однако, в современных системах разработчики не имеют подобной возможности. Современные программы создаются с использованием большого числа библиотек (см. п. 2), каждая из которых требует собственный набор ресурсов.

Рассмотрим пример. Во многих ОС библиотеки содержат код, отвечающий за фиксацию в системных журналах информации о различных событиях в работе программы. Например, о событиях информационной безопасности. Журнал

ОС является ресурсом. Следовательно, программа, использующая библиотеку с функционалом журналирования, затребует в момент запуска соответствующие журналы и будет удерживать их до завершения своей работы. В этом случае запуск второй программы, использующей ту же библиотеку, будет невозможен. При этом абсолютно не важно, будет ли в процессе функционирования первой программы сделана хотя бы одна запись или нет.

Вместе с тем, несмотря на явные ограничения, данный метод нашел достойное применение, пусть и в измененном (упрощенном) виде. Суть этого упрощения заключается в выделении класса ресурсов, для которых применяется принцип: «Все или ничего!».

Примером может служить *система запуска задач* в отечественном суперкомпьютере МВС-1000 [26]. Разработчик программы для этой системы, помимо самой программы, подготавливает *паспорт задачи*, который в том числе содержит количество необходимых задаче процессоров. В тот момент, когда задача передается на выполнение, ей в монопольное пользование выделяется требуемый набор процессоров. Разграничение доступа к остальным ресурсам системы производится иными методами.

5.2.4. Неперераспределяемость занятых (ожидаемых) ресурсов

Нарушение условия «неперераспределяемости ресурсов» достигается путем освобождения процессом всех ранее занятых ресурсов в случае отказа со стороны ОС в предоставлении очередного ресурса. В современных системах реализовать подобный механизм значительно проще, чем запрос всех ресурсов. В первую очередь это связано с тем, что нет необходимости контролировать полный перечень ресурсов, который требуется запросить при запуске программы. Однако, такой подход имеет собственные недостатки.

Рассмотрим пример. Допустим, что пользователь решил отретушировать фото из старого семейного архива. С этой целью пользователь запускает специализированный редактор. С помощью этого редактора он сканирует фотографию, редактирует её и планирует распечатать результат. Однако, в момент обращения к принтеру ОС отказывает процессу в предоставлении этого ресурса. В этом случае процесс редактора должен освободить ранее занятые ресурсы, а именно сканер и область оперативной памяти, в которой хранится отредактированное фото. Это означает, что после освобождения принтера процесс редактирования придется начинать заново.

5.2.5. Круговое ожидание

При выполнении тех необходимых условий возникновения взаимоблокировки неупорядоченный доступ к ресурсам неизбежно приведет к ситуации кругового ожидания.

С целью исключения подобной ситуации был разработан метод «линейного упорядочивания ресурсов» (рис. 5.2).

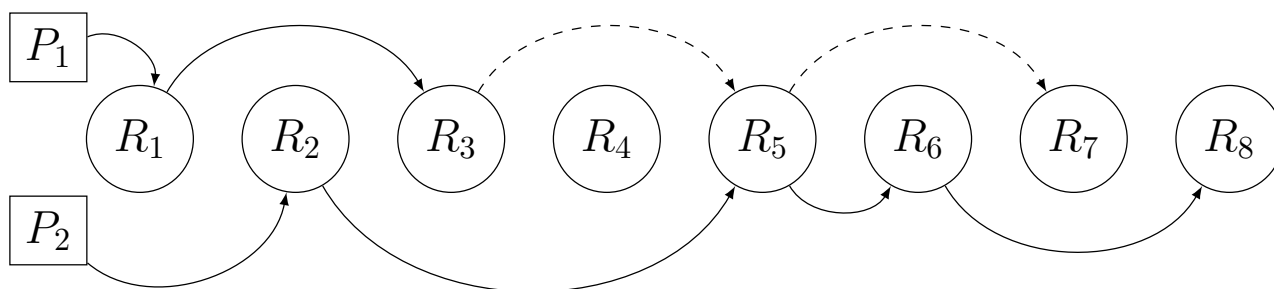


Рисунок 5.2. Линейное упорядочивание ресурсов

Суть метода заключается в том, чтобы пронумеровать все имеющиеся ресурсы в ОС и разрешать доступ к ним исключительно в порядке возрастания их номеров. В том случае, если процессу не удалось завладеть очередным ресурсом, он не может затребовать следующий. Таким образом, возникновение кругового ожидания становится невозможным.

Как и в предыдущих случаях, указанный метод накладывает ряд существенных ограничений на всю систему в целом.

Во-первых, исключена возможность использования ресурсов в произвольном порядке. В связи с этим, процессу для доступа к первому необходимому ресурсу потребуется запросить ряд ресурсов, которые могут потребоваться ему позднее.

Во-вторых, для устойчивой работы системы необходимо сохранение нумерации ресурсов в течение долгого времени. Смена нумерации ресурсов может потребовать внесения изменений во всё программное обеспечение, функционирующее в такой системе.

В-третьих, с учетом первых двух проблем, задача добавления очередного ресурса в систему также является трудоемкой.

5.2.6. Задача об обедающих философях

Проиллюстрируем применение описанных механизмов предотвращения взаимоблокировок на классической задаче, получившей название «Задача об обедающих философях».

Исходные условия. В некотором эфемерном пространстве за круглым столом сидят 5 философов. На столе перед ними стоят 5 тарелок и лежат 5 вилок (рис. 5.3²⁴)

²⁴Художник Богачева Д.Н.

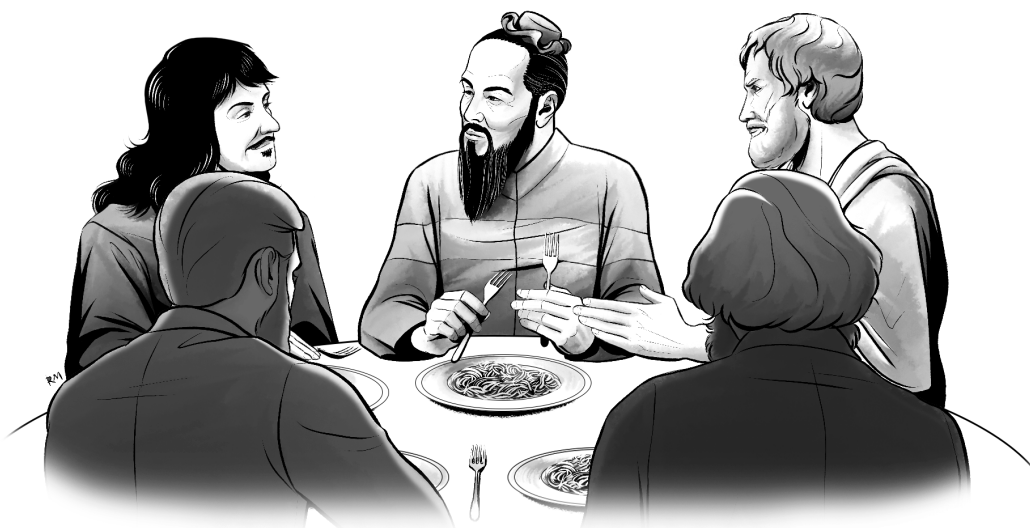


Рисунок 5.3. Задача об обедающих философах

Все свое время философы придают размышлениям, лишь изредка прерываясь на прием пищи. В момент, когда философ решает перекусить, в его тарелке появляется порция спагетти. Однако, есть маленький нюанс - философы едят спагетти исключительно с использованием двух вилок. При таком подходе возможна ситуация, когда каждый из философов возьмет по вилке, и тем самым они приведут систему, частью которой являются, в состояние взаимоблокировки.

Задача. Определить правила для философов, придерживаясь которых они смогут предотвратить взаимоблокировку.

Решение № 1. Несложно заметить, что в рамках данной задачи вилка – это ресурс, а философ – процесс. Таким образом, применимы методы обхода взаимоблокировок, описанные выше. Очевидно, что отказ от использования механизмов взаимоисключения недопустим, поскольку он приведет к физической борьбе философов за вилки. Однако, возможен выбор наиболее подходящего механизма взаимоисключения. В данном случае идеально подходит монитор. В терминах решаемой задачи указанным механизмом мог бы стать новый субъект – «официант» (рис. 5.4). В обязанности официанта будет входить распределение вилок между соседними философами.

Решение № 2. Философ может сразу попытаться взять две вилки и, если это получилось, начать прием пищи. Если ему это не удастся – подождать следующей попытки.

Решение № 3. Философ берет вилки по очереди. В случае, если взяв одну вилку, он не может взять вторую, то первую он возвращает на место.

Решение № 4. Каждой вилке на столе присваивается собственный порядковый номер. Философ вначале должен взять вилку с меньшим номером, а только потом с большим. При этом, если второй вилки нет, то возвращать на стол первую не обязательно.



Рисунок 5.4. Решение задачи об обедающих философах с использованием официанта

Не трудно заметить, что все решения достаточно просты. Однако, при каждом из предложенных решений возможна ситуация, при которых какому-то философу придется ждать своей очереди. Подобные ситуации могут продолжаться в течение всего периода существования системы, что негативно скажется на ее работе. Описанное состояние получило название «бесконечное откладывание».

Бесконечное откладывание (ресурсное голодание) — ситуация, при которой предоставление процессу некоторого ресурса будет откладываться на неопределённо долгий срок, в то время как система будет уделять внимание другим процессам.

5.3. Обход взаимоблокировок

Механизмы, связанные с обходом взаимоблокировок, предполагают установление контроля за распределением ресурсов, не допускающего появления взаимоблокировок. Для этого в системе определяются два состояния:

- «**безопасное**» — состояние, в котором хотя бы один процесс может завершиться штатным образом;
- «**небезопасное**» — состояние, в котором ни для одного процесса нет гарантии штатного завершения.

Все механизмы обхода взаимоблокировок направлены на сохранение системы в «безопасном состоянии».

Наиболее известным методом, относящимся к механизмам обхода взаимоблокировок, является метод, получивший название «алгоритм банкира». В качестве исходных данных для «алгоритма» рассматриваются:

- t – число ресурсов в системе;
- n – число процессов в системе;
- M_i – максимальное число ресурсов для процесса P_i ;
- $\mathcal{L}_i$ – число ресурсов, уже выделенное процессу;
- $\mathcal{C}_i$ – число ресурсов, которые понадобятся процессу:

$$\mathcal{C}_i = M_i - \mathcal{L}_i;$$

- a – оставшееся число ресурсов в системе:

$$a = t - \sum_{i=1}^n \mathcal{L}_i.$$

Используя введенные обозначения, определим понятие «безопасное состояние» следующим образом.

$$\exists i \in [1, \dots, n] : a \geq \mathcal{C}_i.$$

При обработке очередного запроса от какого-либо процесса на получение новой порции ресурсов, система принимает решение, удовлетворить запрос или отказать, исходя из возможности перехода в «небезопасное» состояние.

Рассмотрим пример. Допустим, в системе существуют 12 единиц некоторого ресурса ($t = 12$) и три процесса ($n = 3$). В таблице 1 приведены примеры «безопасного» и «небезопасного» состояний.

Таблица 1. Примеры «безопасного» и «небезопасного» состояние системы

Процессы	«безопасное состояние»			«небезопасное состояние»		
	M	$\mathcal{L}$	$\mathcal{C}$	M	$\mathcal{L}$	$\mathcal{C}$
P_1	4	1	3	4	2	2
P_2	6	4	2	6	4	2
P_3	8	5	3	8	5	3

Для «безопасного» состояния из $a = 2$, и, следовательно, процесс P_2 имеет возможность завершить свою работу, получив необходимое количество ресурсов. Для «небезопасного» состояния $a = 1$, что приводит к ситуации, когда ни одному процессу не хватает ресурсов для завершения.

Как и многие из описанных выше механизмы, «алгоритм банкира» имеет ряд особенностей. Во-первых, для его функционирования необходимо, чтобы в системе присутствовал *фиксированный набор ресурсов и процессов*. Во-вторых, требуется, чтобы для каждого процесса *заранее было известно значение M_i* . В-третьих, все ресурсы должны быть *возвращены в систему за некоторый конечный промежуток времени*. Кроме того, алгоритм не применим для систем *реального времени*.

5.4. Обнаружение взаимоблокировок

Как и в случае обхода взаимоблокировок, при обнаружении взаимоблокировок предполагается, что три обязательных условия возникновения взаимоблокировок в системе выполнены. Таким образом, обнаружение взаимоблокировок сводится к поиску процессов, находящихся в круговом ожидании.

Рассмотрим простейший случай, при котором в системе каждый ресурс существует в единственном экземпляре. По результатам анализа системы строится ориентированный граф $G = (E, V)$. Множество вершин E содержит все процессы системы. Множество ребер V определяет распределение ресурсов, а именно показывает какому процессу $e_i \in E$ принадлежит ресурс, запрашиваемый процессом $e_j \in E$. Графическое представление графа G является упрощенным вариантом схемы кругового ожидания, представленного на рис. 5.1 (рис. 5.5).

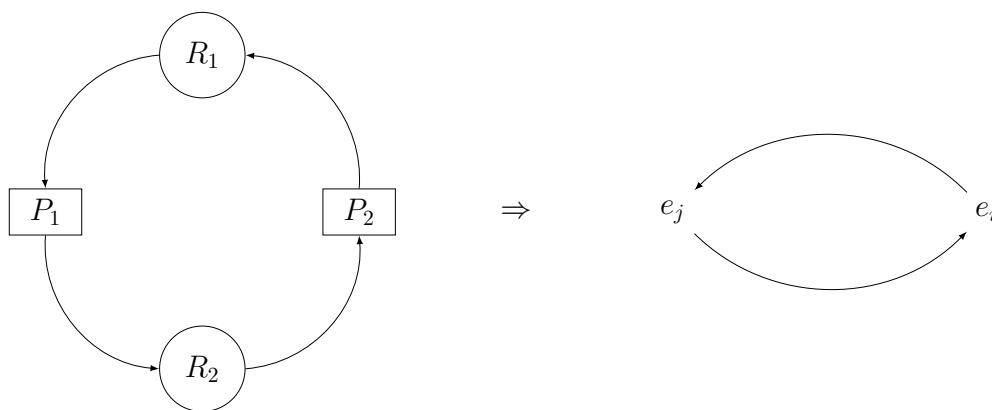


Рисунок 5.5. Круговое ожидание и граф распределения ресурсов

После того, как будет построен указанный граф, к нему применяется любой алгоритм поиска циклов в графе. И если цикл будет найден, то это будет означать, что в системе присутствует взаимоблокировка.

Указанный подход, может применяться как для системы в целом, так и для конкретного процесса. При этом анализ может быть проведен на этапе разработки ПО. Для этих целей могут быть использованы методы формальной верификации программного обеспечения, например метод «проверки моделей» (model checking) [27].

В случае, если в системе существует более одного экземпляра каждого ресурса, используется метод обнаружения взаимоблокировок, внешне схожий с «алгоритмом банкира». Единственное различие заключается в том, что рассматриваемый не требует знания максимально необходимого числа ресурсов для функционирования процессов. Вместо этого анализируется текущее распределение ресурсов и запросы на получение новых ресурсов. В качестве исходных данных рассматриваются:

- m – общее число различных классов ресурсов в системе;
- n – общее число процессов, функционирующих в настоящий момент в системе;
- вектор $T = (t_1, \dots, t_m)$, где t_i – общее число ресурсов i -го класса;
- матрица $C = \{c_{i,j}\}_{n \times m}$ – матрица текущего распределения ресурсов, где значение $c_{i,j}$ определяет число экземпляров ресурса j -го класса, занятых i -ым процессом;
- матрица $R = \{r_{i,j}\}_{n \times m}$ – матрица текущего состояния запросов на получение ресурсов, где значение $r_{i,j}$ определяет число экземпляров ресурса j -го класса, которое запросил i -ый процесс;
- вектор $A = (a_1, \dots, a_m)$, где a_j – количество экземпляров ресурсов j -го класса, доступных (нераспределенных) в текущий момент времени.

Тогда $a_j = t_j - \sum_{i=1}^n c_{i,j}, \forall j \in 1, \dots, m$. Таким образом, вектор A может быть получен из вектора T и матрицы C .

Алгоритм поиска взаимоблокировок состоит из следующих шагов.

1. В матрице R выбирается i -я строка, удовлетворяющая следующему условию.

$$\begin{cases} \sum_{j=1}^m c_{i,j} \neq 0, \\ r_{i,j} \leq a_j, \forall j \in 1, \dots, m \end{cases}$$

если строка найдена – перейти к шагу 2, если нет – алгоритм завершен.

2. Прибавить к элементам вектора A соответствующие элементы i -ой строки матрицы C . Перейти к шагу 3.
3. Все элементы i -ой строки матрицы R обнулить. Перейти к шагу 1.

В случае, если после завершения работы алгоритма в матрице R останутся ненулевые строки, это будет означать, что процессы, соответствующие данным строкам, находятся в состоянии взаимоблокировки.

Основная идея, заложенная в алгоритм, заключается в поиске процессов, для удовлетворения запросов на получение ресурсов которых в системе существует достаточное количество ресурсов. Если такой процесс находится, то предполагается, что он способен штатно завершить свое выполнение и вернуть в систему ранее занятые ресурсы.

В дополнение...

Необходимо отметить, что алгоритм не учитывает тот факт, что некоторые процессы в дальнейшем могут дополнительно запрашивать очередной набор ресурсов.

5.5. Восстановление после взаимоблокировок

5.5.1. Общий подход

Приведенный ниже набор механизмов предназначен для тех ситуаций, когда наступление взаимоблокировки предотвратить не удалось, но удалось ее обнаружить.

Основные цели использования механизмов восстановления заключаются в:

- восстановлении штатного функционирования системы без необходимости ее перезапуска;
- минимизации ущерба от возникновения взаимоблокировок.

Для достижения указанных целей используются:

- приоритетное овладение ресурсами;
- откат к контрольным точкам;
- уничтожение части процессов.

5.5.2. Приоритетное овладение ресурсами

Данный механизм предполагает, что в случае возникновения взаимоблокировок следует забрать часть занятых ресурсов у менее значимого процесса и передать его более значимому процессу. При этом критерии значимости могут быть различными. После того, как более значимый процесс освободит полученные ресурсы (в том числе, до момента своего завершения), они возвращаются «прежнему владельцу». Таким образом, система принудительно может нарушать условие «неперераспределяемости ранее занятых ресурсов». Очевидно, что в период «перераспределения» менее значимый процесс будет находиться в состоянии блокировки, а, следовательно, он не заметит факта передачи ресурса.

Однако, такой подход применим не для всех ресурсов.

Рассмотрим очередную классификацию, в соответствии с которой все ресурсы в системе делятся на:

- реэнтерабельные;
- нереэнтерабельные;

Реэнтерабельные ресурсы не требуют инициализации перед своим использованием. Подобное свойство ресурсов данного класса позволяет без последствий принудительно перераспределять их между процессами, когда занявший их процесс приостановил свое выполнение и находится либо в состоянии «ожидания», либо в состоянии «блокировки». Таким образом, в системах с разделением времени ресурсы данного класса могут считаться «перераспределяемыми». Для многозадачных систем это не всегда справедливо.

Нереэнтерабельные ресурсы требуют инициализации до момента первого использования, а следовательно не могут быть принудительно перераспределены. Это связано с тем, что возврат ресурса процессу, у которого он был забран, приведет к некорректному функционированию процесса или даже всей системы. Примером ресурса данного класса является принтер. Перед использованием принтера ему следует передать несколько команд. Во-первых, команду сброса текущего задания. Если команда не будет передана, то вместе с необходимой информацией принтер может распечатать что-то еще. Во-вторых, следует передать команду настройки формата печати и загрузки шрифтов. Очевидно, что если какой-то процесс настроил принтер, а впоследствии данный ресурс перераспределили, то результат печати предугадать будет сложно.

5.5.3. Использование контрольных точек

Механизм заключается в периодическом сохранении состояния системы (процесса, потока). В случае, если будет обнаружена взаимоблокировка, система (процесс, поток) откатываются к последней контрольной точке. В ходе отката отменяются последние выполненные операции, в том числе – запрос ресурсов. Вследствие этого, появляется возможность завершения всех процессов штатным образом без потери важных данных.

К недостаткам указанного механизма можно отнести значительные затраты памяти на хранение данных о контрольных точках и временные затраты на повторные попытки выполнения всех отмененных операций.

В дополнение...

Использование контрольных точек активно используется при проведении транзакций в базах данных или файловых системах.

Транзакция – это набор операций, которые должны быть выполнены полностью. Если хотя бы одна из операций, входящих в транзакцию, не будет выполнена, отменяются результаты выполнения всех предыдущих операций транзакции.

5.5.4. Уничтожение процессов

Крайней мерой для восстановления работы системы при взаимоблокировке является уничтожение процессов, вовлеченных в круговое ожидание. При этом, в ряде случаев достаточно уничтожить всего один процесс.

Для минимизации ущерба системе при использовании указанного механизма требуется точное определение всех процессов, находящихся во взаимоблокировке, а также связи между ними. Это позволяет определить минимальный набор процессов, которые требуется уничтожить. На рис.5.6 приведен пример состояния системы, в котором от выбора уничтожаемого процесса зависит размер возможного ущерба.

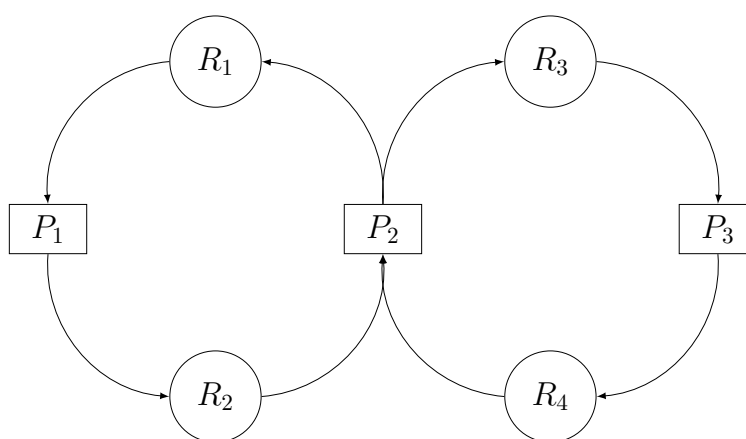


Рисунок 5.6. Граф распределения ресурсов при двойном круговом ожидании

В случае, когда все процессы равнозначны для системы, уничтожение единственного процесса P_2 позволит решить проблему с взаимоблокировками. Если же задачи, решаемые процессом P_2 , более критичны, то для минимизации ущерба потребуется уничтожить процессы P_1, P_3 .

5.6. Вопросы для самопроверки

1. Нарисуйте схему, аналогичную рис.5.1, но иллюстрирующую круговое ожидание для примера программного кода из листинга 23.
2. Приведите собственные примеры перераспределяемых и неперераспределяемых ресурсов.
3. Для какого класса ОС применение метода «линейного упорядочивания ресурсов» может быть оправдано?
4. Какие решения задачи об обедающих философам могут привести к бесконечному откладыванию? Приведите примеры.
5. Приведите примеры заполнения матриц C и R , а также векторов T и A , соответствующие ОС, в которой существует взаимоблокировка.

6. ПЛАНИРОВАНИЕ ПРОЦЕССОВ

6.1. Основные определения и задачи планирования

Немаловажным фактором эффективного функционирования ОС является выбор алгоритма планирования процессов (здесь и далее – также потоков).

Введём ряд определений.

Планировщик процессов — программный модуль ОС, реализующий политику планирования процессов.

Политика планирования процессов — алгоритм выбора процессов планировщиком процессов.

Целью оптимизации политики планирования является достижение следующих свойств системы:

- *максимальная пропускная способность* – свойство определяющее возможность выполнения максимального числа задач за некоторый конечный промежуток времени;
- *приемлемое время отклика*²⁵ – свойство системы, определяющее возможность в приемлемый (с точки зрения пользователя) отрезок времени обработать запрос и предоставить пользователю результат его выполнения (возможно, промежуточный);
- *максимальное использование ресурсов* – свойство системы, допускающее незадействованность ресурса только в том случае, когда не существует ни одного процесса, которому этот ресурс необходим;
- *исключение бесконечного откладывания (равноправие)* – свойство системы, гарантирующее каждому процессу получения доступа к процессору в конечный период времени;
- *минимизация накладных расходов* – свойство системы позволяющее снижать общий объём памяти и процессорного времени, требуемых для функционирования непосредственно планировщика процессов, в сравнении с общим объёмом ресурсов, используемых системой и иным программным обеспечением;
- *масштабируемость* – зависимость накладных расходов планировщика от числа процессов, функционирующих в ОС;
- *предсказуемость поведения системы* – свойство системы функционировать одинаково в одинаковых исходных условиях.

Наиболее важными из перечисленных выше свойств являются: равноправие, предсказуемость и масштабируемость.

²⁵Данной свойство наиболее актуально для интерактивных систем.

6.2. Уровни планирования

Учитывая, что процессор также является ресурсом системы, можно сказать, что политика планирования определяет последовательность доступа процессов к ресурсам. В связи с этим, при выборе политики планирования следует учитывать, что все процессы в разной степени зависят от использования внешних относительно процессора ресурсов. Можно выделить два класса процессов:

- *ограниченные возможностями процессора*, т.е. процессы, ориентированные на вычисления и требующие, в большей степени, для своей работы доступ к процессору;
- *ограниченные возможностями устройств ввода-вывода* – процессы, использующие процессорное время для формирования запросов к устройствам ввода-вывода и бездействующие во время ожидания их завершения.

В связи с этим, можно говорить о построении некоторой иерархии ресурсов (рис. 6.1).

На верхнем уровне располагаются внешние относительно процессора ресурсы. Объем подобных ресурсов в системе, как правило, ограничено, а факт их использования (включая количественные характеристики) могут быть заранее известны.

В этом случае применимы механизмы балансирования нагрузки с целью «обхода взаимоблокировок» (см. п. 5.3). Это достигается за счёт применения *политик планирования верхнего уровня*. Следует отметить, что планирование верхнего уровня требует предварительного определения набора необходимых процессу ресурсов. Т.е. перед запуском программного обеспечения ОС должна знать какой набор ресурсов и в каком объеме может понадобится процессу в работе. Это накладывает дополнительные ограничения, особенно при реализации в универсальных ОС, т.к. перечень и объем ресурсов, которые будут использоваться при функционировании системы, зависит от пользователя и используемого им программного обеспечения.

Если процесс, с точки зрения политики планирования верхнего уровня, может быть передан на исполнение, информация о нем передаётся *планировщику процессов среднего уровня*. Указанный планировщик отслеживает текущую загруженность системы с точки зрения процессорного времени и объема оперативной памяти. Если текущая загруженность не превышает пороговых значений, то планировщик среднего уровня передаёт информацию о процессе *планировщику нижнего уровня*.

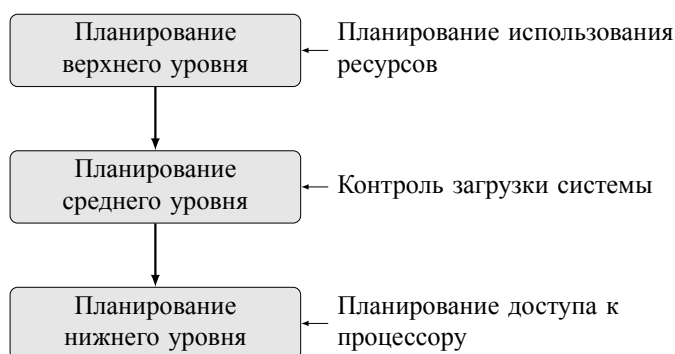


Рисунок 6.1. Уровни планирования задач

Политика планирования нижнего уровня решает вопрос распределения процессорного времени между переданными ей процессами. Следует отметить, что практически во всех универсальных, а также в большинстве специализированных ОС, изначально реализуется только политика планирования низкого уровня. Политики планирования более высокого уровня реализуются в качестве некоторой надстройки. Одним из примеров подобной реализации являются системы управления планированием и выполнением задач, реализуемые в суперкомпьютерах [26].

Что касается политик планирования нижнего уровня, то в зависимости от систем, на которых они реализуются, их возможно разделить на следующие классы:

- **планирование в системах пакетной обработки** — максимальный приоритет отдаётся пропускной способности и минимизации накладных расходов;
- **планирование в интерактивных системах** — приоритет отдаётся достижению приемлемого времени отклика на запрос пользователя;
- **планирование в системах реального времени** — максимальный приоритет отдаётся обеспечению предсказуемости поведения системы.

6.3. Планирование в системах пакетной обработки

6.3.1. Очередь (FIFO — *first input first output*)

Самым простым с точки зрения понимания и реализации является политика планирования «очередь» (рис. 6.2). Суть данной политики заключается в том, что процессы, последовательно добавляемые в систему, в той же последовательности получают монопольный доступ к вычислительным ресурсам системы и задействуют их до тех пор, пока не завершат свою работу.

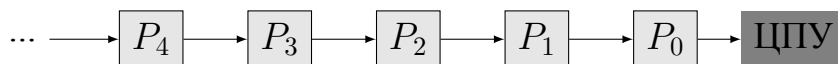


Рисунок 6.2. Политика планирования «очередь»

К недостаткам указанной политики можно отнести два аспекта. Во-первых, указанная политика не позволяет эффективно использовать все ресурсы системы. Во-вторых, ошибка в текущем процессе, приводящая к его блокировке, приведёт к блокировке всей системы.

Вместе с тем, указанная политика обеспечивает наименьшие накладные расходы и обладает максимальной масштабируемостью, что позволяет добиться высоких показателей с точки зрения пропускной способности системы в целом.

6.3.2. *Кратчайший процесс — первый (SPF – shortest process first)*

С целью увеличения максимальной пропускной способности систем, осуществлена модификация политики планирования «очередь». Суть модификации сводится к предварительному упорядочиванию процессов перед выполнением. В качестве основной метрики, используемой при упорядочивании, является время выполнения процесса. Это сохраняет²⁶ общее время выполнения всех процессов, однако помогает снизить среднее время выполнения каждого процесса.

Поясним этот эффект. Время выполнения процесса с точки зрения пользователя определяется временем с момента передачи программы на исполнение до момента ее завершения. Указанное время реально делится на два временных промежутка: *время ожидания* в очереди на получение доступа к процессору (l_i) и *время непосредственного исполнения* на процессоре (t_i). Таким образом, среднее время выполнения (T) каждого из N процессов может быть определено следующим образом:

$$T = \frac{\sum_{n=1}^N (t_i + l_i)}{N}.$$

Нетрудно заметить, что

$$l_i = \begin{cases} 0, & \text{при } i = 1 \\ l_{i-1} + t_{i-1}, & \text{при } i > 1. \end{cases}$$

Таким образом, T может быть выражено следующей величиной:

$$T = \frac{N \cdot t_1 + (N - 1) \cdot t_2 + \dots + 2 \cdot t_{N-1} + t_N}{N}.$$

Очевидно, что T минимизируется при условии, что $t_1 \geq t_2 \geq \dots \geq t_N$.

В дополнение...

С бытовой точки зрения указанная политика схожа с ситуацией, когда в супермаркете на кассу подходят два покупателя, один из которых везет полную тележку, а второй – одну покупку. Общее время, которое кассир потратит на обоих клиентов, не зависит от последовательности их обслуживания. Однако, если первым пройдет покупатель с меньшим числом покупок, то время оплаты товара для него сильно сократиться, а второй покупатель (заранее готовый к долгому процессу оплаты) не заметит этой разницы во времени.

Основным недостатком указанной политики планирования является сложность выбора критерия оценки времени выполнения процесса.

²⁶Без учёта времени, потраченного на упорядочивание.

Рассмотрим пример двух процедур, которые реализуют одинаковый функционал с использованием цикла и без него (листинги 24 и 25).

Листинг 24. Процедура с циклом

```

1 #include <time.h>
2 int func1(void)
3 {
4     int a = 0;
5     int i;
6     for (i=0; i<7; i++) {
7         a+=time(NULL);
8     }
9     return a;
10 }
```

Листинг 25. Процедура без цикла

```

1 #include <time.h>
2 int func2(void)
3 {
4     int a = 0;
5     a+=time(NULL);
6     a+=time(NULL);
7     a+=time(NULL);
8     a+=time(NULL);
9     a+=time(NULL);
10    a+=time(NULL);
11    a+=time(NULL);
12    return a;
13 }
```

Несмотря на то, что код одной процедуры в два раза короче кода другой, выполняется она в несколько раз дольше. Для обоснования этого рассмотрим дизассемблированный код обеих процедур (рис. 6.3). Нетрудно заметить, что на выполнение одной операции из трёх машинных команд дополнительно тратятся три машинные команды, реализующие цикл.

```

-----
..... ;*****
..... ; function func1 (global)
..... ;*****
..... func1:
.....     push        ebp
8048421 !     mov         ebp, esp
8048423 !     sub         esp, 28h
8048426 !     mov         dword ptr [ebp-0ch], 0
804842d !     mov         dword ptr [ebp-10h], 0
8048434 !     jmp         loc_8048449
8048436 !
..... loc_8048436:
.....     mov         dword ptr [esp], 0
804843d !     call        time
8048442 !     add         [ebp-0ch], eax
8048445 !     add         dword ptr [ebp-10h], 1
8048449 !
..... loc_8048449:
.....     cmp         dword ptr [ebp-10h], 6
804844d !     jng         loc_8048436
804844f !     mov         eax, [ebp-0ch]
8048452 !     leave
8048453 !     ret
8048454 !
-----
```

(a)

```

..... ;*****
..... ; function func2 (global)
..... ;*****
..... func2:
.....     push        ebp
8048455 !     mov         ebp, esp
8048457 !     sub         esp, 28h
804845a !     mov         dword ptr [ebp-0ch], 0
8048461 !     mov         dword ptr [esp], 0
8048468 !     call        time
804846d !     add         [ebp-0ch], eax
8048470 !     mov         dword ptr [esp], 0
8048477 !     call        time
804847c !     add         [ebp-0ch], eax
804847f !     mov         dword ptr [esp], 0
8048486 !     call        time
804848b !     add         [ebp-0ch], eax
804848e !     mov         dword ptr [esp], 0
8048495 !     call        time
804849a !     add         [ebp-0ch], eax
804849d !     mov         dword ptr [esp], 0
80484a4 !     call        time
80484a9 !     add         [ebp-0ch], eax
80484ac !     mov         dword ptr [esp], 0
80484b3 !     call        time
80484b8 !     add         [ebp-0ch], eax
80484bb !     mov         dword ptr [esp], 0
80484c2 !     call        time
80484c7 !     add         [ebp-0ch], eax
80484ca !     mov         eax, [ebp-0ch]
80484cd !     leave
80484ce !     ret
80484cf !     nop
80484d0 !
-----
```

(b)

Рисунок 6.3. Результат дизассемблирования процедур из листингов 24 и 25

Наиболее эффективным подходом в определении времени выполнения процессов является накопление статистики их выполнения. Имея информацию о результатах предыдущих запусков процесса и параметрах каждого запуска, можно спрогнозировать поведение процесса при очередном запуске. Однако такой подход уместен только в тех системах, где набор программ в течении долгого периода времени остаётся неизменным.

6.4. Вытеснение процессов

Основной недостаток политик планирования для систем пакетной обработки — невозможность пользователю вмешиваться в процесс работы ЭВМ до завершения текущей задачи (штатного или аварийного).

Для решения этой проблемы в интерактивных системах было принято решение о разработке механизмов приостановки выполнения текущей задачи с возможностью возобновления ее работы в дальнейшем. Указанный механизм получил название *вытеснение*.

Вытеснение — прерывание выполнения процесса с целью постановки на выполнение какого-либо иного процесса.

В зависимости от уровня реализации этого механизма, выделяют два его основных типа:

- *кооперативное вытеснение* — концепция вытеснения, в рамках которой процесс самостоятельно передает управление планировщику процессов, освобождая тем самым процессор для других задач;
- *приоритетное вытеснение* — концепция вытеснения, в рамках которой процессы «не подозревают о возможности собственной приостановки», а решение о вытеснении текущего процесса принимает планировщик процессов.

Решение о вытеснении текущего процесса может быть принято на основании различных событий. В качестве основных можно выделить четыре:

- добровольный запрос процесса на вытеснение;
- блокирование процесса в связи с обращением к устройству ввода-вывода;
- возникновение аппаратного прерывания, а, как следствие, необходимость выполнения соответствующего обработчика;
- завершение отведенного для процесса *кванта времени*.

Квант (времени выполнения) — временной промежуток, выделяемый процессу для использования ресурсов процессора.

6.5. Планирование в интерактивных системах

6.5.1. Циклическая очередь (RR – round robin)

Наиболее тривиальной политикой планирования с вытеснением является политика «циклическая очередь» (рис. 6.4). Суть данной политики заключается в том, что процессы, последовательно добавляемые в систему, в той же последовательности получают монопольный доступ к вычислительным ресурсам системы и задействуют их до наступления события, приводящего к вытеснению. После вытеснения процесс переносится в конец очереди, где ждет очередной возможности получить доступ к процессору.

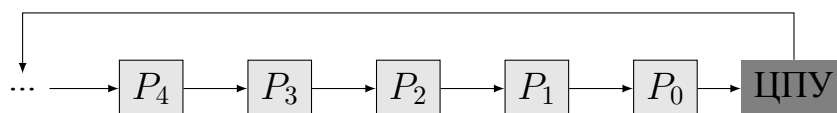


Рисунок 6.4. Политика планирования «Циклическая очередь»

Недостатки политики «циклическая очередь» идентичны недостаткам политики «очередь». Вместе с тем, в случае использования приоритетного вытеснения исключается ситуация, при которой ошибка в текущем процессе может привести к блокировке всей системы.

Основным недостатком является увеличение времени отклика системы при увеличении числа запущенных процессов. Решение указанной проблемы было достигнуто путем введения в политику планирования *приоритетов* процессов.

6.5.2. Наименьшее оставшееся время (SRT – shortest remaining time)

Аналогично ситуации с политикой планирования «очередь», преобразовавшейся в связи с использованием механизмов вытеснения в «циклическую очередь», политика SPF преобразовалась в политику SRT.

Как и в случае с SPF, политика планирования SRT предполагает упорядочивание процессов в соответствии с временем, оставшееся процессу до его завершения. Поскольку задача оценки данного времени сводится к задаче оценке времени возможного выполнения процесса с последующим вычитанием времени, которое процесс уже выполнялся, то недостатки указанных политик похожи.

6.5.3. Эгоистическое циклическое планирование (SRR – selfish round robin)

Политика планирования SRR в качестве приоритета использует метрику, получившую название *индекса старения* (a_i), который определяется общим временем, которое процесс непосредственно выполнялся.

В дальнейшем, перед выбором очередного процесса, производится упорядочивание очереди процессов в соответствии со значением их индексов старения. После этого на выполнение выбирается процесс с наименьшим

индексом старения. Квант времени, выделяемый процессам для выполнения до момента вытеснения, является постоянной для системы величиной. После его истечения планировщик процессов проверяет условие $a_1 < a_0$ (т.е. наличие индексов старения выше текущего индекса) и если условие верно, то текущий процесс вытесняется.

Достоинствами рассматриваемой политики являются простота реализации и наличие возможности влияния на порядок выполнения процессов. Последнее может реализовываться непосредственно в процессе. К примеру, если в момент ожидания некоторого события, процесс будет добровольно освобождать процессор, то в момент наступления указанного события индекс старения процесса будет такой, что позволит ему раньше снова попасть на выполнение.

Вместе с тем, оценка абсолютного времени выполнения процесса имеет негативные последствия. Если в системе присутствуют процессы, функционирующие в течение всего периода работы СВТ и решающие критически важные задачи, то по истечении некоторого времени появление процессов, выполняющихся в относительно короткий промежуток времени, может привести к длительному откладыванию ранее выполнявшихся процессов.

6.5.4. Относительное время реакции (*HRRN – highest response ratio next*)

Более «справедливым» вариантом расчета приоритета является оценка *относительного времени реакции процесса*, вычисляемого по следующему правилу.

$$P_i = \frac{a_i + l_i}{a_i}, \text{ где}$$

a_i – индекс старения процесса;

l_i – время бездействия процесса.

Использование подобного подхода позволяет обеспечить оперативное выполнение процессов, ограниченных ресурсами устройств ввода-вывода. В связи с тем, что именно посредством устройств ввода-вывода происходит взаимодействие с пользователем, рассматриваемая политика позволяет добиться низкого времени отклика для таких процессов.

Вместе с тем, процессы, ограниченные возможностями процессора, будут иметь весьма низкий приоритет.

6.5.5. Многоуровневая очередь с обратной связью (*MFQ – multilevel feedback queue*)

Все описанные выше политики – плохо масштабируемые. Это связано, в первую очередь, с тем, что после вытеснения очередного процесса требуется выполнение следующих операций: поиск соответствующего места в очереди на основе приоритета процесса, вставка элемента, удаление элемента.

В зависимости от способа реализации очереди, верхняя оценка вычислительной сложности указанных алгоритмов может быть определена согласно таб. 3²⁷[28].

Таблица 3. Вычислительная сложность основных операций планировщика процессов

Реализация	Поиск	Вставка	Удаление
Список	$O(n)$	$O(1)$	$O(1)$
Дерево	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$

Несложно заметить, что с увеличением числа процессов в системе, будет расти время выполнения планировщиком своих задач. Такая ситуация в ряде случаев, недопустима.

Выходом из сложившейся ситуации стала разработка политики планирования MFQ (рис. 6.5), в которой определено конечное число приоритетов. Процессы с одним приоритетом помещаются в соответствующую очередь и планируются внутри нее в соответствии с алгоритмом «FIFO». После вытеснения процесса его приоритет пересчитывается, и процесс помещается в конец очереди, соответствующей его приоритету.

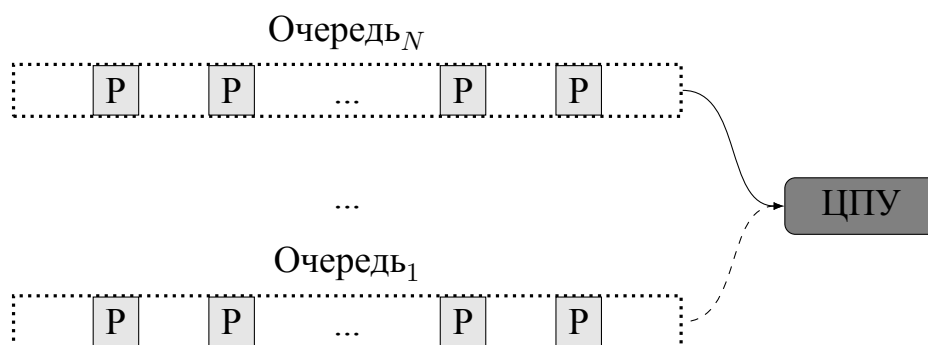


Рисунок 6.5. Политика планирования MFQ

Следует отметить, что алгоритм определения приоритета в данном случае может использоваться любой.

Главным достоинством указанной политики планирования является высокая масштабируемость. Это означает, что добавление очередного процесса не приводит к ухудшению показателей работы планировщика.

6.5.6. Планировщик Linux («О большое от единицы»)

В качестве частного примера политики планирования «MFQ» рассмотрим планировщик процессов, реализованный в ядре ОС Linux. Из-за вычислительной

²⁷В таблице n – число процессов, функционирующих в системе.

сложности указанный планировщик получил название $O(1)$ («О большое от единицы»), т.е. время выбора нового процесса не зависит от числа процессов в системе. Планировщик оперирует двумя наборами очередей – *активных процессов* и *неактивных процессов* (рис. 6.6) .

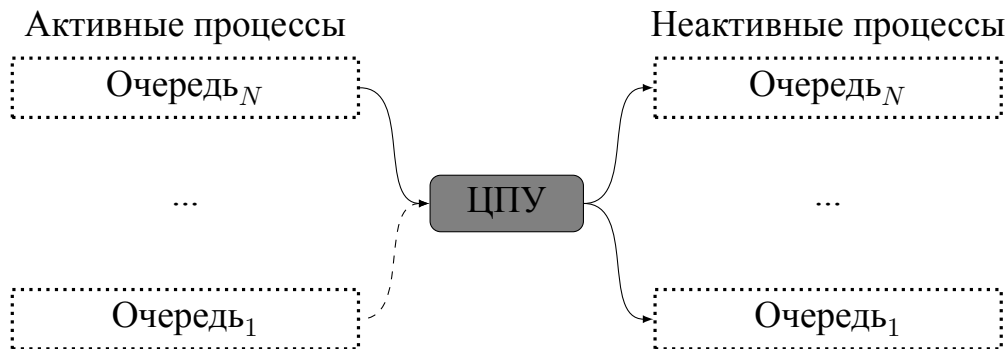


Рисунок 6.6. Планировщик процессов $O(1)$

После вытеснения процесса для него рассчитывается новое значение приоритета и он добавляется в соответствующую очередь из неактивного набора. Далее планировщик просматривает все очереди из активного набора, начиная с очереди процессов с наивысшим приоритетом, в поисках первой не пустой очереди. Как только планировщик находит подходящую очередь, очередной процесс из неё передаётся на выполнение. Если все очереди активного набора окажутся пустыми, неактивный набор становится активным, а активный – неактивным. Именно этим объясняются соответствующие состояния («активный» и «неактивный») жизненного цикла процессов в Linux (п. 3.7.2).

В каждый набор входят 140 очередей. При этом 100 из них с наиболее высоким приоритетом используются для «мягкого планирования реального времени», а оставшиеся 40 — для пользовательских процессов. Приоритет определяется по правилам, схожими с теми, что были рассмотрены в политике планирования HRRM (п. 6.5.4). Кроме того, пользователь сам может влиять на приоритет задачи путём назначения ей дополнительного приоритета в пределах от -20 до 20 единиц.

Следует отметить, что для наиболее приоритетных очередей может быть задан алгоритм планирования FIFO или RR, в следствии чего процессы из данных очередей будут выполняться без вытеснения, либо после вытеснения будут возвращены в соответствующую очередь активного набора.

6.6. Планирование в системах реального времени

Ранее (п. 1.) было указано, что операционные системы реального времени должны обеспечивать реакцию на какие-либо события внутри системы в течение жестко ограниченного отрезка времени.

В настоящий момент можно встретить два вида политик планирования реального времени.

1. Политика **мягкого планирования** реального времени.
2. Политика **жёсткого планирования** реального времени.

Политики планирования первого вида допускают некоторую «дельту» относительно времени, в течение которого должно быть обработано то или иное событие. В связи с чем, говорить о том, что системы, в которых реализованы подобные политики (например, Linux) являются системами реального времени не совсем корректно. Указанный вид планирования был введён как некоторый маркетинговый ход.

Полноценные системы реального времени имеют реализацию политик жёсткого планирования реального времени, то есть каждое событие в таких системах обязано быть обработано в срок.

В связи с тем, что системы реального времени, в основном, являются специализированными системами, реализация их политик планирования допускает разделение планировщика процессов на две составляющие:

1. *Планировщик периода разработки* – осуществляет планирование на основании информации о прикладных задачах и структуре их взаимодействия, имеющейся у разработчика до начала работы системы.
2. *Планировщик периода исполнения* – осуществляет планирование на основании данных, полученных в процессе работы на основе результатов планирования периода разработки.

Планировщик периода разработки обладает высокой предсказуемостью и низкими накладными расходами. Если на этапе разработки имеется полная информация о решаемых задачах, разработчик может отказаться от создания планировщика периода исполнения и реализовать *статический планировщик реального времени*. В противном случае реализуется *динамический планировщик реального времени*.

Кроме того, все политики планирования реального времени можно классифицировать в соответствии с рис. 6.7 [29].

Более подробное описание политик планирования реального времени выходит за рамки курса.

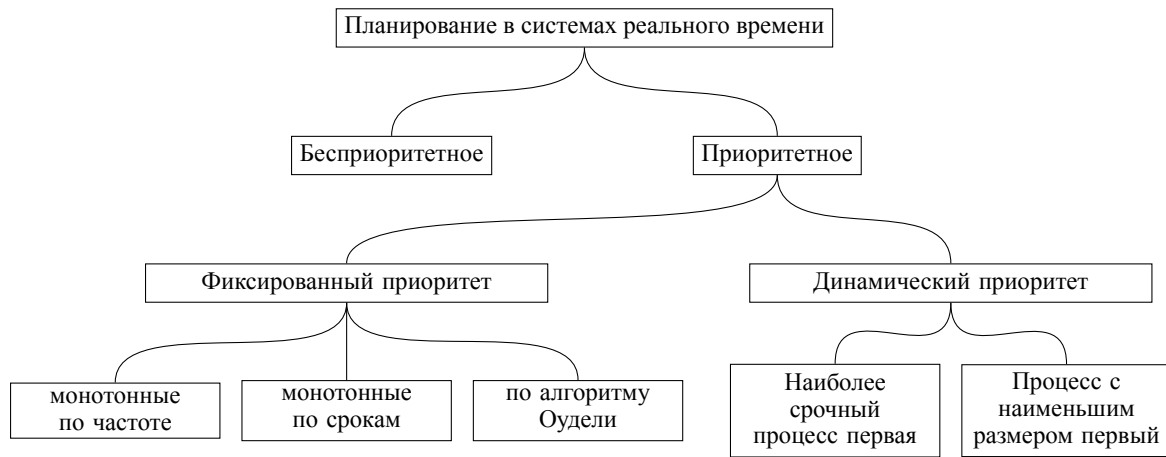


Рисунок 6.7. Классификация политик планирования реального времени

6.7. Вопросы для самопроверки

1. Для различных классов ОС (п. 1.3) из представленных политик планирования нижнего уровня выберите политики наиболее подходящие для использования на верхнем и среднем уровнях. Поясните.
2. Какие дополнительные накладные расходы проявляются в случае перехода ОС от политики планирования «очередь» к политике планирования «циклическая очередь»?
3. Поясните причины перехода между состояниями «активный», «не активный» и «выполнение» в жизненном цикле ОС Linux (рис. 3.11), опираясь на описание планировщика $O(1)$.

7. ОПЕРАТИВНАЯ ПАМЯТЬ

7.1. Иерархия памяти

Как отмечалось в п. 2, память ЭВМ имеет различные свойства и назначение. Вместе с тем, для памяти можно выделить ряд характеристик, общих для всех типов памяти. К ним, в том числе, относятся объем, скорость доступа и возможность непосредственного обращения.

В настоящее время параметры объем и скорость доступа имеют обратно пропорциональное соотношение (рис. 7.1).

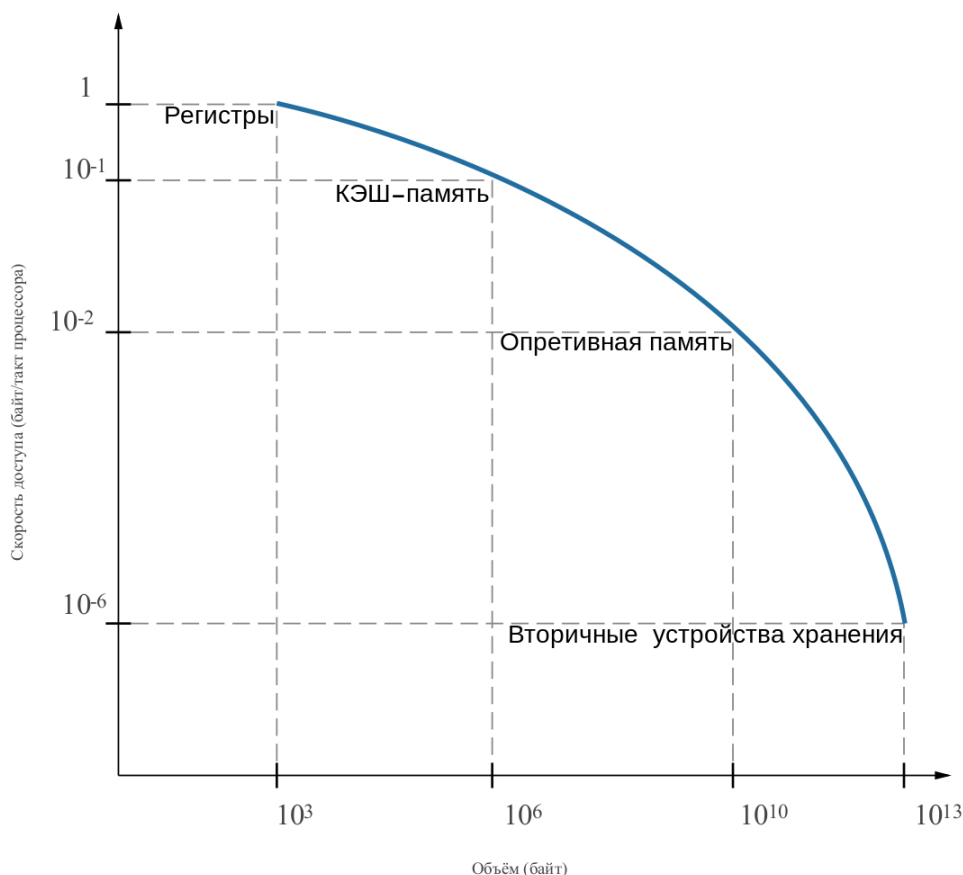


Рисунок 7.1. Соотношение объема памяти и скорости доступа к ней

С точки зрения возможности непосредственного обращения, память делится на два класса.

1. Память, к которой процессор осуществляет непосредственное обращение.
2. Память, к которой процессор может получить доступ после загрузки данных в память первого класса.

К памяти первого класса относятся регистры, кэш-память, оперативная память. К памяти второго класса – внешние носители информации, в т.ч. сетевые хранилища.

Таким образом, в СВТ выстраивается иерархия памяти (рис. 7.2), в рамках которой данные с вторичных устройств хранения первоначально загружаются в оперативную память. В процессе работы к этим данным производит обращение процессор. В тех случаях, когда некоторая область данных запрашивается достаточно часто, указанная область памяти переносится в кэш-память. Таким образом, достигается ускорение при работе с памятью.

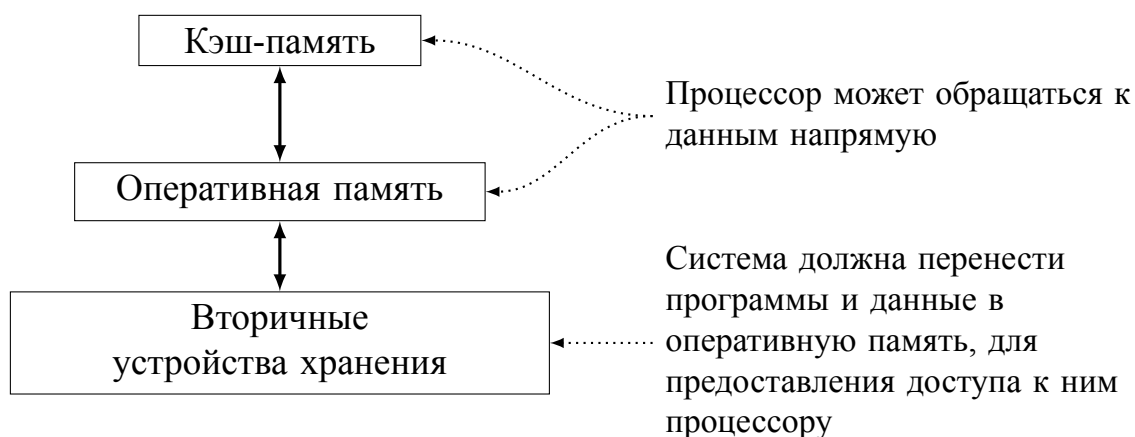


Рисунок 7.2. Иерархия памяти СВТ

Несмотря на то, что управление кэш-памятью осуществляется автоматически, система команд большинства процессоров позволяет программисту влиять на работу с ней. В частности, программист может заранее загрузить блок данных в кэш-память, получив при этом значительный выигрыш в производительности.

В дополнение...

Одним из наиболее эффективных подходов оптимизации программы, реализующей сложные вычисления, является предварительное вычисление значений некоторых преобразований с последующим обращением к ним. Однако есть одно ограничение. Дело в том, что подобная оптимизация программного модуля позволяет получить многократный прирост производительности лишь тогда, когда вычисленные значения полностью могут быть размещены в кэш-памяти. В противном случае накладные расходы на обращение к оперативной памяти могут полностью нивелировать полученный выигрыш.

Подробнее об оптимизации программного обеспечения в части использования оперативной памяти можно узнать в [30].

Управление оперативной памятью полностью реализуется программным обеспечением, в том числе с использованием механизмов, предоставляемых процессором.

7.2. Диспетчер оперативной памяти

Эффективное управление оперативной памятью предполагает решение следующих задач:

- выбор подхода к разделению памяти на области (разделы);
- поддержка динамического изменения размера разделов;
- привязка процесса к области памяти;
- увеличение количества процессов, находящихся одновременно в памяти.

Безусловно, приведенный перечень нельзя считать исчерпывающим, и с течением времени будут появляться новые задачи, а старые, наоборот, будут уходить в прошлое.

Учитывая задачи, описанные выше, несложно понять, что управление памятью для конкретной системы требует комплексного подхода. Выделяются три класса тривиальных стратегий управления памятью, комбинируя которые можно получить оптимальную стратегию для конкретной системы.

1. **Стратегии загрузки** — набор методик, определяющих порядок (последовательность) загрузки данных в память.
2. **Стратегии размещения** — набор методик, определяющих выбор области памяти, в которую будут загружены данные.
3. **Стратегии замены** — набор методик, определяющих выбор области памяти, данные из которой должны быть выгружены для размещения новых блоков (рассмотрена в п. 8.3.3).

Основным программным модулем, отвечающим за управление оперативной памятью, является *диспетчер оперативной памяти*, иногда называемый *менеджером памяти*.

Диспетчер оперативной памяти — компонент операционной системы, реализующий выбранную модель организации памяти и стратегию управления этой памятью [17].

Стратегия управления памятью — методика, по которой при конкретной организации памяти выполняется загрузка данных в память, их размещение и замена другими данными [17].

7.3. Подходы к разбиению памяти

Служебные данные
Операционная система
Пользовательское ПО
Неиспользуемая память

Рисунок 7.3. Разделение памяти в однозадачных системах

Во время функционирования вычислительной системы, в оперативной памяти располагаются три сущности: служебные данные аппаратной платформы СВТ, программные модули ОС, а также пользовательские программные модули. При этом области памяти, в которых могут размещаться служебные данные и основные модули операционной системы, четко определены [10]. Остальная память отдается под пользовательское программное обеспечение и данные. Именно для этой памяти выбираются наиболее подходящие способы разбиения. Наиболее простым случаем является распределение памяти в однозадачных системах, в которых вся свободная оперативная память отдается под нужды конкретного пользовательского

программного обеспечения (рис. 7.3). В иных системах свободная память тем или иным образом должна делиться под нужды нескольких процессов, при этом позволяя обеспечить возможность разграничения областей между процессами.

В дополнение...

До появления аппаратных механизмов разграничение памяти между процессами, разделение памяти носило условный характер. Это означало, что фактически процессы могли располагаться в разных областях памяти, но иметь возможность проникнуть в адресное пространство друг друга. На основании данной особенности, в 1972 году было придумано соревнование, по условиям которой в память компьютера загружалось два процесса. В задачи каждого из них входил поиск и «уничтожение» второго процесса в памяти. Игра получила название «Дарвин»^а.

^а<https://www.cs.dartmouth.edu/~doug/darwin.pdf>

Выбор подхода к разбиению памяти на блоки предполагает ответ на три основных вопроса.

1. Когда должно происходить разбиение?
2. Какие размеры должны иметь блоки?
3. Допустимо ли пересечение блоков?

На первый вопрос существует два ответа. Первый и наиболее простой ответ предполагает, что разбиение памяти должно происходить на этапе

проектирования и разработки системы, либо непосредственно перед ее запуском, и не меняться в процессе функционирования. Такой подход получил название **статического разбиения памяти**. Второй ответ предполагает, что разбиение памяти на блоки должно происходить в процессе функционирования. Такой подход получил название **динамического разбиения памяти**.

Преимуществом статического разбиения памяти является простота реализации диспетчера памяти, поскольку в нем не требуется наличие специальных модулей, отслеживающих текущее состояние распределения блоков памяти. Вместе с тем, некорректно выбранное разбиение может негативно сказаться на производительности всей системы.

Ответ на *второй вопрос* выбирается между: разбиением памяти на **блоки равной длины** и разделением памяти на блоки **разной длины**.

Варианты ответов на *третий вопрос* просты: **могут** или **не могут**.

Таким образом, может существовать всего 8 различных подходов к разбиению памяти. В современных системах используются два основных подхода:

- **сегментный** – динамическое разбиение памяти на блоки различной длины, которые могут пресекаться;
- **страничный** – статическое разбиение памяти на блоки равной длины, которые не могут пресекаться.

В дополнение...

Подобно вопросу разграничения доступа к памяти до появления соответствующих аппаратных механизмов, задание размера сегмента, а также различие их размеров были условными. Дело в том, что адресация внутри сегмента осуществляется посредством некоторого указателя, значение которого ограничивается исключительно размером области памяти, в которой он хранится.

В реальном режиме работы процессора с архитектурой Intel, для адресации используется пара регистров: сегментный регистр и регистр общего назначения. В сегментном регистре хранится адрес начала сегмента^а, а в регистре общего назначения хранится смещение относительно начала сегмента. В случае, когда регистр общего назначения имеет размер 16 бит, максимальный размер сегмента ограничивается значением 64 Кб. При этом, если разработчик решил, что сегмент должен иметь меньший размер, то он обязан контролировать значение смещения самостоятельно, чтобы не выйти за пределы сегмента.

^аПоскольку реальный адрес сегмента в процессорах Intel всегда должен быть выравнен на границу в 16 байт, в сегментном регистре хранится реальный адрес, деленный на 16.

7.4. Проблемные вопросы распределения памяти

Независимо от выбранного способа разбиения памяти на блоки, существует ряд проблемных вопросов, связанных с ее использованием.

Наиболее важным из них является вопрос превышения программным модулем размера отведенного ему блока памяти. В те времена, когда аппаратные компоненты вычислительных систем обладали низкими ресурсами при высокой стоимости и внушительном размере, подобная ситуация была не редкой.

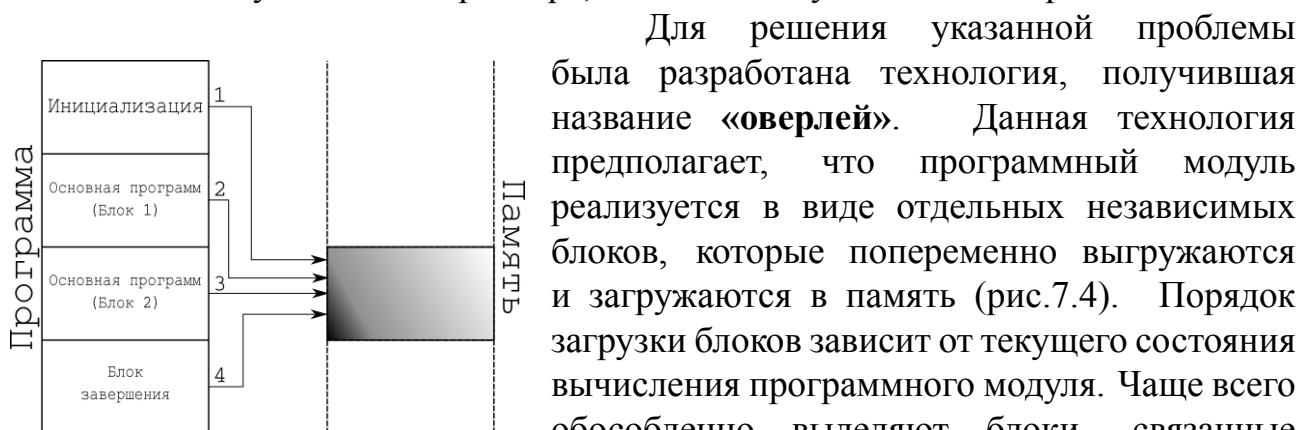


Рисунок 7.4. «Оверлей»

Для решения указанной проблемы была разработана технология, получившая название **«оверлей»**. Данная технология предполагает, что программный модуль реализуется в виде отдельных независимых блоков, которые попеременно выгружаются и загружаются в память (рис.7.4). Порядок загрузки блоков зависит от текущего состояния вычисления программного модуля. Чаще всего обособленно выделяют блоки, связанные с **инициализацией** программного модули и настройкой аппаратного обеспечения, которое планировалось задействовать в работе; **основным алгоритмом** работы программы, а также освобождением ресурсов после **завершения** основного алгоритма работы программы.

Второй, не менее важный вопрос, заключался в нерациональном использовании памяти, а именно в появлении блоков памяти, которые физически не могут быть использованы для размещения в них каких-либо данных. Данная ситуация получила название **«фрагментация»**. Следует отметить, что причины возникновения фрагментации при статическом и динамическом разбиении памяти различны.

При статическом разбиении памяти возможно возникновение ситуации, при которой загруженный в память программный модуль займет не весь объем отведенного ему блока. Подобная ситуация получила название **«внутренняя фрагментация»** (рис. 7.5). В целях уменьшения объема незадействованной памяти используются следующие подходы.

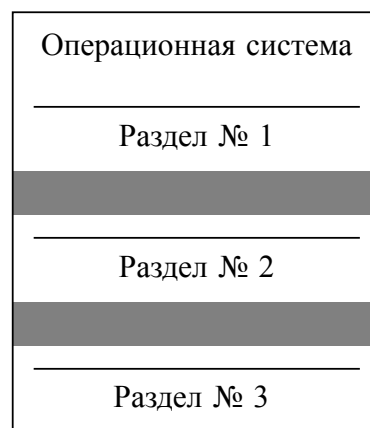


Рисунок 7.5. Внутренняя фрагментация при статическом разбиении

1. Уменьшение размера блоков при разбиении на блоки равной длины.
2. Выбор подходящей стратегии размещения (см. п. 7.6) при разбиении на блоки различной длины.

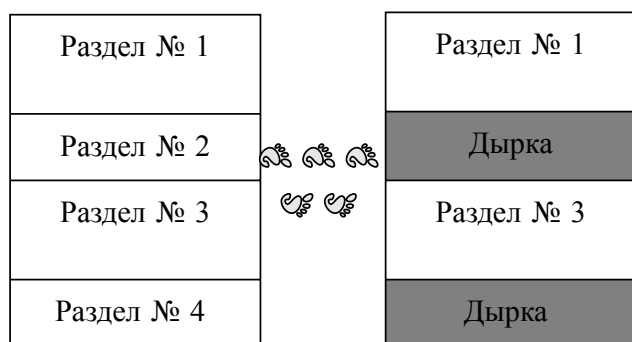


Рисунок 7.6. Внешняя фрагментация при динамическом разбиении памяти

осуществлена из-за того, что в системе больше не будет процессов столь же малого размера (рис. 7.6). Подобная ситуация получила название **«внешняя фрагментация»**, а свободные блоки получили название **«дырки»**.

В целях снижения внешней фрагментации были предложены два подхода. В рамках первого **соседние «дырки» объединяются** (рис. 7.7а), что позволяет получить единый блок большего размера. Подход является очевидным и легко реализуемым, однако эффективен лишь в случае последовательного размещения «дырок». Альтернативным подходом является **«уплотнение» памяти**²⁸ (рис. 7.7б). Данный подход предполагает перемещение всех занятых блоков к началу памяти, что приводит к «вытеснению» всех дырок в свободную область, где они могут быть объединены в один большой свободный блок памяти. Такой подход связан с дополнительными накладными расходами на перемещение блоков, однако, позволяет добиться наибольшей эффективности использования оперативной памяти.

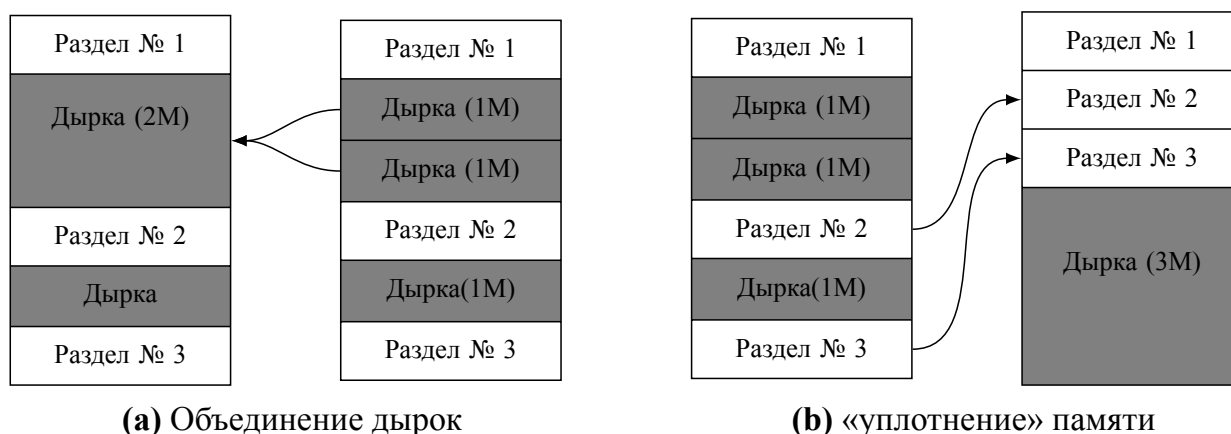


Рисунок 7.7. Подходы к снижению внешней фрагментации

²⁸В настоящее время большее распространение получило альтернативное название метода, а именно **«сборка мусора»** (англ. **garbage collection**).

7.5. Стратегии загрузки данных в память

Стратегии загрузки определяют подходы к порядку и последовательности загрузки блоков данных в память. Варианты стратегий определяются в зависимости от объемов доступной памяти и ограничений, накладываемых программным обеспечением.

С точки зрения объемов памяти выделяются следующие стратегии.

1. Предварительная загрузка.
2. Загрузка по требованию.

Предварительная загрузка предполагает, что необходимые для функционирования данные загружаются в память заранее. Стратегия позволяет *снизить накладные расходы на управление памятью*, однако, увеличивается общий объем занятой памяти верхнего уровня.

Загрузка по требованию предполагает, что данные загружаются в память лишь в том случае, когда процесс произвел непосредственное обращение к ним. Отсутствие необходимости держать в оперативной памяти данные, не задействованные в текущий момент при функционировании процесса, *позволяет снизить объем занятой оперативной памяти*. Вместе с тем, при каждом обращении к блоку данных, отсутствующему в оперативной памяти, операционной системе требуется загрузить его, что приводит к *увеличению накладных расходов на управление памятью*, а следовательно снижению общей производительности системы.

С точки зрения ограничений программного обеспечения выделяются следующие стратегии.

1. Стратегия с абсолютной загрузкой.
2. Стратегия с перемещаемой загрузкой.

Стратегия с абсолютной загрузкой предполагает, что в программных модулях, которые предполагается загрузить в память, все адреса имеют абсолютные значения. Это означает, что загрузка какого-либо компонента по иному адресу может привести к нарушению функционирования, как процесса, так и всей системы в целом. В рамках подобной стратегии для процессов формируются несколько очередей, учитывающих адреса начальной загрузки каждого из них (рис. 7.8).

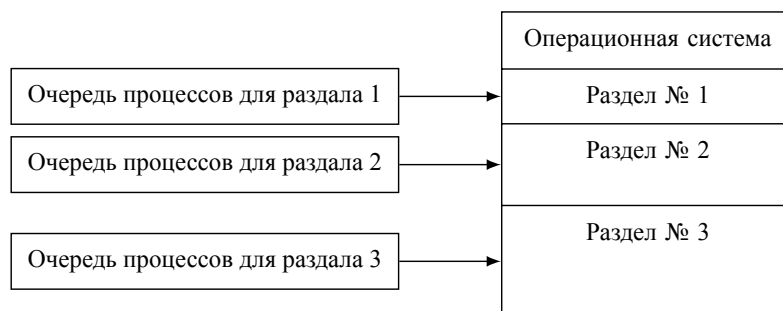


Рисунок 7.8. Стратегия абсолютной загрузкой данных в память

Подобная ситуация создает предпосылки для неэффективного использования ресурсов системы, в случае неравномерного заполнения очередей (рис. 7.9).

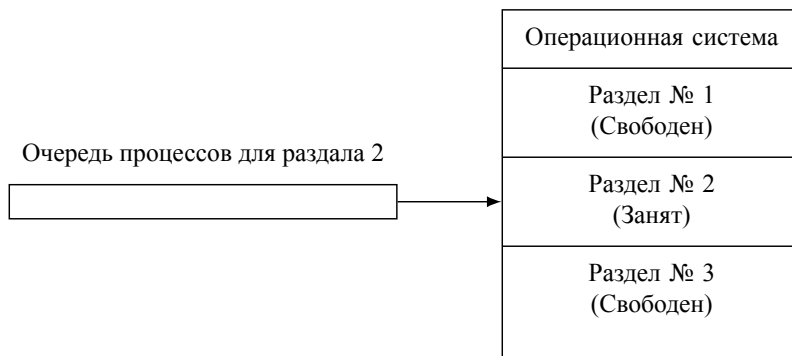


Рисунок 7.9. Неэффективное использование памяти при абсолютной загрузке данных в память

В случае перемещаемой загрузки, адресация осуществляется относительно некоторого базового адреса внутри каждого раздела. Данная стратегия позволяет производить загрузку программного модуля в любую доступную область памяти, повышая тем самым её эффективность использования (рис. 7.10).

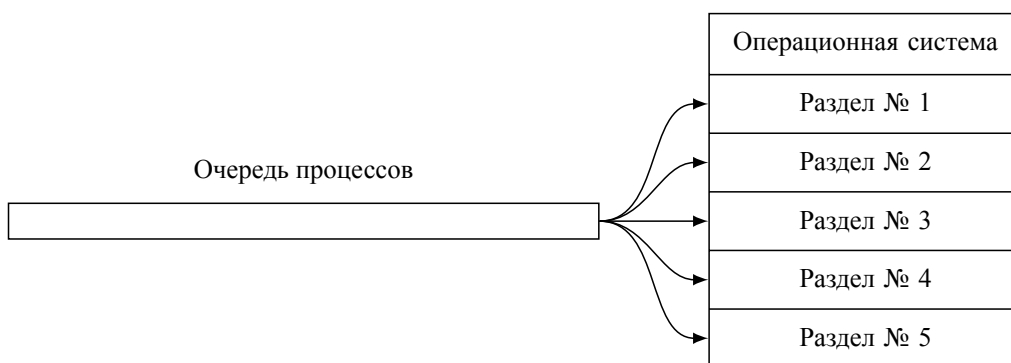


Рисунок 7.10. Стратегия с перемещаемой загрузкой

Следует отметить, что при использовании стратегии с абсолютной загрузкой (в отличие от стратегии с перемещаемой загрузкой) нет необходимости использования *стратегий размещения*. Таким образом, использование абсолютной загрузки позволяет упростить реализацию системы, однако делает ее менее гибкой.

В современных системах используются обе стратегии (с абсолютной и перемещаемой загрузкой). В виде программных модулей с абсолютной загрузкой реализуются все *непосредственно исполняемые бинарные модули*. Проблемы с неэффективным использованием оперативной памяти решаются за счет применения механизмов виртуальной памяти (п. 8).

В виде программных модулей с перемещаемой загрузкой реализуются все *библиотеки*. Это позволяет производить их загрузку в адресное пространство процесса любого размера и не обращать внимание на порядок их загрузки в случае использования процессом нескольких библиотек.

7.6. Стратегии размещения данных в памяти

Как было отмечено выше, стратегии размещения определяют выбор области памяти, в которую будут загружены код и данные процесса.

Наиболее очевидной стратегией является стратегия размещения в **первом подходящем блоке** памяти. В рамках данной стратегии система производит последовательный анализ свободных блоков памяти с целью поиска того блока, размер которого позволяет разместить в нем необходимые данные. Описанная стратегия является одной из наиболее простых в реализации. Однако, она обладает рядом недостатков. Во-первых, данная стратегия предполагает повторный анализ всех областей памяти при необходимости загрузки очередного блока данных. Во-вторых, стратегия не учитывает размеры всех свободных блоков, что может привести к неэффективному использованию памяти (например, в случае статического разбиения на блоки различного размера).

В целях исключения необходимости повторного анализа всех областей памяти, была предложена стратегия размещения, получившая название «**следующий подходящий**». Основная идея данной стратегии схожа с предыдущей, однако, поиск свободных блоков в ней осуществляется не от начала оперативной памяти, а от последней выделенного блока.

Как отмечалось выше, в тех случаях, когда в системе используется динамическое разбиение памяти, либо статическое разбиение на блоки различной длины, применение первых двух стратегий может привести к неэффективному распределению памяти. В системах с подобным разбиением получили распространение иные стратегии. Первая из них – стратегия, в рамках которой для размещения данных используется область памяти с **наиболее подходящим** размером. Это означает, что из всех подходящих для размещения блоков оперативной памяти выбирается тот, размер которой минимален. Это позволит не затрагивать блоки большего размера, оставляя их для размещения более широкого спектра данных. Недостатком данной стратегии является наличие предпосылок к возникновению фрагментации за счет того, что после размещения блока данных в области памяти может оставаться небольшой свободный фрагмент.

В случае со статическим разбиением, решение проблемы фрагментации при использовании описанной выше стратегии практически невозможно. В случае с динамическим разбиением, решение этой проблемы достигается посредством использования стратегии размещения данных в **наименее подходящем** блоке оперативной памяти. Это позволяет «подгонять» размер выделенного блока под загружаемые данные, оставляя при этом достаточно свободного пространства для загрузки иных блоков данных.

Несмотря на указанные плюсы и минусы последних двух стратегий, однозначно определить наиболее эффективную из них нельзя. Для демонстрации данного тезиса рассмотрим пример. В таблице 5 приведено два варианта размещения блоков в зависимости от выбранной стратегии. Исходным условием в примере являются два блока памяти, размером 20 и 15 условных единиц. Далее, в зависимости от запроса, происходит выделение памяти в одном из свободных блоков с изменением его размера на соответствующую величину.

Таблица 5. Сравнение стратегий размещения «наиболее подходящий» и «наименее подходящий»

Запросы	Наименее подходящий		Наиболее подходящий	
–	20	15	20	15
12	8	15	20	3
16	Блокировка		4	3
–	20	15	20	15
12	8	15	20	3
14	8	1	6	3
7	1	1	Блокировка	

Как видно из примера, ни одна из стратегий не позволила обеспечить стабильную работу системы даже в тех условиях, когда в ней оставались свободные ресурсы. Во многом именно эта ситуация и привела к отказу от динамического разбиения областей памяти в пользу статического.

7.7. Вопросы для самопроверки

1. Приведите все возможные подходы к разбиению памяти. Какие из подходов являются нереализуемыми и почему?
2. Какой вид фрагментации с Вашей точки зрения является наиболее опасным? Поясните ответ.
3. Приведите пример из повседневной жизни, демонстрирующий преимущества и недостатки стратегий с абсолютной и перемещаемой загрузкой друг по отношению к друга.
4. Проведите сравнение всех стратегий размещения данных в памяти по аналогии со сравнением стратегий размещения «наиболее подходящий» и «наименее подходящий» (таблица 5).
5. Какая стратегия размещения данных в памяти наиболее эффективна с Вашей точки зрения? Поясните.

8. ВИРТУАЛЬНАЯ ПАМЯТЬ

8.1. Архитектура виртуальной памяти

Развитие компонентной базы СВТ позволило частично перенести реализацию механизмов управления оперативной памятью на аппаратный уровень. Результатом такого переноса стало создание *блока управления памятью*.

Блок управления памятью (англ. Memory management unit, MMU) — специализированное аппаратное устройство, выполняющее, в частности, трансляцию виртуальных адресов в физические[17].

При использовании MMU все обращения к оперативной памяти транслируются через него и лишь потом передаются на выполнение. Такой подход позволяет производить дополнительные операции над адресами оперативной памяти, вплоть до их полной подмены. В связи с этим, в СВТ выделяются два вида адресации (рис. 8.1):

- **физическая (реальная) адресация** – при обращении к памяти используется адрес, явно или нет указанный в качестве операнда команды;
- **виртуальная адресация** – реальный адрес памяти, к которому требуется произвести обращение, получается в результате обработки адреса, явно или нет указанного в качестве операнда команды.

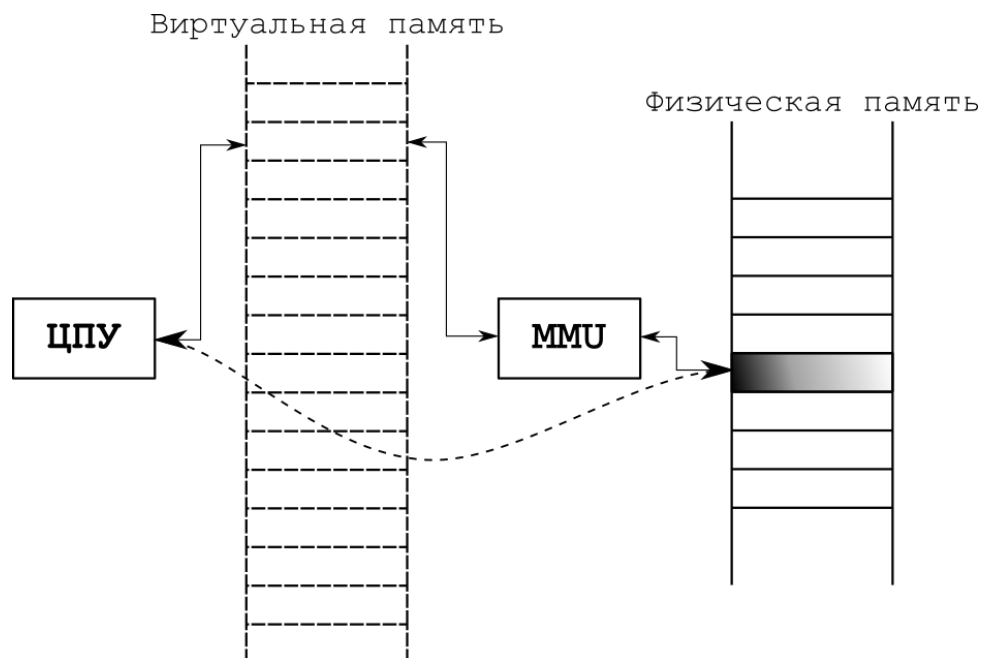


Рисунок 8.1. Принцип работы MMU

Общий принцип виртуальной адресации предполагает, что в ОС существует специальная таблица (набор таблиц), в которых хранятся адреса

областей физической памяти и дополнительная информация о них (размер, атрибуты доступа и т.д.).

В дополнение...

Атрибуты доступа для областей памяти определяют не только возможность чтения, записи и выполнения, но и уровень привилегий, на котором находится область (например, кольцо защиты). Таким образом, перемещение области по уровням привилегий процессора сводится к модификации соответствующего атрибута. При этом код не может произвести непосредственное обращение к содержимому областей памяти с большим уровнем привилегий.

Адрес таблицы, которую предполагается использовать для трансляции адреса виртуальной памяти, размещается в специализированной области T_1 . Сам виртуальный адрес разделяется на несколько индексов, каждый из которых независимо используется для получения реального физического адреса памяти.

В случае с двумя индексами (idx_1 , idx_2), первый индекс определяет соответствующую запись в таблице областей памяти, а второй – смещение от начала области в физической памяти системы. Таким образом организуется *прямая (линейная) трансляция адреса* (рис. 8.2).

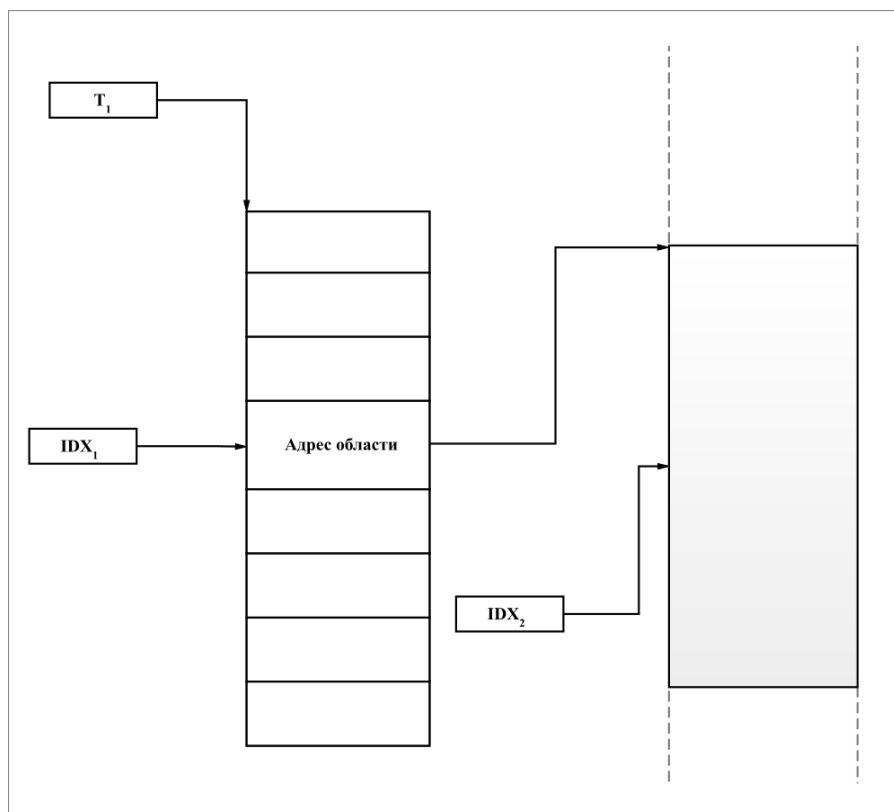


Рисунок 8.2. Прямая трансляция адреса

В случае, когда виртуальный адрес разбивается на более чем два индекса ($idx_1, \dots, idx_N$), реализуется механизм *многоуровневой трансляции адреса*. В этом случае MMU задействует $N - 1$ таблицу, причем адрес очередной таблицы T_i ($i > 1$) хранится в записи с индексом idx_{i-1} таблицы, размещенной по адресу T_{i-1} . Наиболее распространенным случаем является использование трех индексов (рис. 8.3).

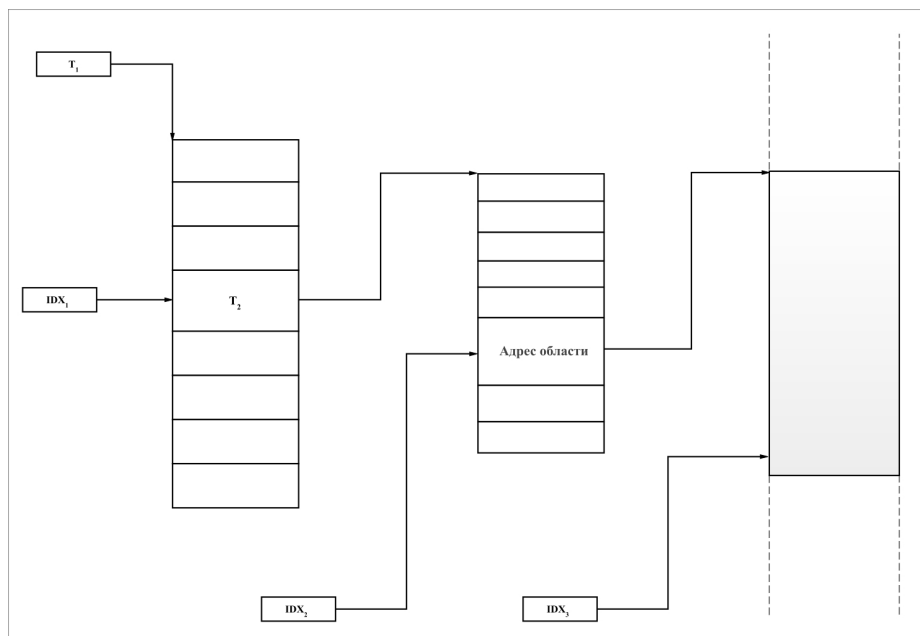


Рисунок 8.3. Многоуровневая трансляция адреса

Использование многоуровневой трансляции адресов имеет смысл в тех случаях, когда физическая память разделяется на большое число областей малого размера, для минимизации внутренней фрагментации памяти.

8.2. Типы адресации в виртуальной памяти

Как отмечено в п. 7.3, для современных систем подходами к разбиению памяти на области являются сегментное и страничное. Это справедливо и для виртуальной памяти. Рассмотрим подходы к трансляции адресов при использовании сегментной и страничной адресации применительно к процессорам с архитектурой x86.

При использовании *сегментной адресации* используется исключительно прямая трансляция адреса. При этом в системе существует два типа таблиц, используемых при трансляции.

1. **Глобальная таблица дескрипторов** (англ. Global Descriptor Table, GDT) — таблица, в которой хранится информация об общем для всей системы разбиении памяти. Использовать данную таблицу могут все процессы.
2. **Локальная таблица дескрипторов** (англ. Local Descriptor Table, LDT) — таблица, создаваемая для каждого процесса и сохраняющая разбиение памяти, уникальное именно для него.

На рис. 8.4 представлена структура записи таблицы дескрипторов. Подробное описание каждого из полей можно найти в [31, 11]. В рамках данного пособия интерес представляют поля P и DPL , размер которых составляет 1 и 2 бита, соответственно. Поле DPL (Descriptor privilege level) определяет уровень привилегий сегмента, т.е. кольцо защиты, на котором находится сегмент. Поле P (Segment present) указывает на присутствие (отсутствие) соответствующего сегмента в физической памяти. В случае, если данное поле равно нулю, то все содержимое записи игнорируется, а при обращении к ней процессор генерирует соответствующее прерывание.

0					7	8					15
Размер сегмента (биты 0:15)											
Адрес (биты 0:15)											
Адрес (16:23)					Тип			S	DPL	P	
Размер (16:19)		A		DB	G	Адрес (24-31)					

Рисунок 8.4. Структура записи таблицы дескрипторов

Адреса данных таблиц хранятся в регистрах gdt_r и ldt_r , соответственно. Для изменения значений данных регистров используются специальные привилегированные команды, которые могут быть выполнены только в $ring_0$ (см. п. 2.4.2). Модификация записей таблицы в зависимости от архитектуры ядра ОС может быть осуществлена, в том числе, и на пользовательском уровне.

При сегментной адресации значение индекса idx_1 соответствует значению сегментного регистра с 2-мя обнуленными младшими битами, используемого в исполняемой машинной команде. Значение индекса idx_2 соответствует значению регистра общего назначения [11].

В случае *страничной адресации* возможна как прямая трансляция адреса, так и многоуровневая. Случай прямой трансляции (рис. 8.5а) используется таблица, получившая название – *каталог страниц* (англ. Page Directory). При многоуровневой трансляции (рис. 8.5б) дополнительно используются *таблицы страниц* (англ. Page Table). Независимо от типа трансляции, адрес основной таблицы хранится в регистре cr_3 .

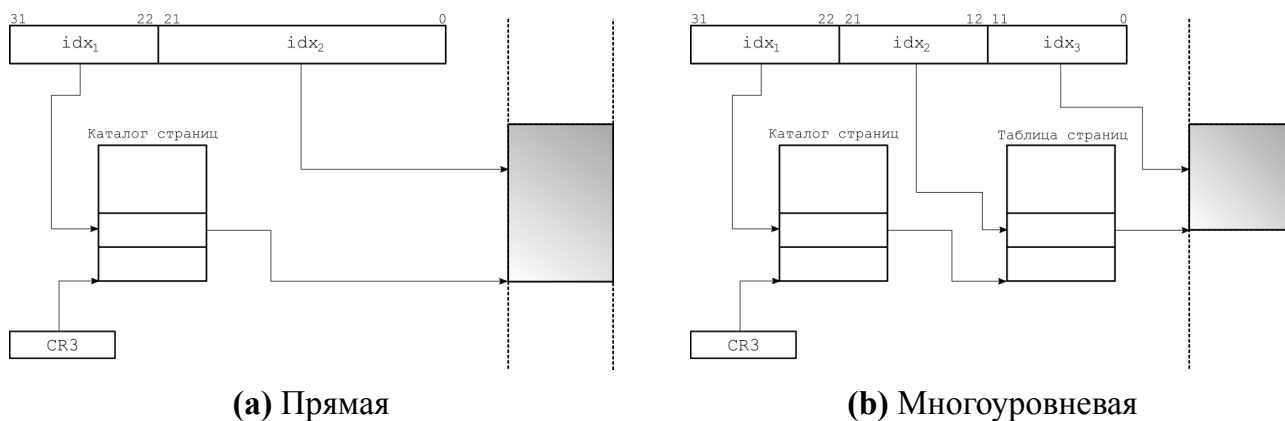


Рисунок 8.5. Виды страничной адресации

На рис. 8.6 представлена общая структура записи каталога и таблицы страниц. Подробное описание каждого из полей можно найти в [31, 11]. В рамках данного пособия интерес представляют поля P и U/S , размер каждого из которых составляет 1 бит. Поле P (Present) указывает: является ли данная запись активной или нет. Поле U/S (User/supervisor) определяет уровень привилегий страницы, т.е. относятся ли данные (код), находящиеся в ней к уровню пользователя или ядра.

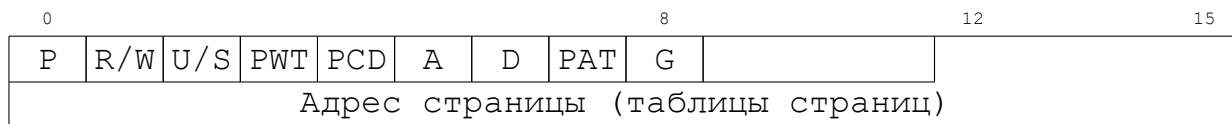


Рисунок 8.6. Структура записи каталога и таблицы страниц

Для 32-х разрядных систем в случае прямой трансляции адреса значение индекса idx_1 соответствует старшим 10 битам регистра общего назначения, а индекс idx_2 – оставшиеся 22 бита. В случае с многоуровневой трансляцией регистр общего назначения разделяется на три части – 10, 10 и 12 бит, значения которых соответствуют индексам idx_1 , idx_2 и idx_3 , соответственно. Сегментные регистры при страничной адресации не используются.

Несложно заметить, что при прямой трансляции адресов при страничной адресации каждый процесс может обратиться к 2^{10} записям о страницах с размером 2^{22} (4Мб). При многоуровневой адресации число страниц возрастает до 2^{20} , однако размер каждой из них равен 2^{12} (4Кб). В связи с этим, наиболее широкое распространение при страничной адресации получила многоуровневая трансляция.

Следует отметить, что наличие в современных процессорах поддержки обоих видов адресации (страничной и сегментной) привело к тому, что в реальности используется смешанный подход (сегментно-страничная адресация). Для каждого процесса ОС определяется набор сегментов памяти, к которым он может обращаться. При этом данные сегменты располагаются в памяти в соответствии с flat моделью памяти (см. п. 3.4). После этого область памяти разбивается на страницы, и уже в рамках данной области используется страничная адресация (рис. 8.7).

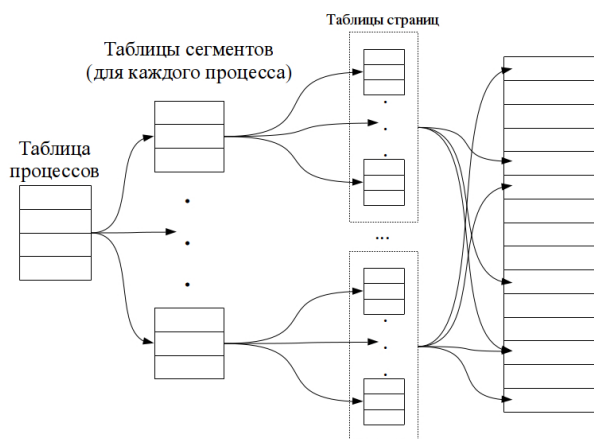


Рисунок 8.7. Сегментно-страничная адресация

Несмотря на «массивность конструкции», адресация осуществляется в соответствии с принципами страничной адресации памяти.

8.3. Подкачка

8.3.1. Назначение механизма

Одним из наиболее важных свойств виртуальной адресации является ее полная «прозрачность» для пользовательских процессов. Это означает, что с точки зрения процесса, он монопольно владеет оперативной памятью, размер которой ограничивается исключительно максимально возможным адресом для конкретной архитектуры. При этом, в реальности большинство адресов виртуального адресного пространства процесса могут не иметь отображение на физическую память СВТ.

Данное свойство в значительной мере упрощает реализацию приложений пользовательского уровня, однако приводит к усложнению подсистемы управления памятью ОС. В первую очередь это связано с необходимостью удовлетворения всех запросов процессов в условиях ограниченности реального ресурса. Для решения данной задачи был разработан специальный механизм, получивший название *подкачка*.

Подкачка (swap) — механизм, позволяющий осуществлять перемещение²⁹ содержимого области памяти процесса на вторичное устройство хранения или «удаление процесса» из памяти с целью передачи освободившейся области другому процессу [17].

Организация области подкачки на вторичных носителях информации может быть различной. Так, в ОС Linux для этих целей используется специальный выделенный раздел на жестком диске, а в ОС Windows – обычный файл.

Следует отметить, что не все области памяти могут быть выгружены. Например, если из памяти выгрузить процесс, отвечающий за функционирование механизма подкачки, то его работа в системе будет уже невозможна. Кроме того, выгрузка данных процессов, для которых критично время выполнения (например, процессов реального времени), также может критически сказаться на возможности выполнения системой своих целевых функций. По этой причине, в API каждой ОС предусмотрен специальный набор процедур, который позволяет указать ОС, какие области памяти процесса следует постоянно держать в физической памяти СВТ.

В дополнение...

Процессы, области памяти которых не могут быть выгружены на вторичные носители информации, создают потенциальную угрозу для работоспособности ОС. Так, в 2017 году на конференции Defcon исследователь в области информационной безопасности Шон Диллон (Sean Dillon) представил новый способ реализации атаки «отказ в

²⁹В разговорной речи более употребим термин «сброс».

обслуживании», основанной на особенностях реализации управления памятью в подсистеме SMB ОС Windows^a. Исследователь заметил, что при инициализации соединения память, выделяемая ОС для обеспечения взаимодействия по протоколу SMB, не может быть выгружена на вторичные устройства хранения и находится в памяти СВТ на протяжении всей сессии^b. Учитывая тот факт, что объем этой памяти определяет не сама система, а приложение, которое инициировало сообщения, исследователь показал, что установление большого числа соединений, в каждом из которых будет запрошен максимальный объем памяти, может привести к исчерпанию ресурсов ОС.

Максимальный объем памяти, который можно запросить для одной сессии, составляет 128Кб, а максимальное число сессий, которое может установить атакующее СВТ, равно количеству сетевых портов – 65535. Таким образом, одно атакующее СВТ может «заблокировать» в системе 8Гб оперативной памяти.

^a<https://www.defcon.org/html/defcon-25/dc-25-speakers.html#Dillon>

^bНеактивная сессия по умолчанию длится около 30 секунд.

8.3.2. Алгоритм работы механизма подкачки

Если инициация выгрузки данных из памяти осуществляется ОС и продиктована необходимостью высвобождения ресурсов для функционирования системы в целом, то причиной обратной загрузки выгруженных данных процесса в память является *промах*.

Промах — ситуация, при которой процесс обращается к виртуальной области адресного пространства, которая не имеет отображения на физическую область памяти.

Промах на аппаратурном уровне генерирует прерывание, что приводит к вызову соответствующего обработчика, который является частью подсистемы управления виртуальной памятью. ОС проверяет причину отсутствия отображения виртуального адреса на физическую память. Если причиной является выгрузка соответствующей области на вторичный носитель, ОС вновь загружает ее в физическую память, делает соответствующие корректировки в таблице трансляции адресов и возвращает управление процессу для обеспечения возможности его успешного завершения. Если соответствия виртуального и физического адреса не существовало (например, из-за ошибки разработчика) — процесс принудительно завершается.

Все описанные процедуры происходят незаметно для процесса. Однако в случае промаха время доступа к памяти значительно возрастает. В связи с этим,

реализация механизма подкачки для системы является краеугольным камнем с точки зрения производительности.

Для получения приемлемого результата требуется решение двух основных задач.

1. Выбор областей памяти, которые могут быть выгружены.
2. Оптимизация обработки промаха.

Далее будут рассмотрены возможные варианты решения указанных задач.

8.3.3. Стратегии замены страниц

Для определения областей памяти, которые могут быть выгружены, используются различные стратегии замены (см. п. 7.2). Поскольку наиболее распространенной адресацией памяти в ОС является страничная адресация, далее будут рассмотрены стратегии замены страниц.

Одной из наиболее простых стратегий с точки зрения реализации является стратегия замены страниц, в которой выбирается **«случайная страница»**. При реализации данной стратегии нет необходимости создания внутри ОС и поддержания в актуальном состоянии специальных структур и массивов данных. Эффективность стратегии достигается в предположении, что вероятность случайного выбора страницы, обращение к которой произойдет в ближайшее время, достаточно низкая.

Еще одна стратегия замены строится на предположении, что страница памяти, которая была загружена раньше всех, была уже использована и является наименее востребованной. Таким образом, данная стратегия замены страниц предполагает, что **первая загруженная страница должна быть первой выгружена**. При реализации данной стратегии в ОС создается очередь, в которую добавляются указатели на страницы, которые загружаются в память. Недостатком данной стратегии являются накладные расходы на поддержание очереди в актуальном состоянии. Основная проблема возникает в том момент, когда некоторый процесс завершается или просто инициирует освобождение ранее занимаемой им памяти. В этом случае требуется исключить из очереди указатели на страницы, освобожденные процессом, которые могут располагаться на различных позициях. Несмотря на это, указанные накладные расходы, как правило, менее критичны, чем время, затрачиваемое ОС на обработку промаха.

Для реализации следующей стратегии необходима фиксация не времени загрузки страницы в память, а времени последнего обращения к странице. Данная стратегия основывается на предположении, что **дольше всего не используемая страница** в перспективе такой и останется. В рамках данной стратегии считается, что отсутствие обращений к странице связано с тем, что данные, хранящиеся в ней, больше не требуются процессу, однако, по какой-то причине, он самостоятельно не инициировал ее освобождение. В качестве примера можно

привести страницы, в которых размещаются секции кода процесса, содержащие процедуры инициализации.

Описанная ранее стратегия требует выгружать из памяти страницу, наиболее давно неиспользуемую, что приводит к затратам на поиск максимального элемента в массиве данных. При увеличении объемов физической памяти возрастают и накладные расходы на выполнение данной операции. Для оптимизации стратегии был предложен менее жесткий критерий выбора страницы. Он предполагает, что в системе определен временной порог, при превышении которого страница считается **давно неиспользуемой** и может быть выгружена из памяти.

Две вышеописанные стратегии, предполагающие выгрузку страниц, которые долгое время не используются, для своего эффективного применения требуют выполнения достаточно сильного условия: долго неиспользованная страница останется такой и в дальнейшем. Однако, следует отметить, что, за исключением ряда частных случаев, чем дольше страница остается неиспользованной, тем выше вероятность, что к ней обратятся вновь. По этой причине был предложен следующий критерий выбора страницы — *частота обращений к странице*. При реализации стратегии, основанной на данном критерии, выгружается **реже всего используемая страница**.

В качестве последней стратегии замены страниц рассмотрим стратегию, которая предусматривает анализ *количества промахов* при обращении к странице. Данная стратегия предполагает, что из памяти следует выгружать ту страницу, для которой частота страничного промаха минимальна. В каком-то смысле данная стратегия похожа на стратегию выбора реже всего используемой страницы. Разница заключается в том, что при определении реже всего используемой страницы используется счетчик обращений, а в данном случае — счетчик промахов.

8.3.4. Стратегии восстановления выгруженных областей

Стратегии восстановления выгруженных страниц основываются на стратегиях загрузки, рассмотренных в п.7.5. Вместе с тем, существует ряд особенностей, выделяющих стратегии восстановления в отдельный подкласс стратегий загрузки.

Аналогично стратегиям загрузки, выделяют следующие стратегии восстановления.

- 1. Восстановление по требованию.**
- 2. Предварительное восстановление.**

Первая особенность стратегий восстановления заключается в том, что в обоих случаях инициация восстановления происходит в процессе обработки промаха.

Восстановление страниц по требованию предполагает, что в оперативную память будут загружены те страницы, попытка доступа к которым привела к промаху. Применение данной стратегии позволяет минимизировать использование физической памяти за счет уменьшения *пространственно-временного показателя* для процессов.

Пространственно-временной показатель — значение, равное произведению времени нахождения процесса в памяти на объем занимаемой им памяти [17].

Следует отметить, что данный показатель вычисляется для каждого временного промежутка, в котором процесс находится на стадии выполнения. Показатель позволяет оценить накладные расходы на загрузку необходимых страниц в память и реальные затраты на непосредственное выполнение процесса. Кроме того, минимизация данного показателя позволяет увеличить число процессов, страницы которых могут постоянно находиться в памяти. Таким образом, уменьшается число промахов при частом переключении между процессами.

Вместе с тем, восстановление по требованию предполагает, что для выполнения всех запросов процесса по доступу к страницам памяти, которые ранее были выгружены, система должна обработать число промахов, равное числу выгруженных страниц. Это, в свою очередь, приводит к появлению дополнительных накладных расходов.

Для минимизации числа промахов была разработана *стратегия предварительного восстановления страниц*. Аналогично стратегии предварительной загрузки, предварительное восстановление страниц предполагает, что при поступлении запроса (наступлении промаха) в память вычислительной системы будет загружен некоторый набор страниц, включая запрошенную. И тут проявляется вторая особенность стратегий восстановления: стратегия предварительной загрузки предполагает загрузку всего набора данных в память, а стратегия восстановления по требованию — только необходимого. Такой набор страниц получил название *рабочий набор*.

Рабочий набор — набор активно используемых процессом страниц оперативной памяти [17].

Нетрудно понять, что использование рабочих наборов приводит к увеличению пространственно-временного показателя для процесса. Кроме того, сам рабочий набор не является чем-то постоянным. В связи с этим, вводится еще одна характеристика, которая должна быть определена при использовании стратегий предварительного восстановления страниц.

Размер «окна» рабочего набора — значение, определяющее, насколько продолжительный интервал времени система должна принимать во внимание, определяя, какие страницы входят в рабочий набор процесса [17].

Определение рабочего набора осуществляется эмпирически. В случае, если размер «окна» рабочего набора равен δ , то в момент времени t рабочий набор определяется как набор страниц, обращение к которым было произведено в промежуток времени от $t - \delta$ до t .

В процессе определения рабочего набора система локализует страницы набора в физической памяти, что позволяет снизить накладные расходы на пересылку данных из вторичных носителей информации в оперативную память. Расположение данных страниц в виртуальной памяти может быть абсолютно различным.

Следует отметить, что чаще всего рабочий набор и в виртуальной памяти локализован. Это свойство рабочих наборов получило название *пространственная локализация*.

Пространственная локализация — эмпирическое свойство, которое в системах подкачки проявляется в том, что процесс, в основном, работает с некоторой частью своих страниц, размещенных поблизости друг от друга в его виртуальном адресном пространстве [17].

Учитывая вышеуказанные факторы, для выбора подходящей стратегии восстановления страниц следует проанализировать следующие факторы:

- объем памяти, доступный для размещения загружаемых страниц;
- количество страниц, предварительно загружаемых за один проход;
- алгоритм выбора предварительно загружаемых страниц.

8.4. Вопросы для самопроверки

1. В ОС, основанных на каких типах ядер, допускается модификация таблиц дескрипторов на пользовательском уровне и почему?
2. Для каждой стратегии замены страниц, используя классификацию (п. 1), опишите наиболее подходящий класс ОС. Обоснуйте.
3. Учитывая факторы выбора стратегии восстановления страниц, приведите характеристики системы, в которой оптимально использование стратегии восстановления страниц по требованию, и системы, в которой оптимально использование стратегии предварительного восстановления.

СПИСОК ЛИТЕРАТУРЫ

- [1] Керниган Брайан У., Ритчи Деннис М. Язык программирования С. — М. : Вильямс, 2013. — 304 с. — ISBN: 0-13-110362-8.
- [2] Hansen P.B. The evolution of operating systems. — Режим доступа: <http://brinch-hansen.net/papers/2001b.pdf> (дата обращения: 01.02.2019).
- [3] Воеводин Вл.В. Эффективность суперкомпьютерных центров в эпоху тотального параллелизма. Пленарный доклад // Параллельные вычислительные технологии (ПаВТ).
- [4] ГОСТ 19781-90. Обеспечение систем обработки информации программное. Термины и определения.
- [5] ISO/IEC 9899:1999. Programming languages – C.
- [6] Ахо Альфред В., Лам Моника С., Сети Рави, Ульман Джеффри Д. Компиляторы: принципы, технологии и инструменты. — 2 изд. — М. : ООО «И.Д. Вильямс», 2008. — 1184 с. — ISBN: 978-5-8459-1349.
- [7] Гриффитс Артур. GCC. Настольная книга пользователей, программистов и системных администраторов. — М. : ТИД «ДС», 2004. — 624 с. — ISBN: 966-7992-34-9.
- [8] Шоу А. Логическое проектирование операционных систем: Пер. с англ. — М. : Мир, 1981. — 360 с.
- [9] Таненбаум Э., Остин Т. Архитектура компьютера. — 6 изд. — СПб. : Питер, 2017. — 816 с. — ISBN: 978-4-496-00337-7.
- [10] Кулаков В. Программирование на аппаратном уровне. Специальный справочник. — Спб. : Питер, 2001. — 496 с. — ISBN: 5-272-00359-4.
- [11] Зубков В.В. Ассемблер для DOS, Windows и Unix. — М. : ДМК Пресс, 2017. — 638 с. — ISBN: 978-5-97060-535-6.
- [12] Project Zero: Reading privileged memory with a side-channel. — Режим доступа: <https://googleprojectzero.blogspot.com/2018/01/reading-privileged-memory-with-a-side-channel.html> (дата обращения: 01.02.2019).
- [13] QEMU. — Режим доступа: <https://qemu.org> (дата обращения: 01.02.2019).

- [14] Таненбаум Э., Бос Х. Современные операционные системы. — 4 изд. — СПб. : Питер, 2018. — 1120 с. — ISBN: 978-5-4461-1155-8.
- [15] VirtualBox Guest Additions ISO Download? — Режим доступа: <https://blogs.oracle.com/joshis/vitruualbox-guest-additions-iso-download> (дата обращения: 01.02.2019).
- [16] Rutkowska Joanna. Subverting Vista Kernel For Fun And Profit. — Режим доступа: <https://blackhat.com/presentations/bh-usa-06/VN-US-06-Rutkowska.pdf> (дата обращения: 01.02.2019).
- [17] Дейтел Х. М., П. Дж. Дейтел, Д.Р. Чофнес. Операционные системы. Часть 1. Основы и принципы. — 3 изд. — М. : Бином-Пресс, 2011. — 1024 с. — ISBN: 0-13-182827-4.
- [18] Эриксон Дж. Хакинг: искусство эксплойта. 2-е издание. — Пер. с англ. — СПб. : Символ-Плюс, 2018. — 512 с. — ISBN: 978-5-93286-158-5.
- [19] Касаткин А.И. Профессиональное программирование на языке Си. Управление ресурсами. — Минск : Высшая школа, 1992. — 432 с. — ISBN: 5-339-00808-8.
- [20] ГОСТ 8.417-2002. Государственная система обеспечения единства измерений. Единицы величин.
- [21] Лав Роберт. Ядро Linux. Описание процесса разработки. — М. : Вильямс, 2014. — 496 с. — ISBN: 978-5-8459-1944-1.
- [22] W. Dijkstra Edsger. Cooperating Sequential Processes // Programming Languages. — 1968. — С. 43–112.
- [23] PETERSON G. L. Myths about the Mutual Exclusion Problem // Information Processing Letters. — 1981. — Т. 12. — С. 115–116.
- [24] Lamport Leslie. A New Solution of Dijkstra's Concurrent Programming Problem // Commun. ACM. — 1974. — Vol. 17. — P. 453–455.
- [25] Борисков А.В., Харламов А.А., Марковский Н.Д. и др. Параллельные вычисления на GPU. Архитектура и программная модель CUDA: Учебное пособие. — М. : Издательство Московского университета, 2012. — 336 с. — ISBN: 978-5-8459-1944-1.
- [26] Баранов А.В., Лацис А.О., Сажин С.В., Храмцов М.Ю. Руководство пользователя системы МВС–100. — Режим доступа: <http://parallel.ru/mvs/user.html> (дата обращения: 01.02.2019).

- [27] Карпов Ю. Г. MODEL CHECKING: Верификация параллельных и распределенных программных систем. — СПб. : БХВ-Петербург, 2010. — 560 с. — ISBN: 978-5-9775-0404-1.
- [28] Кормен Томас Х, Лейзерсон Чарльз И., Ривест Рональд Л., Штайн Клиффорд. Алгоритмы: построение и анализ. — 2 изд. — М. : Вильямс, 2005. — 1296 с.
- [29] Данилов М.В. Методы планирования выполнения задач в системах реального времени // Программные продукты и системы. — 2001. — № 4. — С. 28–35.
- [30] Касперски Крисс. Техника оптимизации программ. Эффективное использование памяти. — СПб. : БХВ-Петербург, 2003. — 464 с. — ISBN: 5-94157-232-8.
- [31] Intel 64 and IA-32 Architectures Software Developer's Manual. — 2019. — Режим доступа: <http://software.intel.com/sites/default/files/managed/39/c5/325462-sdm-vol-1-2abcd-3abcd.pdf> (дата обращения: 20.02.2019).

Учебное издание

ВВЕДЕНИЕ В ОПЕРАЦИОННЫЕ СИСТЕМЫ
ЧАСТЬ I

Учебно-методическое пособие

Пугачев Андрей Васильевич
Дружинин Кирилл Юрьевич
Зязин Валентин Петрович

Иллюстрации – Богачева Д.Н., Мелков А.А.

Печатается в авторской редакции

Подписано в печать 2019. Формат 60×84 1/16.
Физ. печ. л. 7,5. Тираж 100 экз. Изд. № 12. Заказ №

МИРЭА – Российский технологический университет
119454, Москва, пр. Вернадского, д. 78